

730
Days

104
Weeks

6
Phases

4000+
Tasks

6
Milestone Projects

Mon	Tue	Wed	Thu	Fri	Sat	Sun
Learn	Learn	Practice	Build	Debug	Project	Review

"The distance between who you are and who you want to be
is what you do every single day."

Table of Contents

Phase 1 — Electronics Fundamentals + HTML/CSS + JS Basics

Week 1 · Electricity Basics / Internet & How Web Works
Week 2 · Ohm's Law & Calculations / HTML Structure & Semantics
Week 3 · Series & Parallel Circuits / HTML Forms & Tables
Week 4 · Breadboard & Component IDs / CSS Selectors & Box Model
Week 5 · LEDs & Resistors / CSS Flexbox Layout
Week 6 · Capacitors & Inductors / CSS Grid Layout
Week 7 · Diodes & Rectifiers / CSS Responsive Design
Week 8 · Voltage Dividers / CSS Animations & Transitions
Week 9 · Multimeter Usage / CSS Variables & Theming
Week 10 · LDR & Photoresistors / JavaScript Variables & Types
Week 11 · Transistors as Switches / JavaScript Functions & Scope
Week 12 · 555 Timer IC / JavaScript Arrays & Methods
Week 13 · Buffer Week: Phase 1 Review / Buffer Week: Catch Up & Refactor
Week 14 · Smart Night Light Circuit Build / Portfolio Website Build [MILESTONE]
Week 15 · Smart Night Light Finalise / Portfolio Deploy & Polish
Week 16 · Phase 1 Wrap-Up & Prep / Phase 1 Wrap-Up & Prep

Phase 2 — Arduino + JavaScript Deep Dive

Week 17 · Arduino IDE Setup & Blink / JavaScript Objects & OOP
Week 18 · Arduino Digital I/O / JavaScript DOM Manipulation
Week 19 · Arduino Analog Read/Write / JavaScript Events & Listeners
Week 20 · Arduino Serial Monitor / JavaScript Fetch API Basics
Week 21 · Arduino millis() Timing / Async/Await & Promises
Week 22 · DHT11 Temp/Humidity Sensor / JSON Parsing & API Calls
Week 23 · Ultrasonic Distance Sensor / LocalStorage & Session Storage
Week 24 · LCD 16x2 via I2C / JavaScript Error Handling
Week 25 · Servo Motor Control / ES6+ Modules & Bundling
Week 26 · IR Sensor & Remote / Vanilla JS Weather App
Week 27 · Multiple Sensors Combined / JavaScript Classes & Prototypes
Week 28 · Arduino EEPROM Storage / Regex & String Manipulation
Week 29 · Buffer Week: Phase 2 Review / Buffer Week: Catch Up & Refactor
Week 30 · Arduino Sensor Dashboard Build / Interactive JS Dashboard Build [MILESTONE]
Week 31 · Sensor Dashboard Finalise / Dashboard Polish & Deploy
Week 32 · Phase 2 Wrap-Up & Prep / Phase 2 Wrap-Up & Prep

Phase 3 — ESP32 + Node.js & Express

Week 33 · ESP32 Introduction & GPIO / Node.js Setup & Core Modules
Week 34 · ESP32 WiFi Connection / npm & package.json Mastery
Week 35 · ESP32 HTTP GET Requests / Express.js Setup & Routing
Week 36 · ESP32 HTTP POST JSON / Express Middleware & Body Parser
Week 37 · ESP32 REST Server Endpoints / REST API Design Principles
Week 38 · ESP32 WebSocket Client / WebSockets with Socket.io
Week 39 · ESP32 mDNS & OTA Updates / JWT Authentication in Express
Week 40 · ESP32 Deep Sleep Modes / Express Error Handling & Logging
Week 41 · ESP32 SPIFFS File System / File I/O in Node.js
Week 42 · ESP32 BLE Basics / Input Validation with Zod
Week 43 · ESP32 Multi-sensor Array / Rate Limiting & Security Headers
Week 44 · ESP32 MQTT Client Basics / Postman API Testing & Collections
Week 45 · Buffer Week: Phase 3 Review / Buffer Week: Catch Up & Refactor
Week 46 · Wireless IoT API System Build / Live Sensor API Deployment Build [MILESTONE]

Week 47 · IoT API System Finalise / API Deploy & Documentation

Week 48 · Phase 3 Wrap-Up & Prep / Phase 3 Wrap-Up & Prep

Phase 4 — Raspberry Pi + React

Week 49 · Raspberry Pi OS Setup / React Introduction & JSX

Week 50 · Pi GPIO with Python / React Components & Props

Week 51 · Pi Serial to Arduino Bridge / React useState Hook

Week 52 · Pi Camera Module / React useEffect Hook

Week 53 · Pi Flask REST API / React Router & Navigation

Week 54 · Pi systemd Services / React Context API

Week 55 · Pi cron Jobs & Scheduling / React Custom Hooks

Week 56 · Pi I2C Sensors via Python / React Forms & Controlled Inputs

Week 57 · Pi SSH Hardening / React useReducer Pattern

Week 58 · Pi Node.js + Express on Pi / React Recharts & Data Viz

Week 59 · Pi Socket.io Real-time / React Performance & Memoization

Week 60 · Pi OpenCV Camera Stream / React Testing with Jest & RTL

Week 61 · Buffer Week: Phase 4 Review / Buffer Week: Catch Up & Refactor

Week 62 · Pi Live Sensor Hub Build / Full React Dashboard App Build [MILESTONE]

Week 63 · Pi Sensor Hub Finalise / React Dashboard Polish & Deploy

Week 64 · Phase 4 Wrap-Up & Prep / Phase 4 Wrap-Up & Prep

Phase 5 — Docker + Databases + Deployment

Week 65 · PostgreSQL Schema Design / Docker Fundamentals & CLI

Week 66 · SQL CRUD Operations / Dockerfile & Image Building

Week 67 · SQL Joins & Aggregations / Docker Compose Multi-service

Week 68 · PostgreSQL Indexing / Docker Volumes & Networking

Week 69 · Prisma ORM Integration / Container Registry & Docker Hub

Week 70 · Redis Caching & Pub/Sub / CI/CD with GitHub Actions

Week 71 · Time-Series Data Queries / Automated Testing Pipeline

Week 72 · Database Migrations / Cloud VPS Setup (Railway/Render)

Week 73 · Connection Pooling / Nginx Reverse Proxy Config

Week 74 · MongoDB & Mongoose / HTTPS with Let's Encrypt

Week 75 · Database Backup & Restore / Environment Management & Secrets

Week 76 · Query Optimisation / Monitoring with Sentry/Uptime

Week 77 · Buffer Week: Phase 5 Review / Buffer Week: Catch Up & Refactor

Week 78 · Full DB + Docker Deploy Build / Complete Cloud Deploy Build [MILESTONE]

Week 79 · Cloud Deploy Finalise / Cloud Deploy Polish & Monitor

Week 80 · Phase 5 Wrap-Up & Prep / Phase 5 Wrap-Up & Prep

Phase 6 — MQTT + IoT Integration + Final Product

Week 81 · MQTT Protocol Deep Dive / TypeScript Fundamentals

Week 82 · Mosquitto Broker on Pi / TypeScript with Express

Week 83 · MQTT QoS Levels & Topics / Next.js Introduction & Routing

Week 84 · MQTT Retained Messages / Next.js SSR & SSG

Week 85 · MQTT Bridge & Federation / Next.js API Routes

Week 86 · Edge AI with TFLite / Stripe Payments Integration

Week 87 · PCB Design with KiCad / Next.js Auth with NextAuth.js

Week 88 · PCB Layout & DRC Check / SaaS Multi-tenant Architecture

Week 89 · Gerber Export & Review / WebRTC Basics for Video Streams

Week 90 · 3D Enclosure Design / System Design Fundamentals

Week 91 · Hardware Testing & QA / Load Testing with k6

Week 92 · Full IoT System Integration / Final App Polish & SEO

Week 93 · Buffer Week: Phase 6 Review / Buffer Week: Catch Up & Refactor

Week 94 · Final IoT Product Build Wk1 / SaaS Dashboard Build Wk1

Week 95 · Final IoT Product Build Wk2 / SaaS Dashboard Build Wk2

Week 96 · Final IoT Product Build Wk3 / SaaS Dashboard Build Wk3

Week 97 · Final IoT Product Testing / SaaS Dashboard Testing

Week 98 · Final IoT Product Polish / SaaS Dashboard Polish

Week 99 · Final IoT Product Docs / SaaS Dashboard Docs & API

Week 100 · Final IoT Product Launch / Production Deployment & Go-live [MILESTONE]

Week 101 · Portfolio Finalise Wk1 / Portfolio Finalise Wk1

Week 102 · Portfolio Finalise Wk2 / Portfolio Finalise Wk2

Week 103 · Career Prep & Job Apps Wk1 / Career Prep & Job Apps Wk1

Week 104 · Career Prep & Job Apps Wk2 / Career Prep — Day 730 Complete [MILESTONE]

Phase

1

Electronics Fundamentals + HTML/CSS + JS Basics

Months 1–4 · Weeks 1–16 · Days 1–112

Electronics: Ohm's Law, Circuits, LEDs, Capacitors, Diodes, Breadboard, 555 Timer

Full Stack: HTML5, CSS3, Flexbox, Grid, Responsive, Animations, JavaScript Basics

Milestone: Smart Night Light + Portfolio Website

Week 1

Electronics: Electricity Basics | Full Stack: Internet & How Web Works

Day 1	Monday	Learn Electronics	Electricity
-------	--------	-------------------	-------------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Electricity Basics'. Pause every 10 min and write key takeaways.

.

2 Read allaboutcircuits.com for 'Electricity Basics'. Copy 3 key formulas or facts into your notes.

.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-1/README.md with today's topic.

.

Output Notes page on 'Electricity Basics' + GitHub README stub created for Week 1.

:

Day 2	Tuesday	Learn Full Stack
-------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Internet & How Web Works'. Code along in VS Code — don't just watch.

.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Internet & How Web Works'. Run every code example locally.

.

3 Commit your notes and code samples to GitHub: 'Week 1 - Day 2 Learn: Internet & How Web Works'.

.

Output Working code examples for 'Internet & How Web Works' committed to GitHub.

:

Day 3	Wednesday	Practice Full Stack + Electronics	Electricity
-------	-----------	-----------------------------------	-------------

1 Write 3 short coding exercises for 'Internet & How Web Works' in VS Code. Each exercise tests one specific concept.

.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.

.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Electricity Basics' from memory.

.

Output 3 working code files for 'Internet & How Web Works' on GitHub. 'Electricity Basics' circuit sketched or simulated.

:

Day 4	Thursday	Build Electronics + Full Stack	Electricity
-------	----------	--------------------------------	-------------

1 Electronics (45 min): Wire the full 'Electricity Basics' circuit on your breadboard. Test with multimeter. Troubleshoot until working.

.

2 Full Stack (60 min): Build the 'Internet & How Web Works' feature inside your week's running project. Real, working code.

.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

.

Output Working 'Electricity Basics' circuit (photographed) + 'Internet & How Web Works' feature committed to GitHub.

:

Day 5	Friday	Debug Electronics + Full Stack	Electricity
-------	--------	--------------------------------	-------------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Electricity Basics' circuit. Fix each one.

.

Document them.

2

Full Stack (45 min): Review your 'Internet & How Web Works' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Electricity Basics' circuit verified. 'Internet & How Web Works' code with error handling on GitHub.

:

Day 6

Saturday

Project — Project Day

Electricity

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Electricity Basics' + 'Internet & How Web Works'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 1 project milestone'.

.

Output

Working project milestone: 'Electricity Basics' data visible via 'Internet & How Web Works' layer. Pushed to GitHub.

:

Day 7

Sunday

Review — Review Day

Electricity

1

Electronics (20 min): Re-draw your 'Electricity Basics' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Internet & How Web Works' out loud for 5 minutes (rubber duck method). Update your Week 1 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 1 code committed. README updated with learnings and next-week goals.

:

Week 2

Electronics: Ohm's Law & Calculations | Full Stack: HTML Structure & Semantics

Day 8	Monday	Learn Electronics	Ohm's
1	Watch 30-45 min: GreatScott or Andreas Spiess on 'Ohm's Law & Calculations'. Pause every 10 min and write key takeaways.		
.			
2	Read allaboutcircuits.com for 'Ohm's Law & Calculations'. Copy 3 key formulas or facts into your notes.		
.			
3	Open GitHub. Create/update your 'iot-roadmap' repo. Add week-2/README.md with today's topic.		
.			
Output	Notes page on 'Ohm's Law & Calculations' + GitHub README stub created for Week 2.		
:			
Day 9	Tuesday	Learn Full Stack	
1	Watch 30-45 min: Traversy Media or The Net Ninja on 'HTML Structure & Semantics'. Code along in VS Code — don't just watch.		
.			
2	Read the official docs (MDN / React Docs / Node.js Docs) for 'HTML Structure & Semantics'. Run every code example locally.		
.			
3	Commit your notes and code samples to GitHub: 'Week 2 - Day 2 Learn: HTML Structure & Semantics'.		
.			
Output	Working code examples for 'HTML Structure & Semantics' committed to GitHub.		
:			
Day 10	Wednesday	Practice Full Stack + Electronics	Ohm's
1	Write 3 short coding exercises for 'HTML Structure & Semantics' in VS Code. Each exercise tests one specific concept.		
.			
2	Add a comment to each exercise explaining what it does. Push all 3 to GitHub.		
.			
3	Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Ohm's Law & Calculations' from memory.		
.			
Output	3 working code files for 'HTML Structure & Semantics' on GitHub. 'Ohm's Law & Calculations' circuit sketched or simulated.		
:			
Day 11	Thursday	Build Electronics + Full Stack	Ohm's
1	Electronics (45 min): Wire the full 'Ohm's Law & Calculations' circuit on your breadboard. Test with multimeter. Troubleshoot until working.		
.			
2	Full Stack (60 min): Build the 'HTML Structure & Semantics' feature inside your week's running project. Real, working code.		
.			
3	Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.		
.			
Output	Working 'Ohm's Law & Calculations' circuit (photographed) + 'HTML Structure & Semantics' feature committed to GitHub.		
:			
Day 12	Friday	Debug Electronics + Full Stack	Ohm's
1	Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Ohm's Law & Calculations' circuit. Fix each one. Document them.		
.			

2

Full Stack (45 min): Review your 'HTML Structure & Semantics' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Ohm's Law & Calculations' circuit verified. 'HTML Structure & Semantics' code with error handling on GitHub.

:

Day 13

Saturday

Project — Project Day

Ohm's

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Ohm's Law & Calculations' + 'HTML Structure & Semantics'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 2 project milestone'.

.

Output

Working project milestone: 'Ohm's Law & Calculations' data visible via 'HTML Structure & Semantics' layer. Pushed to GitHub.

:

Day 14

Sunday

Review — Review Day

Ohm's

1

Electronics (20 min): Re-draw your 'Ohm's Law & Calculations' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'HTML Structure & Semantics' out loud for 5 minutes (rubber duck method). Update your Week 2 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 2 code committed. README updated with learnings and next-week goals.

:

Week 3

Electronics: Series & Parallel Circuits | Full Stack: HTML Forms & Tables

Day 15	Monday	Learn Electronics	Series
--------	--------	-------------------	--------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Series & Parallel Circuits'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Series & Parallel Circuits'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-3/README.md with today's topic.

Output Notes page on 'Series & Parallel Circuits' + GitHub README stub created for Week 3.

Day 16	Tuesday	Learn Full Stack
--------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'HTML Forms & Tables'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'HTML Forms & Tables'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 3 - Day 2 Learn: HTML Forms & Tables'.

Output Working code examples for 'HTML Forms & Tables' committed to GitHub.

Day 17	Wednesday	Practice Full Stack + Electronics	Series
--------	-----------	-----------------------------------	--------

- 1 Write 3 short coding exercises for 'HTML Forms & Tables' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Series & Parallel Circuits' from memory.

Output 3 working code files for 'HTML Forms & Tables' on GitHub. 'Series & Parallel Circuits' circuit sketched or simulated.

Day 18	Thursday	Build Electronics + Full Stack	Series
--------	----------	--------------------------------	--------

- 1 Electronics (45 min): Wire the full 'Series & Parallel Circuits' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'HTML Forms & Tables' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Series & Parallel Circuits' circuit (photographed) + 'HTML Forms & Tables' feature committed to GitHub.

Day 19	Friday	Debug Electronics + Full Stack	Series
--------	--------	--------------------------------	--------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Series & Parallel Circuits' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'HTML Forms & Tables' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Series & Parallel Circuits' circuit verified. 'HTML Forms & Tables' code with error handling on GitHub.

Day 20

Saturday

Project — Project Day

Series

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Series & Parallel Circuits' + 'HTML Forms & Tables'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 3 project milestone'.

.

Output

Working project milestone: 'Series & Parallel Circuits' data visible via 'HTML Forms & Tables' layer. Pushed to GitHub.

:

Day 21

Sunday

Review — Review Day

Series

1

Electronics (20 min): Re-draw your 'Series & Parallel Circuits' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'HTML Forms & Tables' out loud for 5 minutes (rubber duck method). Update your Week 3 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 3 code committed. README updated with learnings and next-week goals.

:

Week 4

Electronics: Breadboard & Component IDs | Full Stack: CSS Selectors & Box Model

Day 22 **Monday** **Learn Electronics** Breadboard

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Breadboard & Component IDs'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Breadboard & Component IDs'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-4/README.md with today's topic.

Output Notes page on 'Breadboard & Component IDs' + GitHub README stub created for Week 4.

Day 23 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'CSS Selectors & Box Model'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'CSS Selectors & Box Model'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 4 - Day 2 Learn: CSS Selectors & Box Model'.

Output Working code examples for 'CSS Selectors & Box Model' committed to GitHub.

Day 24 **Wednesday** **Practice Full Stack + Electronics** Breadboard

- 1 Write 3 short coding exercises for 'CSS Selectors & Box Model' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Breadboard & Component IDs' from memory.

Output 3 working code files for 'CSS Selectors & Box Model' on GitHub. 'Breadboard & Component IDs' circuit sketched or simulated.

Day 25 **Thursday** **Build Electronics + Full Stack** Breadboard

- 1 Electronics (45 min): Wire the full 'Breadboard & Component IDs' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'CSS Selectors & Box Model' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Breadboard & Component IDs' circuit (photographed) + 'CSS Selectors & Box Model' feature committed to GitHub.

Day 26 **Friday** **Debug Electronics + Full Stack** Breadboard

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Breadboard & Component IDs' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'CSS Selectors & Box Model' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output Bug log in README. 'Breadboard & Component IDs' circuit verified. 'CSS Selectors & Box Model' code with error handling on GitHub.

Day 27

Saturday

Project — Project Day

Breadboard

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Breadboard & Component IDs' + 'CSS Selectors & Box Model'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 4 project milestone'.

Output Working project milestone: 'Breadboard & Component IDs' data visible via 'CSS Selectors & Box Model' layer. Pushed to GitHub.

Day 28

Sunday

Review — Review Day

Breadboard

1 Electronics (20 min): Re-draw your 'Breadboard & Component IDs' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'CSS Selectors & Box Model' out loud for 5 minutes (rubber duck method). Update your Week 4 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output All Week 4 code committed. README updated with learnings and next-week goals.

Week 5

Electronics: LEDs & Resistors | Full Stack: CSS Flexbox Layout

Day 29	Monday	Learn Electronics	LEDs
--------	--------	-------------------	------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'LEDs & Resistors'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'LEDs & Resistors'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-5/README.md with today's topic.
- Output** Notes page on 'LEDs & Resistors' + GitHub README stub created for Week 5.

Day 30	Tuesday	Learn Full Stack
--------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'CSS Flexbox Layout'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'CSS Flexbox Layout'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 5 - Day 2 Learn: CSS Flexbox Layout'.
- Output** Working code examples for 'CSS Flexbox Layout' committed to GitHub.

Day 31	Wednesday	Practice Full Stack + Electronics	LEDs
--------	-----------	-----------------------------------	------

- 1 Write 3 short coding exercises for 'CSS Flexbox Layout' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'LEDs & Resistors' from memory.
- Output** 3 working code files for 'CSS Flexbox Layout' on GitHub. 'LEDs & Resistors' circuit sketched or simulated.

Day 32	Thursday	Build Electronics + Full Stack	LEDs
--------	----------	--------------------------------	------

- 1 Electronics (45 min): Wire the full 'LEDs & Resistors' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'CSS Flexbox Layout' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'LEDs & Resistors' circuit (photographed) + 'CSS Flexbox Layout' feature committed to GitHub.

Day 33	Friday	Debug Electronics + Full Stack	LEDs
--------	--------	--------------------------------	------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'LEDs & Resistors' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'CSS Flexbox Layout' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'LEDs & Resistors' circuit verified. 'CSS Flexbox Layout' code with error handling on GitHub.

:

Day 34

Saturday

Project — Project Day

LEDs

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'LEDs & Resistors' + 'CSS Flexbox Layout'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 5 project milestone'.

.

Output

Working project milestone: 'LEDs & Resistors' data visible via 'CSS Flexbox Layout' layer. Pushed to GitHub.

:

Day 35

Sunday

Review — Review Day

LEDs

1

Electronics (20 min): Re-draw your 'LEDs & Resistors' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'CSS Flexbox Layout' out loud for 5 minutes (rubber duck method). Update your Week 5 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 5 code committed. README updated with learnings and next-week goals.

:

Week 6

Electronics: Capacitors & Inductors | Full Stack: CSS Grid Layout

Day 36	Monday	Learn Electronics	Capacitors
--------	--------	-------------------	------------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Capacitors & Inductors'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Capacitors & Inductors'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-6/README.md with today's topic.

Output Notes page on 'Capacitors & Inductors' + GitHub README stub created for Week 6.

Day 37	Tuesday	Learn Full Stack
--------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'CSS Grid Layout'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'CSS Grid Layout'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 6 - Day 2 Learn: CSS Grid Layout'.

Output Working code examples for 'CSS Grid Layout' committed to GitHub.

Day 38	Wednesday	Practice Full Stack + Electronics	Capacitors
--------	-----------	-----------------------------------	------------

- 1 Write 3 short coding exercises for 'CSS Grid Layout' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Capacitors & Inductors' from memory.

Output 3 working code files for 'CSS Grid Layout' on GitHub. 'Capacitors & Inductors' circuit sketched or simulated.

Day 39	Thursday	Build Electronics + Full Stack	Capacitors
--------	----------	--------------------------------	------------

- 1 Electronics (45 min): Wire the full 'Capacitors & Inductors' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'CSS Grid Layout' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Capacitors & Inductors' circuit (photographed) + 'CSS Grid Layout' feature committed to GitHub.

Day 40	Friday	Debug Electronics + Full Stack	Capacitors
--------	--------	--------------------------------	------------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Capacitors & Inductors' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'CSS Grid Layout' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Capacitors & Inductors' circuit verified. 'CSS Grid Layout' code with error handling on GitHub.

:

Day 41

Saturday

Project — Project Day

Capacitors

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Capacitors & Inductors' + 'CSS Grid Layout'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 6 project milestone'.

.

Output

Working project milestone: 'Capacitors & Inductors' data visible via 'CSS Grid Layout' layer. Pushed to GitHub.

:

Day 42

Sunday

Review — Review Day

Capacitors

1

Electronics (20 min): Re-draw your 'Capacitors & Inductors' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'CSS Grid Layout' out loud for 5 minutes (rubber duck method). Update your Week 6 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 6 code committed. README updated with learnings and next-week goals.

:

Week 7

Electronics: Diodes & Rectifiers | Full Stack: CSS Responsive Design

Day 43	Monday	Learn Electronics	Diodes
--------	--------	-------------------	--------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Diodes & Rectifiers'. Pause every 10 min and write key takeaways.

.

2 Read allaboutcircuits.com for 'Diodes & Rectifiers'. Copy 3 key formulas or facts into your notes.

.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-7/README.md with today's topic.

.

Output Notes page on 'Diodes & Rectifiers' + GitHub README stub created for Week 7.

:

Day 44	Tuesday	Learn Full Stack
--------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'CSS Responsive Design'. Code along in VS Code — don't just watch.

.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'CSS Responsive Design'. Run every code example locally.

.

3 Commit your notes and code samples to GitHub: 'Week 7 - Day 2 Learn: CSS Responsive Design'.

.

Output Working code examples for 'CSS Responsive Design' committed to GitHub.

:

Day 45	Wednesday	Practice Full Stack + Electronics	Diodes
--------	-----------	-----------------------------------	--------

1 Write 3 short coding exercises for 'CSS Responsive Design' in VS Code. Each exercise tests one specific concept.

.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.

.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Diodes & Rectifiers' from memory.

.

Output 3 working code files for 'CSS Responsive Design' on GitHub. 'Diodes & Rectifiers' circuit sketched or simulated.

:

Day 46	Thursday	Build Electronics + Full Stack	Diodes
--------	----------	--------------------------------	--------

1 Electronics (45 min): Wire the full 'Diodes & Rectifiers' circuit on your breadboard. Test with multimeter. Troubleshoot until working.

.

2 Full Stack (60 min): Build the 'CSS Responsive Design' feature inside your week's running project. Real, working code.

.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

.

Output Working 'Diodes & Rectifiers' circuit (photographed) + 'CSS Responsive Design' feature committed to GitHub.

:

Day 47	Friday	Debug Electronics + Full Stack	Diodes
--------	--------	--------------------------------	--------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Diodes & Rectifiers' circuit. Fix each one.

.

Document them.

2

Full Stack (45 min): Review your 'CSS Responsive Design' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Diodes & Rectifiers' circuit verified. 'CSS Responsive Design' code with error handling on GitHub.

:

Day 48

Saturday

Project — Project Day

Diodes

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Diodes & Rectifiers' + 'CSS Responsive Design'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 7 project milestone'.

.

Output

Working project milestone: 'Diodes & Rectifiers' data visible via 'CSS Responsive Design' layer. Pushed to GitHub.

:

Day 49

Sunday

Review — Review Day

Diodes

1

Electronics (20 min): Re-draw your 'Diodes & Rectifiers' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'CSS Responsive Design' out loud for 5 minutes (rubber duck method). Update your Week 7 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 7 code committed. README updated with learnings and next-week goals.

:

Week 8

Electronics: Voltage Dividers | Full Stack: CSS Animations & Transitions

Day 50	Monday	Learn Electronics	Voltage
--------	--------	-------------------	---------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Voltage Dividers'. Pause every 10 min and write key takeaways.
.

2 Read allaboutcircuits.com for 'Voltage Dividers'. Copy 3 key formulas or facts into your notes.
.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-8/README.md with today's topic.
.

Output Notes page on 'Voltage Dividers' + GitHub README stub created for Week 8.

:

Day 51	Tuesday	Learn Full Stack
--------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'CSS Animations & Transitions'. Code along in VS Code — don't just watch.
.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'CSS Animations & Transitions'. Run every code example locally.
.

3 Commit your notes and code samples to GitHub: 'Week 8 - Day 2 Learn: CSS Animations & Transitions'.
.

Output Working code examples for 'CSS Animations & Transitions' committed to GitHub.

:

Day 52	Wednesday	Practice Full Stack + Electronics	Voltage
--------	-----------	-----------------------------------	---------

1 Write 3 short coding exercises for 'CSS Animations & Transitions' in VS Code. Each exercise tests one specific concept.
.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Voltage Dividers' from memory.
.

Output 3 working code files for 'CSS Animations & Transitions' on GitHub. 'Voltage Dividers' circuit sketched or simulated.

:

Day 53	Thursday	Build Electronics + Full Stack	Voltage
--------	----------	--------------------------------	---------

1 Electronics (45 min): Wire the full 'Voltage Dividers' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
.

2 Full Stack (60 min): Build the 'CSS Animations & Transitions' feature inside your week's running project. Real, working code.
.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
.

Output Working 'Voltage Dividers' circuit (photographed) + 'CSS Animations & Transitions' feature committed to GitHub.

:

Day 54	Friday	Debug Electronics + Full Stack	Voltage
--------	--------	--------------------------------	---------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Voltage Dividers' circuit. Fix each one.

. Document them.

2 Full Stack (45 min): Review your 'CSS Animations & Transitions' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output Bug log in README. 'Voltage Dividers' circuit verified. 'CSS Animations & Transitions' code with error handling on GitHub.

:

Day 55

Saturday

Project — Project Day

Voltage

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Voltage Dividers' + 'CSS Animations & Transitions'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 8 project milestone'.

.

Output Working project milestone: 'Voltage Dividers' data visible via 'CSS Animations & Transitions' layer. Pushed to GitHub.

:

Day 56

Sunday

Review — Review Day

Voltage

1 Electronics (20 min): Re-draw your 'Voltage Dividers' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'CSS Animations & Transitions' out loud for 5 minutes (rubber duck method). Update your Week 8 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output All Week 8 code committed. README updated with learnings and next-week goals.

:

Week 9

Electronics: Multimeter Usage | Full Stack: CSS Variables & Theming

Day 57	Monday	Learn Electronics	Multimeter
--------	--------	-------------------	------------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Multimeter Usage'. Pause every 10 min and write key takeaways.

.

2 Read allaboutcircuits.com for 'Multimeter Usage'. Copy 3 key formulas or facts into your notes.

.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-9/README.md with today's topic.

.

Output Notes page on 'Multimeter Usage' + GitHub README stub created for Week 9.

:

Day 58	Tuesday	Learn Full Stack
--------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'CSS Variables & Theming'. Code along in VS Code — don't just watch.

.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'CSS Variables & Theming'. Run every code example locally.

.

3 Commit your notes and code samples to GitHub: 'Week 9 - Day 2 Learn: CSS Variables & Theming'.

.

Output Working code examples for 'CSS Variables & Theming' committed to GitHub.

:

Day 59	Wednesday	Practice Full Stack + Electronics	Multimeter
--------	-----------	-----------------------------------	------------

1 Write 3 short coding exercises for 'CSS Variables & Theming' in VS Code. Each exercise tests one specific concept.

.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.

.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Multimeter Usage' from memory.

.

Output 3 working code files for 'CSS Variables & Theming' on GitHub. 'Multimeter Usage' circuit sketched or simulated.

:

Day 60	Thursday	Build Electronics + Full Stack	Multimeter
--------	----------	--------------------------------	------------

1 Electronics (45 min): Wire the full 'Multimeter Usage' circuit on your breadboard. Test with multimeter. Troubleshoot until working.

.

2 Full Stack (60 min): Build the 'CSS Variables & Theming' feature inside your week's running project. Real, working code.

.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

.

Output Working 'Multimeter Usage' circuit (photographed) + 'CSS Variables & Theming' feature committed to GitHub.

:

Day 61	Friday	Debug Electronics + Full Stack	Multimeter
--------	--------	--------------------------------	------------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Multimeter Usage' circuit. Fix each one.

.

Document them.

2

Full Stack (45 min): Review your 'CSS Variables & Theming' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Multimeter Usage' circuit verified. 'CSS Variables & Theming' code with error handling on GitHub.

:

Day 62

Saturday

Project — Project Day

Multimeter

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Multimeter Usage' + 'CSS Variables & Theming'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 9 project milestone'.

.

Output

Working project milestone: 'Multimeter Usage' data visible via 'CSS Variables & Theming' layer. Pushed to GitHub.

:

Day 63

Sunday

Review — Review Day

Multimeter

1

Electronics (20 min): Re-draw your 'Multimeter Usage' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'CSS Variables & Theming' out loud for 5 minutes (rubber duck method). Update your Week 9 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 9 code committed. README updated with learnings and next-week goals.

:

Week 10

Electronics: LDR & Photoresistors | Full Stack: JavaScript Variables & Types

Day 64	Monday	Learn Electronics	LDR
--------	--------	-------------------	-----

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'LDR & Photoresistors'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'LDR & Photoresistors'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-10/README.md with today's topic.

Output Notes page on 'LDR & Photoresistors' + GitHub README stub created for Week 10.

Day 65	Tuesday	Learn Full Stack
--------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'JavaScript Variables & Types'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'JavaScript Variables & Types'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 10 - Day 2 Learn: JavaScript Variables & Types'.

Output Working code examples for 'JavaScript Variables & Types' committed to GitHub.

Day 66	Wednesday	Practice Full Stack + Electronics	LDR
--------	-----------	-----------------------------------	-----

- 1 Write 3 short coding exercises for 'JavaScript Variables & Types' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'LDR & Photoresistors' from memory.

Output 3 working code files for 'JavaScript Variables & Types' on GitHub. 'LDR & Photoresistors' circuit sketched or simulated.

Day 67	Thursday	Build Electronics + Full Stack	LDR
--------	----------	--------------------------------	-----

- 1 Electronics (45 min): Wire the full 'LDR & Photoresistors' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'JavaScript Variables & Types' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'LDR & Photoresistors' circuit (photographed) + 'JavaScript Variables & Types' feature committed to GitHub.

Day 68	Friday	Debug Electronics + Full Stack	LDR
--------	--------	--------------------------------	-----

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'LDR & Photoresistors' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'JavaScript Variables & Types' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output Bug log in README. 'LDR & Photoresistors' circuit verified. 'JavaScript Variables & Types' code with error handling on GitHub.

Day 69**Saturday****Project — Project Day****LDR**

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'LDR & Photoresistors' + 'JavaScript Variables & Types'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 10 project milestone'.

Output Working project milestone: 'LDR & Photoresistors' data visible via 'JavaScript Variables & Types' layer. Pushed to GitHub.

Day 70**Sunday****Review — Review Day****LDR**

1 Electronics (20 min): Re-draw your 'LDR & Photoresistors' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'JavaScript Variables & Types' out loud for 5 minutes (rubber duck method). Update your Week 10 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output All Week 10 code committed. README updated with learnings and next-week goals.

Week 11

Electronics: Transistors as Switches | Full Stack: JavaScript Functions & Scope

Day 71	Monday	Learn Electronics	Transistors
--------	--------	-------------------	-------------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Transistors as Switches'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Transistors as Switches'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-11/README.md with today's topic.

Output Notes page on 'Transistors as Switches' + GitHub README stub created for Week 11.

Day 72	Tuesday	Learn Full Stack
--------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'JavaScript Functions & Scope'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'JavaScript Functions & Scope'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 11 - Day 2 Learn: JavaScript Functions & Scope'.

Output Working code examples for 'JavaScript Functions & Scope' committed to GitHub.

Day 73	Wednesday	Practice Full Stack + Electronics	Transistors
--------	-----------	-----------------------------------	-------------

- 1 Write 3 short coding exercises for 'JavaScript Functions & Scope' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Transistors as Switches' from memory.

Output 3 working code files for 'JavaScript Functions & Scope' on GitHub. 'Transistors as Switches' circuit sketched or simulated.

Day 74	Thursday	Build Electronics + Full Stack	Transistors
--------	----------	--------------------------------	-------------

- 1 Electronics (45 min): Wire the full 'Transistors as Switches' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'JavaScript Functions & Scope' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Transistors as Switches' circuit (photographed) + 'JavaScript Functions & Scope' feature committed to GitHub.

Day 75	Friday	Debug Electronics + Full Stack	Transistors
--------	--------	--------------------------------	-------------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Transistors as Switches' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'JavaScript Functions & Scope' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output Bug log in README. 'Transistors as Switches' circuit verified. 'JavaScript Functions & Scope' code with error handling on GitHub.

Day 76

Saturday

Project — Project Day

Transistors

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Transistors as Switches' + 'JavaScript Functions & Scope'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 11 project milestone'.

Output Working project milestone: 'Transistors as Switches' data visible via 'JavaScript Functions & Scope' layer. Pushed to GitHub.

Day 77

Sunday

Review — Review Day

Transistors

1 Electronics (20 min): Re-draw your 'Transistors as Switches' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'JavaScript Functions & Scope' out loud for 5 minutes (rubber duck method). Update your Week 11 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output All Week 11 code committed. README updated with learnings and next-week goals.

Week 12

Electronics: 555 Timer IC | Full Stack: JavaScript Arrays & Methods

Day 78	Monday	Learn Electronics	555
--------	--------	-------------------	-----

1 Watch 30-45 min: GreatScott or Andreas Spiess on '555 Timer IC'. Pause every 10 min and write key takeaways.

.

2 Read allaboutcircuits.com for '555 Timer IC'. Copy 3 key formulas or facts into your notes.

.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-12/README.md with today's topic.

.

Output Notes page on '555 Timer IC' + GitHub README stub created for Week 12.

:

Day 79	Tuesday	Learn Full Stack
--------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'JavaScript Arrays & Methods'. Code along in VS Code — don't just watch.

.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'JavaScript Arrays & Methods'. Run every code example locally.

.

3 Commit your notes and code samples to GitHub: 'Week 12 - Day 2 Learn: JavaScript Arrays & Methods'.

.

Output Working code examples for 'JavaScript Arrays & Methods' committed to GitHub.

:

Day 80	Wednesday	Practice Full Stack + Electronics	555
--------	-----------	-----------------------------------	-----

1 Write 3 short coding exercises for 'JavaScript Arrays & Methods' in VS Code. Each exercise tests one specific concept.

.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.

.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for '555 Timer IC' from memory.

.

Output 3 working code files for 'JavaScript Arrays & Methods' on GitHub. '555 Timer IC' circuit sketched or simulated.

:

Day 81	Thursday	Build Electronics + Full Stack	555
--------	----------	--------------------------------	-----

1 Electronics (45 min): Wire the full '555 Timer IC' circuit on your breadboard. Test with multimeter. Troubleshoot until working.

.

2 Full Stack (60 min): Build the 'JavaScript Arrays & Methods' feature inside your week's running project. Real, working code.

.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

.

Output Working '555 Timer IC' circuit (photographed) + 'JavaScript Arrays & Methods' feature committed to GitHub.

:

Day 82	Friday	Debug Electronics + Full Stack	555
--------	--------	--------------------------------	-----

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your '555 Timer IC' circuit. Fix each one. Document them.

.

2

Full Stack (45 min): Review your 'JavaScript Arrays & Methods' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. '555 Timer IC' circuit verified. 'JavaScript Arrays & Methods' code with error handling on GitHub.

:

Day 83

Saturday

Project — Project Day

555

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects '555 Timer IC' + 'JavaScript Arrays & Methods'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 12 project milestone'.

.

Output

Working project milestone: '555 Timer IC' data visible via 'JavaScript Arrays & Methods' layer. Pushed to GitHub.

:

Day 84

Sunday

Review — Review Day

555

1

Electronics (20 min): Re-draw your '555 Timer IC' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'JavaScript Arrays & Methods' out loud for 5 minutes (rubber duck method). Update your Week 12 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 12 code committed. README updated with learnings and next-week goals.

:

Week 13

Electronics: Buffer Week: Phase 1 Review | Full Stack: Buffer Week: Catch Up & Refactor

Day 85	Monday	Learn / Review	Buffer
--------	--------	----------------	--------

1 Review all electronics notes from the past 4 weeks. Re-draw 3 circuits from memory without looking at your notes.

.

2 Read through your GitHub commits for the past 4 weeks. Identify the 2 weakest areas.

.

3 Write a 'Mid-Phase Reflection' entry in your week README: what clicked, what is still unclear.

.

Output Electronics review complete. 2 weak areas identified and noted in README.

:

Day 86	Tuesday	Review / Refactor	Buffer
--------	---------	-------------------	--------

1 Review all Full Stack code from the past 4 weeks. Pick the project you are least proud of.

.

2 Refactor it: improve naming, add comments, fix any errors, add error handling.

.

3 Commit refactored code with message: 'refactor: buffer week improvements'.

.

Output Refactored code committed. README updated with improvement notes.

:

Day 87	Wednesday	Catch Up
--------	-----------	----------

1 Finish any incomplete mini-projects or exercises from the past 4 weeks.

.

2 Push all pending code to GitHub. Ensure every week folder has a complete README.

.

3 Run LeetCode: solve 3 problems you struggled with before. Time yourself.

.

Output All pending work committed. LeetCode practice done.

:

Day 88	Thursday	Deep Dive
--------	----------	-----------

1 Pick the hardest concept from the past 4 weeks. Spend the full session going deeper on it.

.

2 Build a tiny demo project that specifically exercises that concept. No tutorials.

.

3 Push the demo to GitHub with a clear README explaining what you explored.

.

Output Deep dive demo project committed to GitHub.

:

Day 89	Friday	System Design
--------	--------	---------------

1 Study system design: read one chapter of system-design-primer (GitHub, free) or watch one ByteByteGo video.

.

2 Draw the architecture diagram for your phase project. Label every component and connection.

.

3 Write the architecture explanation in your README as if explaining it to a junior developer.

.

Output Architecture diagram drawn and explained in README.

:

Day 90

Saturday

Phase Project Prep

Buffer

1 Plan the upcoming phase project in detail: list all components, features, and acceptance criteria.

.

2 Set up the project repository structure. Create all folders and stub files.

.

3 Order any missing hardware components. Write the hardware shopping list in README.

.

Output Phase project repo created with full plan in README. Hardware ordered.

:

Day 91

Sunday

Rest & Plan

1 Rest fully. No coding, no circuits. Sustainable learning requires recovery.

.

2 Optional: review next week's topics for 15 min. Bookmark 1-2 key resources.

.

3 Write 3 clear goals for the next 4 weeks at the bottom of your buffer README.

.

Output Well rested. 3 goals for next phase written. Resources bookmarked.

:

Week 14

Electronics: Smart Night Light Circuit Build | Full Stack: Portfolio Website Build

MILESTONE PROJECT: Smart Night Light + Portfolio Website

Day 92	Monday	Plan & Design	
1	Design the full project architecture: draw the hardware circuit and software stack on paper.		
.			
2	List all required components for 'Smart Night Light Circuit Build' and 'Portfolio Website Build'. Verify you have everything.		
.			
3	Set up the project GitHub repo. Create README with: project goal, tech stack, architecture diagram placeholder.		
.			
Output	Architecture designed. Project repo created. README outline committed.		
:			
Day 93	Tuesday	Build Core	Smart
1	Build the core hardware component: 'Smart Night Light Circuit Build'. Test it works in isolation with multimeter.		
.			
2	Set up the core software scaffold: 'Portfolio Website Build'. Create all files and folder structure.		
.			
3	Connect hardware and software at the most basic level. Verify data flows correctly.		
.			
Output	Core 'Smart Night Light Circuit Build' hardware working. 'Portfolio Website Build' scaffold committed to GitHub.		
:			
Day 94	Wednesday	Build Features	Smart
1	Implement the main feature of 'Smart Night Light Circuit Build'. Document every step in README.		
.			
2	Build the primary user-facing feature of 'Portfolio Website Build'. Write clean, commented code.		
.			
3	Integration check: hardware + software + web layer all communicating correctly.		
.			
Output	Main feature built and integrated. Code committed with documentation.		
:			
Day 95	Thursday	Build Full	Smart
1	Complete all remaining features for 'Smart Night Light Circuit Build'. Photograph every stage.		
.			
2	Complete all remaining features for 'Portfolio Website Build'. Add proper error handling.		
.			
3	End-to-end test: full pipeline from hardware input to web output. Fix all issues.		
.			
Output	Full project working end-to-end. All code committed to GitHub.		
:			
Day 96	Friday	Debug & Polish	Smart

- 1 Stress test the project: run it for 30 minutes continuously. Log any failures.
.
- 2 Fix all bugs found. Add input validation and error messages. Improve UX.
.
- 3 Code review: clean up all magic numbers, add comments, ensure consistent naming.
.

Output Project stable under load. All bugs fixed. Code cleaned and committed.

:

Day 97 Saturday Document & Demo

- 1 Write comprehensive README: project overview, setup guide, usage instructions, photos.
.
- 2 Record a 2-minute demo video showing the full working project.
.
- 3 Add the project to your portfolio page. Write a 100-word project description.
.

Output README complete. Demo video recorded. Portfolio updated.

:

Day 98 Sunday Launch & Reflect

- 1 Final commit with version tag v1.0.0. Push everything to GitHub.
.
- 2 Write a 200-word reflection: what you built, what you learned, what you'd do differently.
.
- 3 Share the GitHub link with someone (social media, forum, friend). External accountability matters.
.

Output Project v1.0.0 released. Reflection written. GitHub link shared.

:

Week 15

Electronics: Smart Night Light Finalise | Full Stack: Portfolio Deploy & Polish

Day 99	Monday	Learn Electronics	Smart
--------	--------	-------------------	-------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Smart Night Light Finalise'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'Smart Night Light Finalise'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-15/README.md with today's topic.
- Output** Notes page on 'Smart Night Light Finalise' + GitHub README stub created for Week 15.

Day 100	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Portfolio Deploy & Polish'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Portfolio Deploy & Polish'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 15 - Day 2 Learn: Portfolio Deploy & Polish'.
- Output** Working code examples for 'Portfolio Deploy & Polish' committed to GitHub.

Day 101	Wednesday	Practice Full Stack + Electronics	Smart
---------	-----------	-----------------------------------	-------

- 1 Write 3 short coding exercises for 'Portfolio Deploy & Polish' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Smart Night Light Finalise' from memory.
- Output** 3 working code files for 'Portfolio Deploy & Polish' on GitHub. 'Smart Night Light Finalise' circuit sketched or simulated.

Day 102	Thursday	Build Electronics + Full Stack	Smart
---------	----------	--------------------------------	-------

- 1 Electronics (45 min): Wire the full 'Smart Night Light Finalise' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'Portfolio Deploy & Polish' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'Smart Night Light Finalise' circuit (photographed) + 'Portfolio Deploy & Polish' feature committed to GitHub.

Day 103	Friday	Debug Electronics + Full Stack	Smart
---------	--------	--------------------------------	-------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Smart Night Light Finalise' circuit. Fix each one. Document them.

- 2 Full Stack (45 min): Review your 'Portfolio Deploy & Polish' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.
 - 3 Write a short bug log in your README: what broke, what caused it, how you fixed it.
- Output** Bug log in README. 'Smart Night Light Finalise' circuit verified. 'Portfolio Deploy & Polish' code with error handling on GitHub.

Day 104	Saturday	Project — Project Day	Smart
---------	----------	-----------------------	-------

- 1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Smart Night Light Finalise' + 'Portfolio Deploy & Polish'.
 - 2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).
 - 3 Test the full flow. Fix any issues. Push to GitHub: 'Week 15 project milestone'.
- Output** Working project milestone: 'Smart Night Light Finalise' data visible via 'Portfolio Deploy & Polish' layer. Pushed to GitHub.

Day 105	Sunday	Review — Review Day	Smart
---------	--------	---------------------	-------

- 1 Electronics (20 min): Re-draw your 'Smart Night Light Finalise' circuit from memory without references. Compare to the real circuit. Note what you missed.
 - 2 Full Stack (20 min): Explain 'Portfolio Deploy & Polish' out loud for 5 minutes (rubber duck method). Update your Week 15 README with a learning summary.
 - 3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.
- Output** All Week 15 code committed. README updated with learnings and next-week goals.

Week 16

Electronics: Phase 1 Wrap-Up & Prep | Full Stack: Phase 1 Wrap-Up & Prep

Day 106	Monday	Learn Electronics	Phase
---------	--------	-------------------	-------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Phase 1 Wrap-Up & Prep'. Pause every 10 min and write key takeaways.
.

2 Read allaboutcircuits.com for 'Phase 1 Wrap-Up & Prep'. Copy 3 key formulas or facts into your notes.
.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-16/README.md with today's topic.
.

Output Notes page on 'Phase 1 Wrap-Up & Prep' + GitHub README stub created for Week 16.
:

Day 107	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Phase 1 Wrap-Up & Prep'. Code along in VS Code — don't just watch.
.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Phase 1 Wrap-Up & Prep'. Run every code example locally.
.

3 Commit your notes and code samples to GitHub: 'Week 16 - Day 2 Learn: Phase 1 Wrap-Up & Prep'.
.

Output Working code examples for 'Phase 1 Wrap-Up & Prep' committed to GitHub.
:

Day 108	Wednesday	Practice Full Stack + Electronics	Phase
---------	-----------	-----------------------------------	-------

1 Write 3 short coding exercises for 'Phase 1 Wrap-Up & Prep' in VS Code. Each exercise tests one specific concept.
.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Phase 1 Wrap-Up & Prep' from memory.
.

Output 3 working code files for 'Phase 1 Wrap-Up & Prep' on GitHub. 'Phase 1 Wrap-Up & Prep' circuit sketched or simulated.
:

Day 109	Thursday	Build Electronics + Full Stack	Phase
---------	----------	--------------------------------	-------

1 Electronics (45 min): Wire the full 'Phase 1 Wrap-Up & Prep' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
.

2 Full Stack (60 min): Build the 'Phase 1 Wrap-Up & Prep' feature inside your week's running project. Real, working code.
.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
.

Output Working 'Phase 1 Wrap-Up & Prep' circuit (photographed) + 'Phase 1 Wrap-Up & Prep' feature committed to GitHub.
:

Day 110	Friday	Debug Electronics + Full Stack	Phase
---------	--------	--------------------------------	-------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Phase 1 Wrap-Up & Prep' circuit. Fix each one. Document them.

- 2 Full Stack (45 min): Review your 'Phase 1 Wrap-Up & Prep' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.
 - 3 Write a short bug log in your README: what broke, what caused it, how you fixed it.
- Output** Bug log in README. 'Phase 1 Wrap-Up & Prep' circuit verified. 'Phase 1 Wrap-Up & Prep' code with error handling on GitHub.

Day 111	Saturday	Project — Project Day	Phase
---------	----------	-----------------------	-------

- 1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Phase 1 Wrap-Up & Prep' + 'Phase 1 Wrap-Up & Prep'.
 - 2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).
 - 3 Test the full flow. Fix any issues. Push to GitHub: 'Week 16 project milestone'.
- Output** Working project milestone: 'Phase 1 Wrap-Up & Prep' data visible via 'Phase 1 Wrap-Up & Prep' layer. Pushed to GitHub.

Day 112	Sunday	Review — Review Day	Phase
---------	--------	---------------------	-------

- 1 Electronics (20 min): Re-draw your 'Phase 1 Wrap-Up & Prep' circuit from memory without references. Compare to the real circuit. Note what you missed.
 - 2 Full Stack (20 min): Explain 'Phase 1 Wrap-Up & Prep' out loud for 5 minutes (rubber duck method). Update your Week 16 README with a learning summary.
 - 3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.
- Output** All Week 16 code committed. README updated with learnings and next-week goals.

Phase

2

Arduino + JavaScript Deep Dive

Months 5–8 · Weeks 17–32 · Days 113–224

Electronics: Arduino IDE, Digital/Analog I/O, Sensors (DHT11, HC-SR04), LCD, Servo, IR

Full Stack: JS OOP, DOM, Fetch API, Async/Await, LocalStorage, ES6+, Chart.js

Milestone: Arduino Sensor Dashboard + Interactive JS Dashboard

Week 17

Electronics: Arduino IDE Setup & Blink | Full Stack: JavaScript Objects & OOP

Day 113	Monday	Learn Electronics	Arduino
---------	--------	-------------------	---------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Arduino IDE Setup & Blink'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'Arduino IDE Setup & Blink'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-17/README.md with today's topic.
- Output** Notes page on 'Arduino IDE Setup & Blink' + GitHub README stub created for Week 17.

Day 114	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'JavaScript Objects & OOP'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'JavaScript Objects & OOP'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 17 - Day 2 Learn: JavaScript Objects & OOP'.
- Output** Working code examples for 'JavaScript Objects & OOP' committed to GitHub.

Day 115	Wednesday	Practice Full Stack + Electronics	Arduino
---------	-----------	-----------------------------------	---------

- 1 Write 3 short coding exercises for 'JavaScript Objects & OOP' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Arduino IDE Setup & Blink' from memory.
- Output** 3 working code files for 'JavaScript Objects & OOP' on GitHub. 'Arduino IDE Setup & Blink' circuit sketched or simulated.

Day 116	Thursday	Build Electronics + Full Stack	Arduino
---------	----------	--------------------------------	---------

- 1 Electronics (45 min): Wire the full 'Arduino IDE Setup & Blink' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'JavaScript Objects & OOP' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'Arduino IDE Setup & Blink' circuit (photographed) + 'JavaScript Objects & OOP' feature committed to GitHub.

Day 117	Friday	Debug Electronics + Full Stack	Arduino
---------	--------	--------------------------------	---------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Arduino IDE Setup & Blink' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'JavaScript Objects & OOP' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Arduino IDE Setup & Blink' circuit verified. 'JavaScript Objects & OOP' code with error handling on GitHub.

Day 118

Saturday

Project — Project Day

Arduino

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Arduino IDE Setup & Blink' + 'JavaScript Objects & OOP'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 17 project milestone'.

.

Output

Working project milestone: 'Arduino IDE Setup & Blink' data visible via 'JavaScript Objects & OOP' layer. Pushed to GitHub.

Day 119

Sunday

Review — Review Day

Arduino

1

Electronics (20 min): Re-draw your 'Arduino IDE Setup & Blink' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'JavaScript Objects & OOP' out loud for 5 minutes (rubber duck method). Update your Week 17 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 17 code committed. README updated with learnings and next-week goals.

Week 18

Electronics: Arduino Digital I/O | Full Stack: JavaScript DOM Manipulation

Day 120	Monday	Learn Electronics	Arduino
---------	--------	-------------------	---------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Arduino Digital I/O'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Arduino Digital I/O'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-18/README.md with today's topic.

Output Notes page on 'Arduino Digital I/O' + GitHub README stub created for Week 18.

Day 121	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'JavaScript DOM Manipulation'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'JavaScript DOM Manipulation'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 18 - Day 2 Learn: JavaScript DOM Manipulation'.

Output Working code examples for 'JavaScript DOM Manipulation' committed to GitHub.

Day 122	Wednesday	Practice Full Stack + Electronics	Arduino
---------	-----------	-----------------------------------	---------

- 1 Write 3 short coding exercises for 'JavaScript DOM Manipulation' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Arduino Digital I/O' from memory.

Output 3 working code files for 'JavaScript DOM Manipulation' on GitHub. 'Arduino Digital I/O' circuit sketched or simulated.

Day 123	Thursday	Build Electronics + Full Stack	Arduino
---------	----------	--------------------------------	---------

- 1 Electronics (45 min): Wire the full 'Arduino Digital I/O' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'JavaScript DOM Manipulation' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Arduino Digital I/O' circuit (photographed) + 'JavaScript DOM Manipulation' feature committed to GitHub.

Day 124	Friday	Debug Electronics + Full Stack	Arduino
---------	--------	--------------------------------	---------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Arduino Digital I/O' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'JavaScript DOM Manipulation' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output Bug log in README. 'Arduino Digital I/O' circuit verified. 'JavaScript DOM Manipulation' code with error handling on GitHub.

Day 125 Saturday Project — Project Day

Arduino

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Arduino Digital I/O' + 'JavaScript DOM Manipulation'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 18 project milestone'.

Output Working project milestone: 'Arduino Digital I/O' data visible via 'JavaScript DOM Manipulation' layer. Pushed to GitHub.

Day 126 Sunday Review — Review Day

Arduino

1 Electronics (20 min): Re-draw your 'Arduino Digital I/O' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'JavaScript DOM Manipulation' out loud for 5 minutes (rubber duck method). Update your Week 18 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output All Week 18 code committed. README updated with learnings and next-week goals.

Week 19

Electronics: Arduino Analog Read/Write | Full Stack: JavaScript Events & Listeners

Day 127	Monday	Learn Electronics	Arduino
---------	--------	-------------------	---------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Arduino Analog Read/Write'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Arduino Analog Read/Write'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-19/README.md with today's topic.

Output Notes page on 'Arduino Analog Read/Write' + GitHub README stub created for Week 19.

Day 128	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'JavaScript Events & Listeners'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'JavaScript Events & Listeners'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 19 - Day 2 Learn: JavaScript Events & Listeners'.

Output Working code examples for 'JavaScript Events & Listeners' committed to GitHub.

Day 129	Wednesday	Practice Full Stack + Electronics	Arduino
---------	-----------	-----------------------------------	---------

- 1 Write 3 short coding exercises for 'JavaScript Events & Listeners' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Arduino Analog Read/Write' from memory.

Output 3 working code files for 'JavaScript Events & Listeners' on GitHub. 'Arduino Analog Read/Write' circuit sketched or simulated.

Day 130	Thursday	Build Electronics + Full Stack	Arduino
---------	----------	--------------------------------	---------

- 1 Electronics (45 min): Wire the full 'Arduino Analog Read/Write' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'JavaScript Events & Listeners' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Arduino Analog Read/Write' circuit (photographed) + 'JavaScript Events & Listeners' feature committed to GitHub.

Day 131	Friday	Debug Electronics + Full Stack	Arduino
---------	--------	--------------------------------	---------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Arduino Analog Read/Write' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'JavaScript Events & Listeners' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output Bug log in README. 'Arduino Analog Read/Write' circuit verified. 'JavaScript Events & Listeners' code with error handling on GitHub.

Day 132 Saturday Project — Project Day

Arduino

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Arduino Analog Read/Write' + 'JavaScript Events & Listeners'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 19 project milestone'.

Output Working project milestone: 'Arduino Analog Read/Write' data visible via 'JavaScript Events & Listeners' layer. Pushed to GitHub.

Day 133 Sunday Review — Review Day

Arduino

1 Electronics (20 min): Re-draw your 'Arduino Analog Read/Write' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'JavaScript Events & Listeners' out loud for 5 minutes (rubber duck method). Update your Week 19 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output All Week 19 code committed. README updated with learnings and next-week goals.

Week 20

Electronics: Arduino Serial Monitor | Full Stack: JavaScript Fetch API Basics

Day 134 **Monday** **Learn Electronics** Arduino

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Arduino Serial Monitor'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Arduino Serial Monitor'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-20/README.md with today's topic.

Output Notes page on 'Arduino Serial Monitor' + GitHub README stub created for Week 20.

Day 135 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'JavaScript Fetch API Basics'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'JavaScript Fetch API Basics'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 20 - Day 2 Learn: JavaScript Fetch API Basics'.

Output Working code examples for 'JavaScript Fetch API Basics' committed to GitHub.

Day 136 **Wednesday** **Practice Full Stack + Electronics** Arduino

- 1 Write 3 short coding exercises for 'JavaScript Fetch API Basics' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Arduino Serial Monitor' from memory.

Output 3 working code files for 'JavaScript Fetch API Basics' on GitHub. 'Arduino Serial Monitor' circuit sketched or simulated.

Day 137 **Thursday** **Build Electronics + Full Stack** Arduino

- 1 Electronics (45 min): Wire the full 'Arduino Serial Monitor' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'JavaScript Fetch API Basics' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Arduino Serial Monitor' circuit (photographed) + 'JavaScript Fetch API Basics' feature committed to GitHub.

Day 138 **Friday** **Debug Electronics + Full Stack** Arduino

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Arduino Serial Monitor' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'JavaScript Fetch API Basics' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output Bug log in README. 'Arduino Serial Monitor' circuit verified. 'JavaScript Fetch API Basics' code with error handling on GitHub.

Day 139 Saturday Project — Project Day

Arduino

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Arduino Serial Monitor' + 'JavaScript Fetch API Basics'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 20 project milestone'.

Output Working project milestone: 'Arduino Serial Monitor' data visible via 'JavaScript Fetch API Basics' layer. Pushed to GitHub.

Day 140 Sunday Review — Review Day

Arduino

1 Electronics (20 min): Re-draw your 'Arduino Serial Monitor' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'JavaScript Fetch API Basics' out loud for 5 minutes (rubber duck method). Update your Week 20 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output All Week 20 code committed. README updated with learnings and next-week goals.

Week 21

Electronics: Arduino millis() Timing | Full Stack: Async/Await & Promises

Day 141	Monday	Learn Electronics	Arduino
---------	--------	-------------------	---------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Arduino millis() Timing'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Arduino millis() Timing'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-21/README.md with today's topic.

Output Notes page on 'Arduino millis() Timing' + GitHub README stub created for Week 21.

Day 142	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Async/Await & Promises'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Async/Await & Promises'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 21 - Day 2 Learn: Async/Await & Promises'.

Output Working code examples for 'Async/Await & Promises' committed to GitHub.

Day 143	Wednesday	Practice Full Stack + Electronics	Arduino
---------	-----------	-----------------------------------	---------

- 1 Write 3 short coding exercises for 'Async/Await & Promises' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Arduino millis() Timing' from memory.

Output 3 working code files for 'Async/Await & Promises' on GitHub. 'Arduino millis() Timing' circuit sketched or simulated.

Day 144	Thursday	Build Electronics + Full Stack	Arduino
---------	----------	--------------------------------	---------

- 1 Electronics (45 min): Wire the full 'Arduino millis() Timing' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Async/Await & Promises' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Arduino millis() Timing' circuit (photographed) + 'Async/Await & Promises' feature committed to GitHub.

Day 145	Friday	Debug Electronics + Full Stack	Arduino
---------	--------	--------------------------------	---------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Arduino millis() Timing' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'Async/Await & Promises' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output Bug log in README. 'Arduino millis() Timing' circuit verified. 'Async/Await & Promises' code with error handling on GitHub.

Day 146 Saturday Project — Project Day

Arduino

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Arduino millis() Timing' + 'Async/Await & Promises'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 21 project milestone'.

Output Working project milestone: 'Arduino millis() Timing' data visible via 'Async/Await & Promises' layer. Pushed to GitHub.

Day 147 Sunday Review — Review Day

Arduino

1 Electronics (20 min): Re-draw your 'Arduino millis() Timing' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'Async/Await & Promises' out loud for 5 minutes (rubber duck method). Update your Week 21 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output All Week 21 code committed. README updated with learnings and next-week goals.

Week 22

Electronics: DHT11 Temp/Humidity Sensor | Full Stack: JSON Parsing & API Calls

Day 148 **Monday** **Learn Electronics** **DHT11**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'DHT11 Temp/Humidity Sensor'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'DHT11 Temp/Humidity Sensor'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-22/README.md with today's topic.

Output Notes page on 'DHT11 Temp/Humidity Sensor' + GitHub README stub created for Week 22.

Day 149 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'JSON Parsing & API Calls'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'JSON Parsing & API Calls'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 22 - Day 2 Learn: JSON Parsing & API Calls'.

Output Working code examples for 'JSON Parsing & API Calls' committed to GitHub.

Day 150 **Wednesday** **Practice Full Stack + Electronics** **DHT11**

- 1 Write 3 short coding exercises for 'JSON Parsing & API Calls' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'DHT11 Temp/Humidity Sensor' from memory.

Output 3 working code files for 'JSON Parsing & API Calls' on GitHub. 'DHT11 Temp/Humidity Sensor' circuit sketched or simulated.

Day 151 **Thursday** **Build Electronics + Full Stack** **DHT11**

- 1 Electronics (45 min): Wire the full 'DHT11 Temp/Humidity Sensor' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'JSON Parsing & API Calls' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'DHT11 Temp/Humidity Sensor' circuit (photographed) + 'JSON Parsing & API Calls' feature committed to GitHub.

Day 152 **Friday** **Debug Electronics + Full Stack** **DHT11**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'DHT11 Temp/Humidity Sensor' circuit. Fix each one. Document them.

- 2 Full Stack (45 min): Review your 'JSON Parsing & API Calls' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.
 - 3 Write a short bug log in your README: what broke, what caused it, how you fixed it.
- Output** Bug log in README. 'DHT11 Temp/Humidity Sensor' circuit verified. 'JSON Parsing & API Calls' code with error handling on GitHub.

Day 153	Saturday	Project — Project Day	DHT11
---------	----------	-----------------------	-------

- 1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'DHT11 Temp/Humidity Sensor' + 'JSON Parsing & API Calls'.
 - 2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).
 - 3 Test the full flow. Fix any issues. Push to GitHub: 'Week 22 project milestone'.
- Output** Working project milestone: 'DHT11 Temp/Humidity Sensor' data visible via 'JSON Parsing & API Calls' layer. Pushed to GitHub.

Day 154	Sunday	Review — Review Day	DHT11
---------	--------	---------------------	-------

- 1 Electronics (20 min): Re-draw your 'DHT11 Temp/Humidity Sensor' circuit from memory without references. Compare to the real circuit. Note what you missed.
 - 2 Full Stack (20 min): Explain 'JSON Parsing & API Calls' out loud for 5 minutes (rubber duck method). Update your Week 22 README with a learning summary.
 - 3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.
- Output** All Week 22 code committed. README updated with learnings and next-week goals.

Week 23

Electronics: Ultrasonic Distance Sensor | Full Stack: LocalStorage & Session Storage

Day 155	Monday	Learn Electronics	Ultrasonic
---------	--------	-------------------	------------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Ultrasonic Distance Sensor'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Ultrasonic Distance Sensor'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-23/README.md with today's topic.

Output Notes page on 'Ultrasonic Distance Sensor' + GitHub README stub created for Week 23.

Day 156	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'LocalStorage & Session Storage'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'LocalStorage & Session Storage'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 23 - Day 2 Learn: LocalStorage & Session Storage'.

Output Working code examples for 'LocalStorage & Session Storage' committed to GitHub.

Day 157	Wednesday	Practice Full Stack + Electronics	Ultrasonic
---------	-----------	-----------------------------------	------------

- 1 Write 3 short coding exercises for 'LocalStorage & Session Storage' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Ultrasonic Distance Sensor' from memory.

Output 3 working code files for 'LocalStorage & Session Storage' on GitHub. 'Ultrasonic Distance Sensor' circuit sketched or simulated.

Day 158	Thursday	Build Electronics + Full Stack	Ultrasonic
---------	----------	--------------------------------	------------

- 1 Electronics (45 min): Wire the full 'Ultrasonic Distance Sensor' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'LocalStorage & Session Storage' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Ultrasonic Distance Sensor' circuit (photographed) + 'LocalStorage & Session Storage' feature committed to GitHub.

Day 159	Friday	Debug Electronics + Full Stack	Ultrasonic
---------	--------	--------------------------------	------------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Ultrasonic Distance Sensor' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'LocalStorage & Session Storage' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Ultrasonic Distance Sensor' circuit verified. 'LocalStorage & Session Storage' code with error handling on GitHub.

:

Day 160

Saturday

Project — Project Day

Ultrasonic

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Ultrasonic Distance Sensor' + 'LocalStorage & Session Storage'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 23 project milestone'.

.

Output

Working project milestone: 'Ultrasonic Distance Sensor' data visible via 'LocalStorage & Session Storage' layer. Pushed to GitHub.

:

Day 161

Sunday

Review — Review Day

Ultrasonic

1

Electronics (20 min): Re-draw your 'Ultrasonic Distance Sensor' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'LocalStorage & Session Storage' out loud for 5 minutes (rubber duck method). Update your Week 23 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 23 code committed. README updated with learnings and next-week goals.

:

Week 24

Electronics: LCD 16x2 via I2C | Full Stack: JavaScript Error Handling

Day 162	Monday	Learn Electronics	LCD
---------	--------	-------------------	-----

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'LCD 16x2 via I2C'. Pause every 10 min and write key takeaways.

2 Read allaboutcircuits.com for 'LCD 16x2 via I2C'. Copy 3 key formulas or facts into your notes.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-24/README.md with today's topic.

Output Notes page on 'LCD 16x2 via I2C' + GitHub README stub created for Week 24.

Day 163	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'JavaScript Error Handling'. Code along in VS Code — don't just watch.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'JavaScript Error Handling'. Run every code example locally.

3 Commit your notes and code samples to GitHub: 'Week 24 - Day 2 Learn: JavaScript Error Handling'.

Output Working code examples for 'JavaScript Error Handling' committed to GitHub.

Day 164	Wednesday	Practice Full Stack + Electronics	LCD
---------	-----------	-----------------------------------	-----

1 Write 3 short coding exercises for 'JavaScript Error Handling' in VS Code. Each exercise tests one specific concept.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'LCD 16x2 via I2C' from memory.

Output 3 working code files for 'JavaScript Error Handling' on GitHub. 'LCD 16x2 via I2C' circuit sketched or simulated.

Day 165	Thursday	Build Electronics + Full Stack	LCD
---------	----------	--------------------------------	-----

1 Electronics (45 min): Wire the full 'LCD 16x2 via I2C' circuit on your breadboard. Test with multimeter. Troubleshoot until working.

2 Full Stack (60 min): Build the 'JavaScript Error Handling' feature inside your week's running project. Real, working code.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'LCD 16x2 via I2C' circuit (photographed) + 'JavaScript Error Handling' feature committed to GitHub.

Day 166	Friday	Debug Electronics + Full Stack	LCD
---------	--------	--------------------------------	-----

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'LCD 16x2 via I2C' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'JavaScript Error Handling' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'LCD 16x2 via I2C' circuit verified. 'JavaScript Error Handling' code with error handling on GitHub.

:

Day 167

Saturday

Project — Project Day

LCD

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'LCD 16x2 via I2C' + 'JavaScript Error Handling'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 24 project milestone'.

.

Output

Working project milestone: 'LCD 16x2 via I2C' data visible via 'JavaScript Error Handling' layer. Pushed to GitHub.

:

Day 168

Sunday

Review — Review Day

LCD

1

Electronics (20 min): Re-draw your 'LCD 16x2 via I2C' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'JavaScript Error Handling' out loud for 5 minutes (rubber duck method). Update your Week 24 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 24 code committed. README updated with learnings and next-week goals.

:

Week 25

Electronics: Servo Motor Control | Full Stack: ES6+ Modules & Bundling

Day 169 **Monday** **Learn Electronics** **Servo**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Servo Motor Control'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Servo Motor Control'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-25/README.md with today's topic.

Output Notes page on 'Servo Motor Control' + GitHub README stub created for Week 25.

Day 170 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'ES6+ Modules & Bundling'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'ES6+ Modules & Bundling'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 25 - Day 2 Learn: ES6+ Modules & Bundling'.

Output Working code examples for 'ES6+ Modules & Bundling' committed to GitHub.

Day 171 **Wednesday** **Practice Full Stack + Electronics** **Servo**

- 1 Write 3 short coding exercises for 'ES6+ Modules & Bundling' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Servo Motor Control' from memory.

Output 3 working code files for 'ES6+ Modules & Bundling' on GitHub. 'Servo Motor Control' circuit sketched or simulated.

Day 172 **Thursday** **Build Electronics + Full Stack** **Servo**

- 1 Electronics (45 min): Wire the full 'Servo Motor Control' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'ES6+ Modules & Bundling' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Servo Motor Control' circuit (photographed) + 'ES6+ Modules & Bundling' feature committed to GitHub.

Day 173 **Friday** **Debug Electronics + Full Stack** **Servo**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Servo Motor Control' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'ES6+ Modules & Bundling' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Servo Motor Control' circuit verified. 'ES6+ Modules & Bundling' code with error handling on GitHub.

:

Day 174

Saturday

Project — Project Day

Servo

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Servo Motor Control' + 'ES6+ Modules & Bundling'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 25 project milestone'.

.

Output

Working project milestone: 'Servo Motor Control' data visible via 'ES6+ Modules & Bundling' layer. Pushed to GitHub.

:

Day 175

Sunday

Review — Review Day

Servo

1

Electronics (20 min): Re-draw your 'Servo Motor Control' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'ES6+ Modules & Bundling' out loud for 5 minutes (rubber duck method). Update your Week 25 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 25 code committed. README updated with learnings and next-week goals.

:

Week 26

Electronics: IR Sensor & Remote | Full Stack: Vanilla JS Weather App

Day 176 **Monday** **Learn Electronics** **IR**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'IR Sensor & Remote'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'IR Sensor & Remote'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-26/README.md with today's topic.

Output Notes page on 'IR Sensor & Remote' + GitHub README stub created for Week 26.

Day 177 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Vanilla JS Weather App'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Vanilla JS Weather App'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 26 - Day 2 Learn: Vanilla JS Weather App'.

Output Working code examples for 'Vanilla JS Weather App' committed to GitHub.

Day 178 **Wednesday** **Practice Full Stack + Electronics** **IR**

- 1 Write 3 short coding exercises for 'Vanilla JS Weather App' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'IR Sensor & Remote' from memory.

Output 3 working code files for 'Vanilla JS Weather App' on GitHub. 'IR Sensor & Remote' circuit sketched or simulated.

Day 179 **Thursday** **Build Electronics + Full Stack** **IR**

- 1 Electronics (45 min): Wire the full 'IR Sensor & Remote' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Vanilla JS Weather App' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'IR Sensor & Remote' circuit (photographed) + 'Vanilla JS Weather App' feature committed to GitHub.

Day 180 **Friday** **Debug Electronics + Full Stack** **IR**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'IR Sensor & Remote' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Vanilla JS Weather App' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'IR Sensor & Remote' circuit verified. 'Vanilla JS Weather App' code with error handling on GitHub.

:

Day 181

Saturday

Project — Project Day

IR

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'IR Sensor & Remote' + 'Vanilla JS Weather App'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 26 project milestone'.

.

Output

Working project milestone: 'IR Sensor & Remote' data visible via 'Vanilla JS Weather App' layer. Pushed to GitHub.

:

Day 182

Sunday

Review — Review Day

IR

1

Electronics (20 min): Re-draw your 'IR Sensor & Remote' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Vanilla JS Weather App' out loud for 5 minutes (rubber duck method). Update your Week 26 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 26 code committed. README updated with learnings and next-week goals.

:

Week 27

Electronics: Multiple Sensors Combined | Full Stack: JavaScript Classes & Prototypes

Day 183	Monday	Learn Electronics	Multiple
---------	--------	-------------------	----------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Multiple Sensors Combined'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'Multiple Sensors Combined'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-27/README.md with today's topic.
- Output** Notes page on 'Multiple Sensors Combined' + GitHub README stub created for Week 27.

Day 184	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'JavaScript Classes & Prototypes'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'JavaScript Classes & Prototypes'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 27 - Day 2 Learn: JavaScript Classes & Prototypes'.
- Output** Working code examples for 'JavaScript Classes & Prototypes' committed to GitHub.

Day 185	Wednesday	Practice Full Stack + Electronics	Multiple
---------	-----------	-----------------------------------	----------

- 1 Write 3 short coding exercises for 'JavaScript Classes & Prototypes' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Multiple Sensors Combined' from memory.
- Output** 3 working code files for 'JavaScript Classes & Prototypes' on GitHub. 'Multiple Sensors Combined' circuit sketched or simulated.

Day 186	Thursday	Build Electronics + Full Stack	Multiple
---------	----------	--------------------------------	----------

- 1 Electronics (45 min): Wire the full 'Multiple Sensors Combined' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'JavaScript Classes & Prototypes' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'Multiple Sensors Combined' circuit (photographed) + 'JavaScript Classes & Prototypes' feature committed to GitHub.

Day 187	Friday	Debug Electronics + Full Stack	Multiple
---------	--------	--------------------------------	----------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Multiple Sensors Combined' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'JavaScript Classes & Prototypes' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Multiple Sensors Combined' circuit verified. 'JavaScript Classes & Prototypes' code with error handling on GitHub.

:

Day 188

Saturday

Project — Project Day

Multiple

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Multiple Sensors Combined' + 'JavaScript Classes & Prototypes'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 27 project milestone'.

.

Output

Working project milestone: 'Multiple Sensors Combined' data visible via 'JavaScript Classes & Prototypes' layer. Pushed to GitHub.

:

Day 189

Sunday

Review — Review Day

Multiple

1

Electronics (20 min): Re-draw your 'Multiple Sensors Combined' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'JavaScript Classes & Prototypes' out loud for 5 minutes (rubber duck method). Update your Week 27 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 27 code committed. README updated with learnings and next-week goals.

:

Week 28

Electronics: Arduino EEPROM Storage | Full Stack: Regex & String Manipulation

Day 190	Monday	Learn Electronics	Arduino
---------	--------	-------------------	---------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Arduino EEPROM Storage'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'Arduino EEPROM Storage'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-28/README.md with today's topic.
- Output** Notes page on 'Arduino EEPROM Storage' + GitHub README stub created for Week 28.

Day 191	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Regex & String Manipulation'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Regex & String Manipulation'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 28 - Day 2 Learn: Regex & String Manipulation'.
- Output** Working code examples for 'Regex & String Manipulation' committed to GitHub.

Day 192	Wednesday	Practice Full Stack + Electronics	Arduino
---------	-----------	-----------------------------------	---------

- 1 Write 3 short coding exercises for 'Regex & String Manipulation' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Arduino EEPROM Storage' from memory.
- Output** 3 working code files for 'Regex & String Manipulation' on GitHub. 'Arduino EEPROM Storage' circuit sketched or simulated.

Day 193	Thursday	Build Electronics + Full Stack	Arduino
---------	----------	--------------------------------	---------

- 1 Electronics (45 min): Wire the full 'Arduino EEPROM Storage' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'Regex & String Manipulation' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'Arduino EEPROM Storage' circuit (photographed) + 'Regex & String Manipulation' feature committed to GitHub.

Day 194	Friday	Debug Electronics + Full Stack	Arduino
---------	--------	--------------------------------	---------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Arduino EEPROM Storage' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Regex & String Manipulation' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output

Bug log in README. 'Arduino EEPROM Storage' circuit verified. 'Regex & String Manipulation' code with error handling on GitHub.

Day 195

Saturday

Project — Project Day

Arduino

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Arduino EEPROM Storage' + 'Regex & String Manipulation'.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 28 project milestone'.

Output

Working project milestone: 'Arduino EEPROM Storage' data visible via 'Regex & String Manipulation' layer. Pushed to GitHub.

Day 196

Sunday

Review — Review Day

Arduino

1

Electronics (20 min): Re-draw your 'Arduino EEPROM Storage' circuit from memory without references. Compare to the real circuit. Note what you missed.

2

Full Stack (20 min): Explain 'Regex & String Manipulation' out loud for 5 minutes (rubber duck method). Update your Week 28 README with a learning summary.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output

All Week 28 code committed. README updated with learnings and next-week goals.

Week 29

Electronics: Buffer Week: Phase 2 Review | Full Stack: Buffer Week: Catch Up & Refactor

Day 197 Monday Learn / Review Buffer

1 Review all electronics notes from the past 4 weeks. Re-draw 3 circuits from memory without looking at your notes.
.

2 Read through your GitHub commits for the past 4 weeks. Identify the 2 weakest areas.
.

3 Write a 'Mid-Phase Reflection' entry in your week README: what clicked, what is still unclear.
.

Output Electronics review complete. 2 weak areas identified and noted in README.

:

Day 198 Tuesday Review / Refactor Buffer

1 Review all Full Stack code from the past 4 weeks. Pick the project you are least proud of.
.

2 Refactor it: improve naming, add comments, fix any errors, add error handling.
.

3 Commit refactored code with message: 'refactor: buffer week improvements'.
.

Output Refactored code committed. README updated with improvement notes.

:

Day 199 Wednesday Catch Up

1 Finish any incomplete mini-projects or exercises from the past 4 weeks.
.

2 Push all pending code to GitHub. Ensure every week folder has a complete README.
.

3 Run LeetCode: solve 3 problems you struggled with before. Time yourself.
.

Output All pending work committed. LeetCode practice done.

:

Day 200 Thursday Deep Dive

1 Pick the hardest concept from the past 4 weeks. Spend the full session going deeper on it.
.

2 Build a tiny demo project that specifically exercises that concept. No tutorials.
.

3 Push the demo to GitHub with a clear README explaining what you explored.
.

Output Deep dive demo project committed to GitHub.

:

Day 201 Friday System Design

1 Study system design: read one chapter of system-design-primer (GitHub, free) or watch one ByteByteGo video.
.

2 Draw the architecture diagram for your phase project. Label every component and connection.

.

3 Write the architecture explanation in your README as if explaining it to a junior developer.

.

Output Architecture diagram drawn and explained in README.

:

Day 202

Saturday

Phase Project Prep

Buffer

1 Plan the upcoming phase project in detail: list all components, features, and acceptance criteria.

.

2 Set up the project repository structure. Create all folders and stub files.

.

3 Order any missing hardware components. Write the hardware shopping list in README.

.

Output Phase project repo created with full plan in README. Hardware ordered.

:

Day 203

Sunday

Rest & Plan

1 Rest fully. No coding, no circuits. Sustainable learning requires recovery.

.

2 Optional: review next week's topics for 15 min. Bookmark 1-2 key resources.

.

3 Write 3 clear goals for the next 4 weeks at the bottom of your buffer README.

.

Output Well rested. 3 goals for next phase written. Resources bookmarked.

:

Week 30

Electronics: Arduino Sensor Dashboard Build | Full Stack: Interactive JS Dashboard Build

MILESTONE PROJECT: Arduino Sensor Dashboard + Interactive JS Dashboard

Day 204	Monday	Plan & Design	
1	Design the full project architecture: draw the hardware circuit and software stack on paper.		
.			
2	List all required components for 'Arduino Sensor Dashboard Build' and 'Interactive JS Dashboard Build'. Verify you have everything.		
.			
3	Set up the project GitHub repo. Create README with: project goal, tech stack, architecture diagram placeholder.		
.			
Output	Architecture designed. Project repo created. README outline committed.		
:			
Day 205	Tuesday	Build Core	Arduino
1	Build the core hardware component: 'Arduino Sensor Dashboard Build'. Test it works in isolation with multimeter.		
.			
2	Set up the core software scaffold: 'Interactive JS Dashboard Build'. Create all files and folder structure.		
.			
3	Connect hardware and software at the most basic level. Verify data flows correctly.		
.			
Output	Core 'Arduino Sensor Dashboard Build' hardware working. 'Interactive JS Dashboard Build' scaffold committed to GitHub.		
:			
Day 206	Wednesday	Build Features	Arduino
1	Implement the main feature of 'Arduino Sensor Dashboard Build'. Document every step in README.		
.			
2	Build the primary user-facing feature of 'Interactive JS Dashboard Build'. Write clean, commented code.		
.			
3	Integration check: hardware + software + web layer all communicating correctly.		
.			
Output	Main feature built and integrated. Code committed with documentation.		
:			
Day 207	Thursday	Build Full	Arduino
1	Complete all remaining features for 'Arduino Sensor Dashboard Build'. Photograph every stage.		
.			
2	Complete all remaining features for 'Interactive JS Dashboard Build'. Add proper error handling.		
.			
3	End-to-end test: full pipeline from hardware input to web output. Fix all issues.		
.			
Output	Full project working end-to-end. All code committed to GitHub.		
:			
Day 208	Friday	Debug & Polish	Arduino

- 1 Stress test the project: run it for 30 minutes continuously. Log any failures.
.
- 2 Fix all bugs found. Add input validation and error messages. Improve UX.
.
- 3 Code review: clean up all magic numbers, add comments, ensure consistent naming.
.

Output Project stable under load. All bugs fixed. Code cleaned and committed.

:

Day 209 Saturday Document & Demo

- 1 Write comprehensive README: project overview, setup guide, usage instructions, photos.
.
- 2 Record a 2-minute demo video showing the full working project.
.
- 3 Add the project to your portfolio page. Write a 100-word project description.
.

Output README complete. Demo video recorded. Portfolio updated.

:

Day 210 Sunday Launch & Reflect

- 1 Final commit with version tag v1.0.0. Push everything to GitHub.
.
- 2 Write a 200-word reflection: what you built, what you learned, what you'd do differently.
.
- 3 Share the GitHub link with someone (social media, forum, friend). External accountability matters.
.

Output Project v1.0.0 released. Reflection written. GitHub link shared.

:

Week 31

Electronics: Sensor Dashboard Finalise | Full Stack: Dashboard Polish & Deploy

Day 211	Monday	Learn Electronics	Sensor
---------	--------	-------------------	--------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Sensor Dashboard Finalise'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Sensor Dashboard Finalise'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-31/README.md with today's topic.

Output Notes page on 'Sensor Dashboard Finalise' + GitHub README stub created for Week 31.

Day 212	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Dashboard Polish & Deploy'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Dashboard Polish & Deploy'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 31 - Day 2 Learn: Dashboard Polish & Deploy'.

Output Working code examples for 'Dashboard Polish & Deploy' committed to GitHub.

Day 213	Wednesday	Practice Full Stack + Electronics	Sensor
---------	-----------	-----------------------------------	--------

- 1 Write 3 short coding exercises for 'Dashboard Polish & Deploy' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Sensor Dashboard Finalise' from memory.

Output 3 working code files for 'Dashboard Polish & Deploy' on GitHub. 'Sensor Dashboard Finalise' circuit sketched or simulated.

Day 214	Thursday	Build Electronics + Full Stack	Sensor
---------	----------	--------------------------------	--------

- 1 Electronics (45 min): Wire the full 'Sensor Dashboard Finalise' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Dashboard Polish & Deploy' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Sensor Dashboard Finalise' circuit (photographed) + 'Dashboard Polish & Deploy' feature committed to GitHub.

Day 215	Friday	Debug Electronics + Full Stack	Sensor
---------	--------	--------------------------------	--------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Sensor Dashboard Finalise' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'Dashboard Polish & Deploy' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output Bug log in README. 'Sensor Dashboard Finalise' circuit verified. 'Dashboard Polish & Deploy' code with error handling on : GitHub.

Day 216

Saturday

Project — Project Day

Sensor

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Sensor Dashboard Finalise' + 'Dashboard Polish & Deploy'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 31 project milestone'.

Output Working project milestone: 'Sensor Dashboard Finalise' data visible via 'Dashboard Polish & Deploy' layer. Pushed to : GitHub.

Day 217

Sunday

Review — Review Day

Sensor

1 Electronics (20 min): Re-draw your 'Sensor Dashboard Finalise' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'Dashboard Polish & Deploy' out loud for 5 minutes (rubber duck method). Update your Week 31 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output All Week 31 code committed. README updated with learnings and next-week goals. :

Week 32

Electronics: Phase 2 Wrap-Up & Prep | Full Stack: Phase 2 Wrap-Up & Prep

Day 218	Monday	Learn Electronics	Phase
---------	--------	-------------------	-------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Phase 2 Wrap-Up & Prep'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'Phase 2 Wrap-Up & Prep'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-32/README.md with today's topic.
- Output** Notes page on 'Phase 2 Wrap-Up & Prep' + GitHub README stub created for Week 32.

Day 219	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Phase 2 Wrap-Up & Prep'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Phase 2 Wrap-Up & Prep'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 32 - Day 2 Learn: Phase 2 Wrap-Up & Prep'.
- Output** Working code examples for 'Phase 2 Wrap-Up & Prep' committed to GitHub.

Day 220	Wednesday	Practice Full Stack + Electronics	Phase
---------	-----------	-----------------------------------	-------

- 1 Write 3 short coding exercises for 'Phase 2 Wrap-Up & Prep' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Phase 2 Wrap-Up & Prep' from memory.
- Output** 3 working code files for 'Phase 2 Wrap-Up & Prep' on GitHub. 'Phase 2 Wrap-Up & Prep' circuit sketched or simulated.

Day 221	Thursday	Build Electronics + Full Stack	Phase
---------	----------	--------------------------------	-------

- 1 Electronics (45 min): Wire the full 'Phase 2 Wrap-Up & Prep' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'Phase 2 Wrap-Up & Prep' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'Phase 2 Wrap-Up & Prep' circuit (photographed) + 'Phase 2 Wrap-Up & Prep' feature committed to GitHub.

Day 222	Friday	Debug Electronics + Full Stack	Phase
---------	--------	--------------------------------	-------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Phase 2 Wrap-Up & Prep' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Phase 2 Wrap-Up & Prep' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Phase 2 Wrap-Up & Prep' circuit verified. 'Phase 2 Wrap-Up & Prep' code with error handling on GitHub.

:

Day 223

Saturday

Project — Project Day

Phase

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Phase 2 Wrap-Up & Prep' + 'Phase 2 Wrap-Up & Prep'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 32 project milestone'.

.

Output

Working project milestone: 'Phase 2 Wrap-Up & Prep' data visible via 'Phase 2 Wrap-Up & Prep' layer. Pushed to GitHub.

:

Day 224

Sunday

Review — Review Day

Phase

1

Electronics (20 min): Re-draw your 'Phase 2 Wrap-Up & Prep' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Phase 2 Wrap-Up & Prep' out loud for 5 minutes (rubber duck method). Update your Week 32 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 32 code committed. README updated with learnings and next-week goals.

:

Phase

3

ESP32 + Node.js & Express

Months 9–12 · Weeks 33–48 · Days 225–336

Electronics: ESP32 GPIO, WiFi, HTTP, WebSocket, OTA, Deep Sleep, SPIFFS, BLE, MQTT

Full Stack: Node.js, Express, REST API, JWT Auth, Socket.io, Zod, Rate Limiting, Postman

Milestone: Wireless IoT API Platform + Live Sensor API Deployment

Week 33

Electronics: ESP32 Introduction & GPIO | Full Stack: Node.js Setup & Core Modules

Day 225 **Monday** **Learn Electronics** **ESP32**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'ESP32 Introduction & GPIO'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'ESP32 Introduction & GPIO'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-33/README.md with today's topic.
- Output** Notes page on 'ESP32 Introduction & GPIO' + GitHub README stub created for Week 33.

Day 226 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Node.js Setup & Core Modules'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Node.js Setup & Core Modules'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 33 - Day 2 Learn: Node.js Setup & Core Modules'.
- Output** Working code examples for 'Node.js Setup & Core Modules' committed to GitHub.

Day 227 **Wednesday** **Practice Full Stack + Electronics** **ESP32**

- 1 Write 3 short coding exercises for 'Node.js Setup & Core Modules' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'ESP32 Introduction & GPIO' from memory.
- Output** 3 working code files for 'Node.js Setup & Core Modules' on GitHub. 'ESP32 Introduction & GPIO' circuit sketched or simulated.

Day 228 **Thursday** **Build Electronics + Full Stack** **ESP32**

- 1 Electronics (45 min): Wire the full 'ESP32 Introduction & GPIO' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'Node.js Setup & Core Modules' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'ESP32 Introduction & GPIO' circuit (photographed) + 'Node.js Setup & Core Modules' feature committed to GitHub.

Day 229 **Friday** **Debug Electronics + Full Stack** **ESP32**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'ESP32 Introduction & GPIO' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Node.js Setup & Core Modules' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'ESP32 Introduction & GPIO' circuit verified. 'Node.js Setup & Core Modules' code with error handling on GitHub.

:

Day 230

Saturday

Project — Project Day

ESP32

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'ESP32 Introduction & GPIO' + 'Node.js Setup & Core Modules'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 33 project milestone'.

.

Output

Working project milestone: 'ESP32 Introduction & GPIO' data visible via 'Node.js Setup & Core Modules' layer. Pushed to GitHub.

:

Day 231

Sunday

Review — Review Day

ESP32

1

Electronics (20 min): Re-draw your 'ESP32 Introduction & GPIO' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Node.js Setup & Core Modules' out loud for 5 minutes (rubber duck method). Update your Week 33 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 33 code committed. README updated with learnings and next-week goals.

:

Week 34

Electronics: ESP32 WiFi Connection | Full Stack: npm & package.json Mastery

Day 232	Monday	Learn Electronics	ESP32
---------	--------	-------------------	-------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'ESP32 WiFi Connection'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'ESP32 WiFi Connection'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-34/README.md with today's topic.

Output Notes page on 'ESP32 WiFi Connection' + GitHub README stub created for Week 34.

Day 233	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'npm & package.json Mastery'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'npm & package.json Mastery'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 34 - Day 2 Learn: npm & package.json Mastery'.

Output Working code examples for 'npm & package.json Mastery' committed to GitHub.

Day 234	Wednesday	Practice Full Stack + Electronics	ESP32
---------	-----------	-----------------------------------	-------

- 1 Write 3 short coding exercises for 'npm & package.json Mastery' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'ESP32 WiFi Connection' from memory.

Output 3 working code files for 'npm & package.json Mastery' on GitHub. 'ESP32 WiFi Connection' circuit sketched or simulated.

Day 235	Thursday	Build Electronics + Full Stack	ESP32
---------	----------	--------------------------------	-------

- 1 Electronics (45 min): Wire the full 'ESP32 WiFi Connection' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'npm & package.json Mastery' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'ESP32 WiFi Connection' circuit (photographed) + 'npm & package.json Mastery' feature committed to GitHub.

Day 236	Friday	Debug Electronics + Full Stack	ESP32
---------	--------	--------------------------------	-------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'ESP32 WiFi Connection' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'npm & package.json Mastery' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'ESP32 WiFi Connection' circuit verified. 'npm & package.json Mastery' code with error handling on GitHub.

Day 237

Saturday

Project — Project Day

ESP32

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'ESP32 WiFi Connection' + 'npm & package.json Mastery'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 34 project milestone'.

.

Output

Working project milestone: 'ESP32 WiFi Connection' data visible via 'npm & package.json Mastery' layer. Pushed to GitHub.

Day 238

Sunday

Review — Review Day

ESP32

1

Electronics (20 min): Re-draw your 'ESP32 WiFi Connection' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'npm & package.json Mastery' out loud for 5 minutes (rubber duck method). Update your Week 34 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 34 code committed. README updated with learnings and next-week goals.

Week 35

Electronics: ESP32 HTTP GET Requests | Full Stack: Express.js Setup & Routing

Day 239 **Monday** **Learn Electronics** **ESP32**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'ESP32 HTTP GET Requests'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'ESP32 HTTP GET Requests'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-35/README.md with today's topic.

Output Notes page on 'ESP32 HTTP GET Requests' + GitHub README stub created for Week 35.

Day 240 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Express.js Setup & Routing'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Express.js Setup & Routing'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 35 - Day 2 Learn: Express.js Setup & Routing'.

Output Working code examples for 'Express.js Setup & Routing' committed to GitHub.

Day 241 **Wednesday** **Practice Full Stack + Electronics** **ESP32**

- 1 Write 3 short coding exercises for 'Express.js Setup & Routing' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'ESP32 HTTP GET Requests' from memory.

Output 3 working code files for 'Express.js Setup & Routing' on GitHub. 'ESP32 HTTP GET Requests' circuit sketched or simulated.

Day 242 **Thursday** **Build Electronics + Full Stack** **ESP32**

- 1 Electronics (45 min): Wire the full 'ESP32 HTTP GET Requests' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Express.js Setup & Routing' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'ESP32 HTTP GET Requests' circuit (photographed) + 'Express.js Setup & Routing' feature committed to GitHub.

Day 243 **Friday** **Debug Electronics + Full Stack** **ESP32**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'ESP32 HTTP GET Requests' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Express.js Setup & Routing' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'ESP32 HTTP GET Requests' circuit verified. 'Express.js Setup & Routing' code with error handling on GitHub.

:

Day 244

Saturday

Project — Project Day

ESP32

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'ESP32 HTTP GET Requests' + 'Express.js Setup & Routing'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 35 project milestone'.

.

Output

Working project milestone: 'ESP32 HTTP GET Requests' data visible via 'Express.js Setup & Routing' layer. Pushed to GitHub.

:

Day 245

Sunday

Review — Review Day

ESP32

1

Electronics (20 min): Re-draw your 'ESP32 HTTP GET Requests' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Express.js Setup & Routing' out loud for 5 minutes (rubber duck method). Update your Week 35 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 35 code committed. README updated with learnings and next-week goals.

:

Week 36

Electronics: ESP32 HTTP POST JSON | Full Stack: Express Middleware & Body Parser

Day 246 **Monday** **Learn Electronics** **ESP32**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'ESP32 HTTP POST JSON'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'ESP32 HTTP POST JSON'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-36/README.md with today's topic.

Output Notes page on 'ESP32 HTTP POST JSON' + GitHub README stub created for Week 36.

Day 247 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Express Middleware & Body Parser'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Express Middleware & Body Parser'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 36 - Day 2 Learn: Express Middleware & Body Parser'.

Output Working code examples for 'Express Middleware & Body Parser' committed to GitHub.

Day 248 **Wednesday** **Practice Full Stack + Electronics** **ESP32**

- 1 Write 3 short coding exercises for 'Express Middleware & Body Parser' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'ESP32 HTTP POST JSON' from memory.

Output 3 working code files for 'Express Middleware & Body Parser' on GitHub. 'ESP32 HTTP POST JSON' circuit sketched or simulated.

Day 249 **Thursday** **Build Electronics + Full Stack** **ESP32**

- 1 Electronics (45 min): Wire the full 'ESP32 HTTP POST JSON' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Express Middleware & Body Parser' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'ESP32 HTTP POST JSON' circuit (photographed) + 'Express Middleware & Body Parser' feature committed to GitHub.

Day 250 **Friday** **Debug Electronics + Full Stack** **ESP32**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'ESP32 HTTP POST JSON' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'Express Middleware & Body Parser' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output Bug log in README. 'ESP32 HTTP POST JSON' circuit verified. 'Express Middleware & Body Parser' code with error handling on GitHub.

Day 251 Saturday Project — Project Day

ESP32

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'ESP32 HTTP POST JSON' + 'Express Middleware & Body Parser'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 36 project milestone'.

Output Working project milestone: 'ESP32 HTTP POST JSON' data visible via 'Express Middleware & Body Parser' layer. Pushed to GitHub.

Day 252 Sunday Review — Review Day

ESP32

1 Electronics (20 min): Re-draw your 'ESP32 HTTP POST JSON' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'Express Middleware & Body Parser' out loud for 5 minutes (rubber duck method). Update your Week 36 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output All Week 36 code committed. README updated with learnings and next-week goals.

Week 37

Electronics: ESP32 REST Server Endpoints | Full Stack: REST API Design Principles

Day 253 **Monday** **Learn Electronics** **ESP32**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'ESP32 REST Server Endpoints'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'ESP32 REST Server Endpoints'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-37/README.md with today's topic.

Output Notes page on 'ESP32 REST Server Endpoints' + GitHub README stub created for Week 37.

Day 254 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'REST API Design Principles'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'REST API Design Principles'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 37 - Day 2 Learn: REST API Design Principles'.

Output Working code examples for 'REST API Design Principles' committed to GitHub.

Day 255 **Wednesday** **Practice Full Stack + Electronics** **ESP32**

- 1 Write 3 short coding exercises for 'REST API Design Principles' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'ESP32 REST Server Endpoints' from memory.

Output 3 working code files for 'REST API Design Principles' on GitHub. 'ESP32 REST Server Endpoints' circuit sketched or simulated.

Day 256 **Thursday** **Build Electronics + Full Stack** **ESP32**

- 1 Electronics (45 min): Wire the full 'ESP32 REST Server Endpoints' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'REST API Design Principles' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'ESP32 REST Server Endpoints' circuit (photographed) + 'REST API Design Principles' feature committed to GitHub.

Day 257 **Friday** **Debug Electronics + Full Stack** **ESP32**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'ESP32 REST Server Endpoints' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'REST API Design Principles' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output Bug log in README. 'ESP32 REST Server Endpoints' circuit verified. 'REST API Design Principles' code with error handling on GitHub.

Day 258 Saturday Project — Project Day

ESP32

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'ESP32 REST Server Endpoints' + 'REST API Design Principles'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 37 project milestone'.

Output Working project milestone: 'ESP32 REST Server Endpoints' data visible via 'REST API Design Principles' layer. Pushed to GitHub.

Day 259 Sunday Review — Review Day

ESP32

1 Electronics (20 min): Re-draw your 'ESP32 REST Server Endpoints' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'REST API Design Principles' out loud for 5 minutes (rubber duck method). Update your Week 37 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output All Week 37 code committed. README updated with learnings and next-week goals.

Week 38

Electronics: ESP32 WebSocket Client | Full Stack: WebSockets with Socket.io

Day 260	Monday	Learn Electronics	ESP32
---------	--------	-------------------	-------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'ESP32 WebSocket Client'. Pause every 10 min and write key takeaways.
.

2 Read allaboutcircuits.com for 'ESP32 WebSocket Client'. Copy 3 key formulas or facts into your notes.
.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-38/README.md with today's topic.
.

Output Notes page on 'ESP32 WebSocket Client' + GitHub README stub created for Week 38.
:

Day 261	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'WebSockets with Socket.io'. Code along in VS Code — don't just watch.
.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'WebSockets with Socket.io'. Run every code example locally.
.

3 Commit your notes and code samples to GitHub: 'Week 38 - Day 2 Learn: WebSockets with Socket.io'.
.

Output Working code examples for 'WebSockets with Socket.io' committed to GitHub.
:

Day 262	Wednesday	Practice Full Stack + Electronics	ESP32
---------	-----------	-----------------------------------	-------

1 Write 3 short coding exercises for 'WebSockets with Socket.io' in VS Code. Each exercise tests one specific concept.
.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'ESP32 WebSocket Client' from memory.
.

Output 3 working code files for 'WebSockets with Socket.io' on GitHub. 'ESP32 WebSocket Client' circuit sketched or simulated.
:

Day 263	Thursday	Build Electronics + Full Stack	ESP32
---------	----------	--------------------------------	-------

1 Electronics (45 min): Wire the full 'ESP32 WebSocket Client' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
.

2 Full Stack (60 min): Build the 'WebSockets with Socket.io' feature inside your week's running project. Real, working code.
.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
.

Output Working 'ESP32 WebSocket Client' circuit (photographed) + 'WebSockets with Socket.io' feature committed to GitHub.
:

Day 264	Friday	Debug Electronics + Full Stack	ESP32
---------	--------	--------------------------------	-------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'ESP32 WebSocket Client' circuit. Fix each one. Document them.
.

2

Full Stack (45 min): Review your 'WebSockets with Socket.io' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output

Bug log in README. 'ESP32 WebSocket Client' circuit verified. 'WebSockets with Socket.io' code with error handling on GitHub.

Day 265

Saturday

Project — Project Day

ESP32

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'ESP32 WebSocket Client' + 'WebSockets with Socket.io'.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 38 project milestone'.

Output

Working project milestone: 'ESP32 WebSocket Client' data visible via 'WebSockets with Socket.io' layer. Pushed to GitHub.

Day 266

Sunday

Review — Review Day

ESP32

1

Electronics (20 min): Re-draw your 'ESP32 WebSocket Client' circuit from memory without references. Compare to the real circuit. Note what you missed.

2

Full Stack (20 min): Explain 'WebSockets with Socket.io' out loud for 5 minutes (rubber duck method). Update your Week 38 README with a learning summary.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output

All Week 38 code committed. README updated with learnings and next-week goals.

Week 39

Electronics: ESP32 mDNS & OTA Updates | Full Stack: JWT Authentication in Express

Day 267 **Monday** **Learn Electronics** **ESP32**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'ESP32 mDNS & OTA Updates'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'ESP32 mDNS & OTA Updates'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-39/README.md with today's topic.

Output Notes page on 'ESP32 mDNS & OTA Updates' + GitHub README stub created for Week 39.

Day 268 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'JWT Authentication in Express'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'JWT Authentication in Express'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 39 - Day 2 Learn: JWT Authentication in Express'.

Output Working code examples for 'JWT Authentication in Express' committed to GitHub.

Day 269 **Wednesday** **Practice Full Stack + Electronics** **ESP32**

- 1 Write 3 short coding exercises for 'JWT Authentication in Express' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'ESP32 mDNS & OTA Updates' from memory.

Output 3 working code files for 'JWT Authentication in Express' on GitHub. 'ESP32 mDNS & OTA Updates' circuit sketched or simulated.

Day 270 **Thursday** **Build Electronics + Full Stack** **ESP32**

- 1 Electronics (45 min): Wire the full 'ESP32 mDNS & OTA Updates' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'JWT Authentication in Express' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'ESP32 mDNS & OTA Updates' circuit (photographed) + 'JWT Authentication in Express' feature committed to GitHub.

Day 271 **Friday** **Debug Electronics + Full Stack** **ESP32**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'ESP32 mDNS & OTA Updates' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'JWT Authentication in Express' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output Bug log in README. 'ESP32 mDNS & OTA Updates' circuit verified. 'JWT Authentication in Express' code with error handling on GitHub.

Day 272 Saturday Project — Project Day

ESP32

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'ESP32 mDNS & OTA Updates' + 'JWT Authentication in Express'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 39 project milestone'.

Output Working project milestone: 'ESP32 mDNS & OTA Updates' data visible via 'JWT Authentication in Express' layer. Pushed to GitHub.

Day 273 Sunday Review — Review Day

ESP32

1 Electronics (20 min): Re-draw your 'ESP32 mDNS & OTA Updates' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'JWT Authentication in Express' out loud for 5 minutes (rubber duck method). Update your Week 39 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output All Week 39 code committed. README updated with learnings and next-week goals.

Week 40

Electronics: ESP32 Deep Sleep Modes | Full Stack: Express Error Handling & Logging

Day 274 Monday Learn Electronics ESP32

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'ESP32 Deep Sleep Modes'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'ESP32 Deep Sleep Modes'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-40/README.md with today's topic.

Output Notes page on 'ESP32 Deep Sleep Modes' + GitHub README stub created for Week 40.

Day 275 Tuesday Learn Full Stack

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Express Error Handling & Logging'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Express Error Handling & Logging'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 40 - Day 2 Learn: Express Error Handling & Logging'.

Output Working code examples for 'Express Error Handling & Logging' committed to GitHub.

Day 276 Wednesday Practice Full Stack + Electronics ESP32

- 1 Write 3 short coding exercises for 'Express Error Handling & Logging' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'ESP32 Deep Sleep Modes' from memory.

Output 3 working code files for 'Express Error Handling & Logging' on GitHub. 'ESP32 Deep Sleep Modes' circuit sketched or simulated.

Day 277 Thursday Build Electronics + Full Stack ESP32

- 1 Electronics (45 min): Wire the full 'ESP32 Deep Sleep Modes' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Express Error Handling & Logging' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'ESP32 Deep Sleep Modes' circuit (photographed) + 'Express Error Handling & Logging' feature committed to GitHub.

Day 278 Friday Debug Electronics + Full Stack ESP32

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'ESP32 Deep Sleep Modes' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'Express Error Handling & Logging' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output Bug log in README. 'ESP32 Deep Sleep Modes' circuit verified. 'Express Error Handling & Logging' code with error handling on GitHub.

Day 279 Saturday Project — Project Day

ESP32

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'ESP32 Deep Sleep Modes' + 'Express Error Handling & Logging'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 40 project milestone'.

Output Working project milestone: 'ESP32 Deep Sleep Modes' data visible via 'Express Error Handling & Logging' layer. Pushed to GitHub.

Day 280 Sunday Review — Review Day

ESP32

1 Electronics (20 min): Re-draw your 'ESP32 Deep Sleep Modes' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'Express Error Handling & Logging' out loud for 5 minutes (rubber duck method). Update your Week 40 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output All Week 40 code committed. README updated with learnings and next-week goals.

Week 41

Electronics: ESP32 SPIFFS File System | Full Stack: File I/O in Node.js

Day 281	Monday	Learn Electronics	ESP32
---------	--------	-------------------	-------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'ESP32 SPIFFS File System'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'ESP32 SPIFFS File System'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-41/README.md with today's topic.
- Output** Notes page on 'ESP32 SPIFFS File System' + GitHub README stub created for Week 41.

Day 282	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'File I/O in Node.js'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'File I/O in Node.js'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 41 - Day 2 Learn: File I/O in Node.js'.
- Output** Working code examples for 'File I/O in Node.js' committed to GitHub.

Day 283	Wednesday	Practice Full Stack + Electronics	ESP32
---------	-----------	-----------------------------------	-------

- 1 Write 3 short coding exercises for 'File I/O in Node.js' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'ESP32 SPIFFS File System' from memory.
- Output** 3 working code files for 'File I/O in Node.js' on GitHub. 'ESP32 SPIFFS File System' circuit sketched or simulated.

Day 284	Thursday	Build Electronics + Full Stack	ESP32
---------	----------	--------------------------------	-------

- 1 Electronics (45 min): Wire the full 'ESP32 SPIFFS File System' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'File I/O in Node.js' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'ESP32 SPIFFS File System' circuit (photographed) + 'File I/O in Node.js' feature committed to GitHub.

Day 285	Friday	Debug Electronics + Full Stack	ESP32
---------	--------	--------------------------------	-------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'ESP32 SPIFFS File System' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'File I/O in Node.js' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'ESP32 SPIFFS File System' circuit verified. 'File I/O in Node.js' code with error handling on GitHub.

:

Day 286

Saturday

Project — Project Day

ESP32

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'ESP32 SPIFFS File System' + 'File I/O in Node.js'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 41 project milestone'.

.

Output

Working project milestone: 'ESP32 SPIFFS File System' data visible via 'File I/O in Node.js' layer. Pushed to GitHub.

:

Day 287

Sunday

Review — Review Day

ESP32

1

Electronics (20 min): Re-draw your 'ESP32 SPIFFS File System' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'File I/O in Node.js' out loud for 5 minutes (rubber duck method). Update your Week 41 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 41 code committed. README updated with learnings and next-week goals.

:

Week 42

Electronics: ESP32 BLE Basics | Full Stack: Input Validation with Zod

Day 288 **Monday** **Learn Electronics** **ESP32**

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'ESP32 BLE Basics'. Pause every 10 min and write key takeaways.
.

2 Read allaboutcircuits.com for 'ESP32 BLE Basics'. Copy 3 key formulas or facts into your notes.
.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-42/README.md with today's topic.
.

Output Notes page on 'ESP32 BLE Basics' + GitHub README stub created for Week 42.
:

Day 289 **Tuesday** **Learn Full Stack**

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Input Validation with Zod'. Code along in VS Code — don't just watch.
.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Input Validation with Zod'. Run every code example locally.
.

3 Commit your notes and code samples to GitHub: 'Week 42 - Day 2 Learn: Input Validation with Zod'.
.

Output Working code examples for 'Input Validation with Zod' committed to GitHub.
:

Day 290 **Wednesday** **Practice Full Stack + Electronics** **ESP32**

1 Write 3 short coding exercises for 'Input Validation with Zod' in VS Code. Each exercise tests one specific concept.
.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'ESP32 BLE Basics' from memory.
.

Output 3 working code files for 'Input Validation with Zod' on GitHub. 'ESP32 BLE Basics' circuit sketched or simulated.
:

Day 291 **Thursday** **Build Electronics + Full Stack** **ESP32**

1 Electronics (45 min): Wire the full 'ESP32 BLE Basics' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
.

2 Full Stack (60 min): Build the 'Input Validation with Zod' feature inside your week's running project. Real, working code.
.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
.

Output Working 'ESP32 BLE Basics' circuit (photographed) + 'Input Validation with Zod' feature committed to GitHub.
:

Day 292 **Friday** **Debug Electronics + Full Stack** **ESP32**

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'ESP32 BLE Basics' circuit. Fix each one.
.

2

Full Stack (45 min): Review your 'Input Validation with Zod' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'ESP32 BLE Basics' circuit verified. 'Input Validation with Zod' code with error handling on GitHub.

:

Day 293

Saturday

Project — Project Day

ESP32

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'ESP32 BLE Basics' + 'Input Validation with Zod'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 42 project milestone'.

.

Output

Working project milestone: 'ESP32 BLE Basics' data visible via 'Input Validation with Zod' layer. Pushed to GitHub.

:

Day 294

Sunday

Review — Review Day

ESP32

1

Electronics (20 min): Re-draw your 'ESP32 BLE Basics' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Input Validation with Zod' out loud for 5 minutes (rubber duck method). Update your Week 42 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 42 code committed. README updated with learnings and next-week goals.

:

Week 43

Electronics: ESP32 Multi-sensor Array | Full Stack: Rate Limiting & Security Headers

Day 295 **Monday** **Learn Electronics** **ESP32**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'ESP32 Multi-sensor Array'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'ESP32 Multi-sensor Array'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-43/README.md with today's topic.

Output Notes page on 'ESP32 Multi-sensor Array' + GitHub README stub created for Week 43.

Day 296 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Rate Limiting & Security Headers'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Rate Limiting & Security Headers'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 43 - Day 2 Learn: Rate Limiting & Security Headers'.

Output Working code examples for 'Rate Limiting & Security Headers' committed to GitHub.

Day 297 **Wednesday** **Practice Full Stack + Electronics** **ESP32**

- 1 Write 3 short coding exercises for 'Rate Limiting & Security Headers' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'ESP32 Multi-sensor Array' from memory.

Output 3 working code files for 'Rate Limiting & Security Headers' on GitHub. 'ESP32 Multi-sensor Array' circuit sketched or simulated.

Day 298 **Thursday** **Build Electronics + Full Stack** **ESP32**

- 1 Electronics (45 min): Wire the full 'ESP32 Multi-sensor Array' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Rate Limiting & Security Headers' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'ESP32 Multi-sensor Array' circuit (photographed) + 'Rate Limiting & Security Headers' feature committed to GitHub.

Day 299 **Friday** **Debug Electronics + Full Stack** **ESP32**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'ESP32 Multi-sensor Array' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Rate Limiting & Security Headers' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'ESP32 Multi-sensor Array' circuit verified. 'Rate Limiting & Security Headers' code with error handling on GitHub.

:

Day 300

Saturday

Project — Project Day

ESP32

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'ESP32 Multi-sensor Array' + 'Rate Limiting & Security Headers'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 43 project milestone'.

.

Output

Working project milestone: 'ESP32 Multi-sensor Array' data visible via 'Rate Limiting & Security Headers' layer. Pushed to GitHub.

:

Day 301

Sunday

Review — Review Day

ESP32

1

Electronics (20 min): Re-draw your 'ESP32 Multi-sensor Array' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Rate Limiting & Security Headers' out loud for 5 minutes (rubber duck method). Update your Week 43 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 43 code committed. README updated with learnings and next-week goals.

:

Week 44

Electronics: ESP32 MQTT Client Basics | Full Stack: Postman API Testing & Collections

Day 302	Monday	Learn Electronics	ESP32
---------	--------	-------------------	-------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'ESP32 MQTT Client Basics'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'ESP32 MQTT Client Basics'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-44/README.md with today's topic.
- Output** Notes page on 'ESP32 MQTT Client Basics' + GitHub README stub created for Week 44.

Day 303	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Postman API Testing & Collections'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Postman API Testing & Collections'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 44 - Day 2 Learn: Postman API Testing & Collections'.
- Output** Working code examples for 'Postman API Testing & Collections' committed to GitHub.

Day 304	Wednesday	Practice Full Stack + Electronics	ESP32
---------	-----------	-----------------------------------	-------

- 1 Write 3 short coding exercises for 'Postman API Testing & Collections' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'ESP32 MQTT Client Basics' from memory.
- Output** 3 working code files for 'Postman API Testing & Collections' on GitHub. 'ESP32 MQTT Client Basics' circuit sketched or simulated.

Day 305	Thursday	Build Electronics + Full Stack	ESP32
---------	----------	--------------------------------	-------

- 1 Electronics (45 min): Wire the full 'ESP32 MQTT Client Basics' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'Postman API Testing & Collections' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'ESP32 MQTT Client Basics' circuit (photographed) + 'Postman API Testing & Collections' feature committed to GitHub.

Day 306	Friday	Debug Electronics + Full Stack	ESP32
---------	--------	--------------------------------	-------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'ESP32 MQTT Client Basics' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Postman API Testing & Collections' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'ESP32 MQTT Client Basics' circuit verified. 'Postman API Testing & Collections' code with error handling on GitHub.

:

Day 307

Saturday

Project — Project Day

ESP32

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'ESP32 MQTT Client Basics' + 'Postman API Testing & Collections'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 44 project milestone'.

.

Output

Working project milestone: 'ESP32 MQTT Client Basics' data visible via 'Postman API Testing & Collections' layer. Pushed to GitHub.

:

Day 308

Sunday

Review — Review Day

ESP32

1

Electronics (20 min): Re-draw your 'ESP32 MQTT Client Basics' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Postman API Testing & Collections' out loud for 5 minutes (rubber duck method). Update your Week 44 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 44 code committed. README updated with learnings and next-week goals.

:

Week 45

Electronics: Buffer Week: Phase 3 Review | Full Stack: Buffer Week: Catch Up & Refactor

Day 309	Monday	Learn / Review	Buffer
---------	--------	----------------	--------

1 Review all electronics notes from the past 4 weeks. Re-draw 3 circuits from memory without looking at your notes.

.

2 Read through your GitHub commits for the past 4 weeks. Identify the 2 weakest areas.

.

3 Write a 'Mid-Phase Reflection' entry in your week README: what clicked, what is still unclear.

.

Output Electronics review complete. 2 weak areas identified and noted in README.

:

Day 310	Tuesday	Review / Refactor	Buffer
---------	---------	-------------------	--------

1 Review all Full Stack code from the past 4 weeks. Pick the project you are least proud of.

.

2 Refactor it: improve naming, add comments, fix any errors, add error handling.

.

3 Commit refactored code with message: 'refactor: buffer week improvements'.

.

Output Refactored code committed. README updated with improvement notes.

:

Day 311	Wednesday	Catch Up
---------	-----------	----------

1 Finish any incomplete mini-projects or exercises from the past 4 weeks.

.

2 Push all pending code to GitHub. Ensure every week folder has a complete README.

.

3 Run LeetCode: solve 3 problems you struggled with before. Time yourself.

.

Output All pending work committed. LeetCode practice done.

:

Day 312	Thursday	Deep Dive
---------	----------	-----------

1 Pick the hardest concept from the past 4 weeks. Spend the full session going deeper on it.

.

2 Build a tiny demo project that specifically exercises that concept. No tutorials.

.

3 Push the demo to GitHub with a clear README explaining what you explored.

.

Output Deep dive demo project committed to GitHub.

:

Day 313	Friday	System Design
---------	--------	---------------

1 Study system design: read one chapter of system-design-primer (GitHub, free) or watch one ByteByteGo video.

.

2 Draw the architecture diagram for your phase project. Label every component and connection.

.

3 Write the architecture explanation in your README as if explaining it to a junior developer.

.

Output Architecture diagram drawn and explained in README.

:

Day 314

Saturday

Phase Project Prep

Buffer

1 Plan the upcoming phase project in detail: list all components, features, and acceptance criteria.

.

2 Set up the project repository structure. Create all folders and stub files.

.

3 Order any missing hardware components. Write the hardware shopping list in README.

.

Output Phase project repo created with full plan in README. Hardware ordered.

:

Day 315

Sunday

Rest & Plan

1 Rest fully. No coding, no circuits. Sustainable learning requires recovery.

.

2 Optional: review next week's topics for 15 min. Bookmark 1-2 key resources.

.

3 Write 3 clear goals for the next 4 weeks at the bottom of your buffer README.

.

Output Well rested. 3 goals for next phase written. Resources bookmarked.

:

Week 46

Electronics: Wireless IoT API System Build | Full Stack: Live Sensor API Deployment Build

MILESTONE PROJECT: Wireless IoT API Platform + Live Sensor API Deployment

Day 316	Monday	Plan & Design	
1	Design the full project architecture: draw the hardware circuit and software stack on paper.		
.			
2	List all required components for 'Wireless IoT API System Build' and 'Live Sensor API Deployment Build'. Verify you have everything.		
.			
3	Set up the project GitHub repo. Create README with: project goal, tech stack, architecture diagram placeholder.		
.			
Output	Architecture designed. Project repo created. README outline committed.		
:			
Day 317	Tuesday	Build Core	Wireless
1	Build the core hardware component: 'Wireless IoT API System Build'. Test it works in isolation with multimeter.		
.			
2	Set up the core software scaffold: 'Live Sensor API Deployment Build'. Create all files and folder structure.		
.			
3	Connect hardware and software at the most basic level. Verify data flows correctly.		
.			
Output	Core 'Wireless IoT API System Build' hardware working. 'Live Sensor API Deployment Build' scaffold committed to GitHub.		
:			
Day 318	Wednesday	Build Features	Wireless
1	Implement the main feature of 'Wireless IoT API System Build'. Document every step in README.		
.			
2	Build the primary user-facing feature of 'Live Sensor API Deployment Build'. Write clean, commented code.		
.			
3	Integration check: hardware + software + web layer all communicating correctly.		
.			
Output	Main feature built and integrated. Code committed with documentation.		
:			
Day 319	Thursday	Build Full	Wireless
1	Complete all remaining features for 'Wireless IoT API System Build'. Photograph every stage.		
.			
2	Complete all remaining features for 'Live Sensor API Deployment Build'. Add proper error handling.		
.			
3	End-to-end test: full pipeline from hardware input to web output. Fix all issues.		
.			
Output	Full project working end-to-end. All code committed to GitHub.		
:			
Day 320	Friday	Debug & Polish	Wireless

- 1 Stress test the project: run it for 30 minutes continuously. Log any failures.
.
- 2 Fix all bugs found. Add input validation and error messages. Improve UX.
.
- 3 Code review: clean up all magic numbers, add comments, ensure consistent naming.
.

Output Project stable under load. All bugs fixed. Code cleaned and committed.
:

Day 321 Saturday Document & Demo

- 1 Write comprehensive README: project overview, setup guide, usage instructions, photos.
.
- 2 Record a 2-minute demo video showing the full working project.
.
- 3 Add the project to your portfolio page. Write a 100-word project description.
.

Output README complete. Demo video recorded. Portfolio updated.
:

Day 322 Sunday Launch & Reflect

- 1 Final commit with version tag v1.0.0. Push everything to GitHub.
.
- 2 Write a 200-word reflection: what you built, what you learned, what you'd do differently.
.
- 3 Share the GitHub link with someone (social media, forum, friend). External accountability matters.
.

Output Project v1.0.0 released. Reflection written. GitHub link shared.
:

Week 47

Electronics: IoT API System Finalise | Full Stack: API Deploy & Documentation

Day 323 **Monday** **Learn Electronics** IoT

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'IoT API System Finalise'. Pause every 10 min and write key takeaways.
.
- 2 Read allaboutcircuits.com for 'IoT API System Finalise'. Copy 3 key formulas or facts into your notes.
.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-47/README.md with today's topic.
.

Output Notes page on 'IoT API System Finalise' + GitHub README stub created for Week 47.

:

Day 324 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'API Deploy & Documentation'. Code along in VS Code — don't just watch.
.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'API Deploy & Documentation'. Run every code example locally.
.
- 3 Commit your notes and code samples to GitHub: 'Week 47 - Day 2 Learn: API Deploy & Documentation'.
.

Output Working code examples for 'API Deploy & Documentation' committed to GitHub.

:

Day 325 **Wednesday** **Practice Full Stack + Electronics** IoT

- 1 Write 3 short coding exercises for 'API Deploy & Documentation' in VS Code. Each exercise tests one specific concept.
.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'IoT API System Finalise' from memory.
.

Output 3 working code files for 'API Deploy & Documentation' on GitHub. 'IoT API System Finalise' circuit sketched or simulated.

:

Day 326 **Thursday** **Build Electronics + Full Stack** IoT

- 1 Electronics (45 min): Wire the full 'IoT API System Finalise' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
.
- 2 Full Stack (60 min): Build the 'API Deploy & Documentation' feature inside your week's running project. Real, working code.
.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
.

Output Working 'IoT API System Finalise' circuit (photographed) + 'API Deploy & Documentation' feature committed to GitHub.

:

Day 327 **Friday** **Debug Electronics + Full Stack** IoT

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'IoT API System Finalise' circuit. Fix each one.
.
- Document them.

2

Full Stack (45 min): Review your 'API Deploy & Documentation' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'IoT API System Finalise' circuit verified. 'API Deploy & Documentation' code with error handling on GitHub.

Day 328

Saturday

Project — Project Day

IoT

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'IoT API System Finalise' + 'API Deploy & Documentation'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 47 project milestone'.

.

Output

Working project milestone: 'IoT API System Finalise' data visible via 'API Deploy & Documentation' layer. Pushed to GitHub.

Day 329

Sunday

Review — Review Day

IoT

1

Electronics (20 min): Re-draw your 'IoT API System Finalise' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'API Deploy & Documentation' out loud for 5 minutes (rubber duck method). Update your Week 47 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 47 code committed. README updated with learnings and next-week goals.

Week 48

Electronics: Phase 3 Wrap-Up & Prep | Full Stack: Phase 3 Wrap-Up & Prep

Day 330	Monday	Learn Electronics	Phase
---------	--------	-------------------	-------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Phase 3 Wrap-Up & Prep'. Pause every 10 min and write key takeaways.
.

2 Read allaboutcircuits.com for 'Phase 3 Wrap-Up & Prep'. Copy 3 key formulas or facts into your notes.
.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-48/README.md with today's topic.
.

Output Notes page on 'Phase 3 Wrap-Up & Prep' + GitHub README stub created for Week 48.
:

Day 331	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Phase 3 Wrap-Up & Prep'. Code along in VS Code — don't just watch.
.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Phase 3 Wrap-Up & Prep'. Run every code example locally.
.

3 Commit your notes and code samples to GitHub: 'Week 48 - Day 2 Learn: Phase 3 Wrap-Up & Prep'.
.

Output Working code examples for 'Phase 3 Wrap-Up & Prep' committed to GitHub.
:

Day 332	Wednesday	Practice Full Stack + Electronics	Phase
---------	-----------	-----------------------------------	-------

1 Write 3 short coding exercises for 'Phase 3 Wrap-Up & Prep' in VS Code. Each exercise tests one specific concept.
.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Phase 3 Wrap-Up & Prep' from memory.
.

Output 3 working code files for 'Phase 3 Wrap-Up & Prep' on GitHub. 'Phase 3 Wrap-Up & Prep' circuit sketched or simulated.
:

Day 333	Thursday	Build Electronics + Full Stack	Phase
---------	----------	--------------------------------	-------

1 Electronics (45 min): Wire the full 'Phase 3 Wrap-Up & Prep' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
.

2 Full Stack (60 min): Build the 'Phase 3 Wrap-Up & Prep' feature inside your week's running project. Real, working code.
.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
.

Output Working 'Phase 3 Wrap-Up & Prep' circuit (photographed) + 'Phase 3 Wrap-Up & Prep' feature committed to GitHub.
:

Day 334	Friday	Debug Electronics + Full Stack	Phase
---------	--------	--------------------------------	-------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Phase 3 Wrap-Up & Prep' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Phase 3 Wrap-Up & Prep' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Phase 3 Wrap-Up & Prep' circuit verified. 'Phase 3 Wrap-Up & Prep' code with error handling on GitHub.

:

Day 335

Saturday

Project — Project Day

Phase

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Phase 3 Wrap-Up & Prep' + 'Phase 3 Wrap-Up & Prep'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 48 project milestone'.

.

Output

Working project milestone: 'Phase 3 Wrap-Up & Prep' data visible via 'Phase 3 Wrap-Up & Prep' layer. Pushed to GitHub.

:

Day 336

Sunday

Review — Review Day

Phase

1

Electronics (20 min): Re-draw your 'Phase 3 Wrap-Up & Prep' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Phase 3 Wrap-Up & Prep' out loud for 5 minutes (rubber duck method). Update your Week 48 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 48 code committed. README updated with learnings and next-week goals.

:

Phase



Raspberry Pi + React

Months 13–16 · Weeks 49–64 · Days 337–448

Electronics: Pi OS, GPIO Python, Camera, Flask API, systemd, cron, I2C, SSH hardening

Full Stack: React JSX, Hooks, Router, Context, Custom Hooks, Recharts, Jest + RTL

Milestone: Pi Live Sensor Hub + Full React Dashboard App

Week 49

Electronics: Raspberry Pi OS Setup | Full Stack: React Introduction & JSX

Day 337 **Monday** **Learn Electronics** **Raspberry**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Raspberry Pi OS Setup'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Raspberry Pi OS Setup'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-49/README.md with today's topic.

Output Notes page on 'Raspberry Pi OS Setup' + GitHub README stub created for Week 49.

Day 338 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'React Introduction & JSX'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'React Introduction & JSX'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 49 - Day 2 Learn: React Introduction & JSX'.

Output Working code examples for 'React Introduction & JSX' committed to GitHub.

Day 339 **Wednesday** **Practice Full Stack + Electronics** **Raspberry**

- 1 Write 3 short coding exercises for 'React Introduction & JSX' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Raspberry Pi OS Setup' from memory.

Output 3 working code files for 'React Introduction & JSX' on GitHub. 'Raspberry Pi OS Setup' circuit sketched or simulated.

Day 340 **Thursday** **Build Electronics + Full Stack** **Raspberry**

- 1 Electronics (45 min): Wire the full 'Raspberry Pi OS Setup' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'React Introduction & JSX' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Raspberry Pi OS Setup' circuit (photographed) + 'React Introduction & JSX' feature committed to GitHub.

Day 341 **Friday** **Debug Electronics + Full Stack** **Raspberry**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Raspberry Pi OS Setup' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'React Introduction & JSX' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Raspberry Pi OS Setup' circuit verified. 'React Introduction & JSX' code with error handling on GitHub.

:

Day 342

Saturday

Project — Project Day

Raspberry

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Raspberry Pi OS Setup' + 'React Introduction & JSX'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 49 project milestone'.

.

Output

Working project milestone: 'Raspberry Pi OS Setup' data visible via 'React Introduction & JSX' layer. Pushed to GitHub.

:

Day 343

Sunday

Review — Review Day

Raspberry

1

Electronics (20 min): Re-draw your 'Raspberry Pi OS Setup' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'React Introduction & JSX' out loud for 5 minutes (rubber duck method). Update your Week 49 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 49 code committed. README updated with learnings and next-week goals.

:

Week 50

Electronics: Pi GPIO with Python | Full Stack: React Components & Props

Day 344 **Monday** **Learn Electronics** **Pi**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Pi GPIO with Python'. Pause every 10 min and write key takeaways.
- .
- 2 Read allaboutcircuits.com for 'Pi GPIO with Python'. Copy 3 key formulas or facts into your notes.
- .
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-50/README.md with today's topic.
- .

Output Notes page on 'Pi GPIO with Python' + GitHub README stub created for Week 50.

:

Day 345 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'React Components & Props'. Code along in VS Code — don't just watch.
- .
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'React Components & Props'. Run every code example locally.
- .
- 3 Commit your notes and code samples to GitHub: 'Week 50 - Day 2 Learn: React Components & Props'.
- .

Output Working code examples for 'React Components & Props' committed to GitHub.

:

Day 346 **Wednesday** **Practice Full Stack + Electronics** **Pi**

- 1 Write 3 short coding exercises for 'React Components & Props' in VS Code. Each exercise tests one specific concept.
- .
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- .
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Pi GPIO with Python' from memory.
- .

Output 3 working code files for 'React Components & Props' on GitHub. 'Pi GPIO with Python' circuit sketched or simulated.

:

Day 347 **Thursday** **Build Electronics + Full Stack** **Pi**

- 1 Electronics (45 min): Wire the full 'Pi GPIO with Python' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- .
- 2 Full Stack (60 min): Build the 'React Components & Props' feature inside your week's running project. Real, working code.
- .
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- .

Output Working 'Pi GPIO with Python' circuit (photographed) + 'React Components & Props' feature committed to GitHub.

:

Day 348 **Friday** **Debug Electronics + Full Stack** **Pi**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Pi GPIO with Python' circuit. Fix each one.
- .
- Document them.

2

Full Stack (45 min): Review your 'React Components & Props' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Pi GPIO with Python' circuit verified. 'React Components & Props' code with error handling on GitHub.

:

Day 349

Saturday

Project — Project Day

Pi

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Pi GPIO with Python' + 'React Components & Props'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 50 project milestone'.

.

Output

Working project milestone: 'Pi GPIO with Python' data visible via 'React Components & Props' layer. Pushed to GitHub.

:

Day 350

Sunday

Review — Review Day

Pi

1

Electronics (20 min): Re-draw your 'Pi GPIO with Python' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'React Components & Props' out loud for 5 minutes (rubber duck method). Update your Week 50 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 50 code committed. README updated with learnings and next-week goals.

:

Week 51

Electronics: Pi Serial to Arduino Bridge | Full Stack: React useState Hook

Day 351 **Monday** **Learn Electronics** **Pi**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Pi Serial to Arduino Bridge'. Pause every 10 min and write key takeaways.
.
- 2 Read allaboutcircuits.com for 'Pi Serial to Arduino Bridge'. Copy 3 key formulas or facts into your notes.
.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-51/README.md with today's topic.
.

Output Notes page on 'Pi Serial to Arduino Bridge' + GitHub README stub created for Week 51.

:

Day 352 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'React useState Hook'. Code along in VS Code — don't just watch.
.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'React useState Hook'. Run every code example locally.
.
- 3 Commit your notes and code samples to GitHub: 'Week 51 - Day 2 Learn: React useState Hook'.
.

Output Working code examples for 'React useState Hook' committed to GitHub.

:

Day 353 **Wednesday** **Practice Full Stack + Electronics** **Pi**

- 1 Write 3 short coding exercises for 'React useState Hook' in VS Code. Each exercise tests one specific concept.
.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Pi Serial to Arduino Bridge' from memory.
.

Output 3 working code files for 'React useState Hook' on GitHub. 'Pi Serial to Arduino Bridge' circuit sketched or simulated.

:

Day 354 **Thursday** **Build Electronics + Full Stack** **Pi**

- 1 Electronics (45 min): Wire the full 'Pi Serial to Arduino Bridge' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
.
- 2 Full Stack (60 min): Build the 'React useState Hook' feature inside your week's running project. Real, working code.
.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
.

Output Working 'Pi Serial to Arduino Bridge' circuit (photographed) + 'React useState Hook' feature committed to GitHub.

:

Day 355 **Friday** **Debug Electronics + Full Stack** **Pi**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Pi Serial to Arduino Bridge' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'React useState Hook' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output Bug log in README. 'Pi Serial to Arduino Bridge' circuit verified. 'React useState Hook' code with error handling on GitHub.

Day 356

Saturday

Project — Project Day

Pi

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Pi Serial to Arduino Bridge' + 'React useState Hook'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 51 project milestone'.

.

Output Working project milestone: 'Pi Serial to Arduino Bridge' data visible via 'React useState Hook' layer. Pushed to GitHub.

Day 357

Sunday

Review — Review Day

Pi

1 Electronics (20 min): Re-draw your 'Pi Serial to Arduino Bridge' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'React useState Hook' out loud for 5 minutes (rubber duck method). Update your Week 51 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output All Week 51 code committed. README updated with learnings and next-week goals.

Week 52

Electronics: Pi Camera Module | Full Stack: React useEffect Hook

Day 358	Monday	Learn Electronics	Pi
---------	--------	-------------------	----

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Pi Camera Module'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Pi Camera Module'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-52/README.md with today's topic.

Output Notes page on 'Pi Camera Module' + GitHub README stub created for Week 52.

:

Day 359	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'React useEffect Hook'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'React useEffect Hook'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 52 - Day 2 Learn: React useEffect Hook'.

Output Working code examples for 'React useEffect Hook' committed to GitHub.

:

Day 360	Wednesday	Practice Full Stack + Electronics	Pi
---------	-----------	-----------------------------------	----

- 1 Write 3 short coding exercises for 'React useEffect Hook' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Pi Camera Module' from memory.

Output 3 working code files for 'React useEffect Hook' on GitHub. 'Pi Camera Module' circuit sketched or simulated.

:

Day 361	Thursday	Build Electronics + Full Stack	Pi
---------	----------	--------------------------------	----

- 1 Electronics (45 min): Wire the full 'Pi Camera Module' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'React useEffect Hook' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Pi Camera Module' circuit (photographed) + 'React useEffect Hook' feature committed to GitHub.

:

Day 362	Friday	Debug Electronics + Full Stack	Pi
---------	--------	--------------------------------	----

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Pi Camera Module' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'React useEffect Hook' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Pi Camera Module' circuit verified. 'React useEffect Hook' code with error handling on GitHub.

:

Day 363

Saturday

Project — Project Day

Pi

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Pi Camera Module' + 'React useEffect Hook'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 52 project milestone'.

.

Output

Working project milestone: 'Pi Camera Module' data visible via 'React useEffect Hook' layer. Pushed to GitHub.

:

Day 364

Sunday

Review — Review Day

Pi

1

Electronics (20 min): Re-draw your 'Pi Camera Module' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'React useEffect Hook' out loud for 5 minutes (rubber duck method). Update your Week 52 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 52 code committed. README updated with learnings and next-week goals.

:

Week 53

Electronics: Pi Flask REST API | Full Stack: React Router & Navigation

Day 365	Monday	Learn Electronics	Pi
---------	--------	-------------------	----

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Pi Flask REST API'. Pause every 10 min and write key takeaways.
.

2 Read allaboutcircuits.com for 'Pi Flask REST API'. Copy 3 key formulas or facts into your notes.
.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-53/README.md with today's topic.
.

Output Notes page on 'Pi Flask REST API' + GitHub README stub created for Week 53.
:

Day 366	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'React Router & Navigation'. Code along in VS Code — don't just watch.
.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'React Router & Navigation'. Run every code example locally.
.

3 Commit your notes and code samples to GitHub: 'Week 53 - Day 2 Learn: React Router & Navigation'.
.

Output Working code examples for 'React Router & Navigation' committed to GitHub.
:

Day 367	Wednesday	Practice Full Stack + Electronics	Pi
---------	-----------	-----------------------------------	----

1 Write 3 short coding exercises for 'React Router & Navigation' in VS Code. Each exercise tests one specific concept.
.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Pi Flask REST API' from memory.
.

Output 3 working code files for 'React Router & Navigation' on GitHub. 'Pi Flask REST API' circuit sketched or simulated.
:

Day 368	Thursday	Build Electronics + Full Stack	Pi
---------	----------	--------------------------------	----

1 Electronics (45 min): Wire the full 'Pi Flask REST API' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
.

2 Full Stack (60 min): Build the 'React Router & Navigation' feature inside your week's running project. Real, working code.
.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
.

Output Working 'Pi Flask REST API' circuit (photographed) + 'React Router & Navigation' feature committed to GitHub.
:

Day 369	Friday	Debug Electronics + Full Stack	Pi
---------	--------	--------------------------------	----

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Pi Flask REST API' circuit. Fix each one.
.

Document them.

2

Full Stack (45 min): Review your 'React Router & Navigation' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Pi Flask REST API' circuit verified. 'React Router & Navigation' code with error handling on GitHub.

:

Day 370

Saturday

Project — Project Day

Pi

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Pi Flask REST API' + 'React Router & Navigation'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 53 project milestone'.

.

Output

Working project milestone: 'Pi Flask REST API' data visible via 'React Router & Navigation' layer. Pushed to GitHub.

:

Day 371

Sunday

Review — Review Day

Pi

1

Electronics (20 min): Re-draw your 'Pi Flask REST API' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'React Router & Navigation' out loud for 5 minutes (rubber duck method). Update your Week 53 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 53 code committed. README updated with learnings and next-week goals.

:

Week 54

Electronics: Pi systemd Services | Full Stack: React Context API

Day 372 Monday Learn Electronics

Pi

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Pi systemd Services'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Pi systemd Services'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-54/README.md with today's topic.

Output Notes page on 'Pi systemd Services' + GitHub README stub created for Week 54.

:

Day 373 Tuesday Learn Full Stack

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'React Context API'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'React Context API'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 54 - Day 2 Learn: React Context API'.

Output Working code examples for 'React Context API' committed to GitHub.

:

Day 374 Wednesday Practice Full Stack + Electronics

Pi

- 1 Write 3 short coding exercises for 'React Context API' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Pi systemd Services' from memory.

Output 3 working code files for 'React Context API' on GitHub. 'Pi systemd Services' circuit sketched or simulated.

:

Day 375 Thursday Build Electronics + Full Stack

Pi

- 1 Electronics (45 min): Wire the full 'Pi systemd Services' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'React Context API' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Pi systemd Services' circuit (photographed) + 'React Context API' feature committed to GitHub.

:

Day 376 Friday Debug Electronics + Full Stack

Pi

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Pi systemd Services' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'React Context API' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Pi systemd Services' circuit verified. 'React Context API' code with error handling on GitHub.

:

Day 377

Saturday

Project — Project Day

Pi

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Pi systemd Services' + 'React Context API'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 54 project milestone'.

.

Output

Working project milestone: 'Pi systemd Services' data visible via 'React Context API' layer. Pushed to GitHub.

:

Day 378

Sunday

Review — Review Day

Pi

1

Electronics (20 min): Re-draw your 'Pi systemd Services' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'React Context API' out loud for 5 minutes (rubber duck method). Update your Week 54 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 54 code committed. README updated with learnings and next-week goals.

:

Week 55

Electronics: Pi cron Jobs & Scheduling | Full Stack: React Custom Hooks

Day 379 **Monday** **Learn Electronics** **Pi**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Pi cron Jobs & Scheduling'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'Pi cron Jobs & Scheduling'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-55/README.md with today's topic.
- Output** Notes page on 'Pi cron Jobs & Scheduling' + GitHub README stub created for Week 55.

Day 380 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'React Custom Hooks'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'React Custom Hooks'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 55 - Day 2 Learn: React Custom Hooks'.
- Output** Working code examples for 'React Custom Hooks' committed to GitHub.

Day 381 **Wednesday** **Practice Full Stack + Electronics** **Pi**

- 1 Write 3 short coding exercises for 'React Custom Hooks' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Pi cron Jobs & Scheduling' from memory.
- Output** 3 working code files for 'React Custom Hooks' on GitHub. 'Pi cron Jobs & Scheduling' circuit sketched or simulated.

Day 382 **Thursday** **Build Electronics + Full Stack** **Pi**

- 1 Electronics (45 min): Wire the full 'Pi cron Jobs & Scheduling' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'React Custom Hooks' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'Pi cron Jobs & Scheduling' circuit (photographed) + 'React Custom Hooks' feature committed to GitHub.

Day 383 **Friday** **Debug Electronics + Full Stack** **Pi**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Pi cron Jobs & Scheduling' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'React Custom Hooks' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Pi cron Jobs & Scheduling' circuit verified. 'React Custom Hooks' code with error handling on GitHub.

Day 384

Saturday

Project — Project Day

Pi

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Pi cron Jobs & Scheduling' + 'React Custom Hooks'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 55 project milestone'.

.

Output

Working project milestone: 'Pi cron Jobs & Scheduling' data visible via 'React Custom Hooks' layer. Pushed to GitHub.

:

Day 385

Sunday

Review — Review Day

Pi

1

Electronics (20 min): Re-draw your 'Pi cron Jobs & Scheduling' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'React Custom Hooks' out loud for 5 minutes (rubber duck method). Update your Week 55 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 55 code committed. README updated with learnings and next-week goals.

:

Week 56

Electronics: Pi I2C Sensors via Python | Full Stack: React Forms & Controlled Inputs

Day 386 **Monday** **Learn Electronics** **Pi**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Pi I2C Sensors via Python'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Pi I2C Sensors via Python'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-56/README.md with today's topic.

Output Notes page on 'Pi I2C Sensors via Python' + GitHub README stub created for Week 56.

:

Day 387 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'React Forms & Controlled Inputs'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'React Forms & Controlled Inputs'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 56 - Day 2 Learn: React Forms & Controlled Inputs'.

Output Working code examples for 'React Forms & Controlled Inputs' committed to GitHub.

:

Day 388 **Wednesday** **Practice Full Stack + Electronics** **Pi**

- 1 Write 3 short coding exercises for 'React Forms & Controlled Inputs' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Pi I2C Sensors via Python' from memory.

Output 3 working code files for 'React Forms & Controlled Inputs' on GitHub. 'Pi I2C Sensors via Python' circuit sketched or simulated.

:

Day 389 **Thursday** **Build Electronics + Full Stack** **Pi**

- 1 Electronics (45 min): Wire the full 'Pi I2C Sensors via Python' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'React Forms & Controlled Inputs' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Pi I2C Sensors via Python' circuit (photographed) + 'React Forms & Controlled Inputs' feature committed to GitHub.

:

Day 390 **Friday** **Debug Electronics + Full Stack** **Pi**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Pi I2C Sensors via Python' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'React Forms & Controlled Inputs' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Pi I2C Sensors via Python' circuit verified. 'React Forms & Controlled Inputs' code with error handling on GitHub.

:

Day 391

Saturday

Project — Project Day

Pi

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Pi I2C Sensors via Python' + 'React Forms & Controlled Inputs'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 56 project milestone'.

.

Output

Working project milestone: 'Pi I2C Sensors via Python' data visible via 'React Forms & Controlled Inputs' layer. Pushed to GitHub.

:

Day 392

Sunday

Review — Review Day

Pi

1

Electronics (20 min): Re-draw your 'Pi I2C Sensors via Python' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'React Forms & Controlled Inputs' out loud for 5 minutes (rubber duck method). Update your Week 56 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 56 code committed. README updated with learnings and next-week goals.

:

Week 57

Electronics: Pi SSH Hardening | Full Stack: React useReducer Pattern

Day 393 **Monday** **Learn Electronics** **Pi**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Pi SSH Hardening'. Pause every 10 min and write key takeaways.
.
- 2 Read allaboutcircuits.com for 'Pi SSH Hardening'. Copy 3 key formulas or facts into your notes.
.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-57/README.md with today's topic.
.

Output Notes page on 'Pi SSH Hardening' + GitHub README stub created for Week 57.

:

Day 394 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'React useReducer Pattern'. Code along in VS Code — don't just watch.
.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'React useReducer Pattern'. Run every code example locally.
.
- 3 Commit your notes and code samples to GitHub: 'Week 57 - Day 2 Learn: React useReducer Pattern'.
.

Output Working code examples for 'React useReducer Pattern' committed to GitHub.

:

Day 395 **Wednesday** **Practice Full Stack + Electronics** **Pi**

- 1 Write 3 short coding exercises for 'React useReducer Pattern' in VS Code. Each exercise tests one specific concept.
.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Pi SSH Hardening' from memory.
.

Output 3 working code files for 'React useReducer Pattern' on GitHub. 'Pi SSH Hardening' circuit sketched or simulated.

:

Day 396 **Thursday** **Build Electronics + Full Stack** **Pi**

- 1 Electronics (45 min): Wire the full 'Pi SSH Hardening' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
.
- 2 Full Stack (60 min): Build the 'React useReducer Pattern' feature inside your week's running project. Real, working code.
.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
.

Output Working 'Pi SSH Hardening' circuit (photographed) + 'React useReducer Pattern' feature committed to GitHub.

:

Day 397 **Friday** **Debug Electronics + Full Stack** **Pi**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Pi SSH Hardening' circuit. Fix each one.
.
- Document them.

2

Full Stack (45 min): Review your 'React useReducer Pattern' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Pi SSH Hardening' circuit verified. 'React useReducer Pattern' code with error handling on GitHub.

:

Day 398

Saturday

Project — Project Day

Pi

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Pi SSH Hardening' + 'React useReducer Pattern'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 57 project milestone'.

.

Output

Working project milestone: 'Pi SSH Hardening' data visible via 'React useReducer Pattern' layer. Pushed to GitHub.

:

Day 399

Sunday

Review — Review Day

Pi

1

Electronics (20 min): Re-draw your 'Pi SSH Hardening' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'React useReducer Pattern' out loud for 5 minutes (rubber duck method). Update your Week 57 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 57 code committed. README updated with learnings and next-week goals.

:

Week 58

Electronics: Pi Node.js + Express on Pi | Full Stack: React Recharts & Data Viz

Day 400 Monday Learn Electronics Pi

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Pi Node.js + Express on Pi'. Pause every 10 min and write key takeaways.
 - .
 - 2 Read allaboutcircuits.com for 'Pi Node.js + Express on Pi'. Copy 3 key formulas or facts into your notes.
 - .
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-58/README.md with today's topic.
 - .
- Output** Notes page on 'Pi Node.js + Express on Pi' + GitHub README stub created for Week 58.
- :

Day 401 Tuesday Learn Full Stack

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'React Recharts & Data Viz'. Code along in VS Code — don't just watch.
 - .
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'React Recharts & Data Viz'. Run every code example locally.
 - .
 - 3 Commit your notes and code samples to GitHub: 'Week 58 - Day 2 Learn: React Recharts & Data Viz'.
 - .
- Output** Working code examples for 'React Recharts & Data Viz' committed to GitHub.
- :

Day 402 Wednesday Practice Full Stack + Electronics Pi

- 1 Write 3 short coding exercises for 'React Recharts & Data Viz' in VS Code. Each exercise tests one specific concept.
 - .
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - .
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Pi Node.js + Express on Pi' from memory.
 - .
- Output** 3 working code files for 'React Recharts & Data Viz' on GitHub. 'Pi Node.js + Express on Pi' circuit sketched or simulated.
- :

Day 403 Thursday Build Electronics + Full Stack Pi

- 1 Electronics (45 min): Wire the full 'Pi Node.js + Express on Pi' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - .
 - 2 Full Stack (60 min): Build the 'React Recharts & Data Viz' feature inside your week's running project. Real, working code.
 - .
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
 - .
- Output** Working 'Pi Node.js + Express on Pi' circuit (photographed) + 'React Recharts & Data Viz' feature committed to GitHub.
- :

Day 404 Friday Debug Electronics + Full Stack Pi

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Pi Node.js + Express on Pi' circuit. Fix each one. Document them.
- .

2

Full Stack (45 min): Review your 'React Recharts & Data Viz' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Pi Node.js + Express on Pi' circuit verified. 'React Recharts & Data Viz' code with error handling on GitHub.

Day 405

Saturday

Project — Project Day

Pi

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Pi Node.js + Express on Pi' + 'React Recharts & Data Viz'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 58 project milestone'.

.

Output

Working project milestone: 'Pi Node.js + Express on Pi' data visible via 'React Recharts & Data Viz' layer. Pushed to GitHub.

Day 406

Sunday

Review — Review Day

Pi

1

Electronics (20 min): Re-draw your 'Pi Node.js + Express on Pi' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'React Recharts & Data Viz' out loud for 5 minutes (rubber duck method). Update your Week 58 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 58 code committed. README updated with learnings and next-week goals.

Week 59

Electronics: Pi Socket.io Real-time | Full Stack: React Performance & Memoization

Day 407 Monday Learn Electronics Pi

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Pi Socket.io Real-time'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Pi Socket.io Real-time'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-59/README.md with today's topic.

Output Notes page on 'Pi Socket.io Real-time' + GitHub README stub created for Week 59.

Day 408 Tuesday Learn Full Stack

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'React Performance & Memoization'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'React Performance & Memoization'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 59 - Day 2 Learn: React Performance & Memoization'.

Output Working code examples for 'React Performance & Memoization' committed to GitHub.

Day 409 Wednesday Practice Full Stack + Electronics Pi

- 1 Write 3 short coding exercises for 'React Performance & Memoization' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Pi Socket.io Real-time' from memory.

Output 3 working code files for 'React Performance & Memoization' on GitHub. 'Pi Socket.io Real-time' circuit sketched or simulated.

Day 410 Thursday Build Electronics + Full Stack Pi

- 1 Electronics (45 min): Wire the full 'Pi Socket.io Real-time' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'React Performance & Memoization' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Pi Socket.io Real-time' circuit (photographed) + 'React Performance & Memoization' feature committed to GitHub.

Day 411 Friday Debug Electronics + Full Stack Pi

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Pi Socket.io Real-time' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'React Performance & Memoization' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output

Bug log in README. 'Pi Socket.io Real-time' circuit verified. 'React Performance & Memoization' code with error handling on GitHub.

Day 412

Saturday

Project — Project Day

Pi

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Pi Socket.io Real-time' + 'React Performance & Memoization'.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 59 project milestone'.

Output

Working project milestone: 'Pi Socket.io Real-time' data visible via 'React Performance & Memoization' layer. Pushed to GitHub.

Day 413

Sunday

Review — Review Day

Pi

1

Electronics (20 min): Re-draw your 'Pi Socket.io Real-time' circuit from memory without references. Compare to the real circuit. Note what you missed.

2

Full Stack (20 min): Explain 'React Performance & Memoization' out loud for 5 minutes (rubber duck method). Update your Week 59 README with a learning summary.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output

All Week 59 code committed. README updated with learnings and next-week goals.

Week 60

Electronics: Pi OpenCV Camera Stream | Full Stack: React Testing with Jest & RTL

Day 414 **Monday** **Learn Electronics** **Pi**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Pi OpenCV Camera Stream'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'Pi OpenCV Camera Stream'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-60/README.md with today's topic.
- Output** Notes page on 'Pi OpenCV Camera Stream' + GitHub README stub created for Week 60.

Day 415 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'React Testing with Jest & RTL'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'React Testing with Jest & RTL'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 60 - Day 2 Learn: React Testing with Jest & RTL'.
- Output** Working code examples for 'React Testing with Jest & RTL' committed to GitHub.

Day 416 **Wednesday** **Practice Full Stack + Electronics** **Pi**

- 1 Write 3 short coding exercises for 'React Testing with Jest & RTL' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Pi OpenCV Camera Stream' from memory.
- Output** 3 working code files for 'React Testing with Jest & RTL' on GitHub. 'Pi OpenCV Camera Stream' circuit sketched or simulated.

Day 417 **Thursday** **Build Electronics + Full Stack** **Pi**

- 1 Electronics (45 min): Wire the full 'Pi OpenCV Camera Stream' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'React Testing with Jest & RTL' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'Pi OpenCV Camera Stream' circuit (photographed) + 'React Testing with Jest & RTL' feature committed to GitHub.

Day 418 **Friday** **Debug Electronics + Full Stack** **Pi**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Pi OpenCV Camera Stream' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'React Testing with Jest & RTL' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output

Bug log in README. 'Pi OpenCV Camera Stream' circuit verified. 'React Testing with Jest & RTL' code with error handling on GitHub.

Day 419

Saturday

Project — Project Day

Pi

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Pi OpenCV Camera Stream' + 'React Testing with Jest & RTL'.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 60 project milestone'.

Output

Working project milestone: 'Pi OpenCV Camera Stream' data visible via 'React Testing with Jest & RTL' layer. Pushed to GitHub.

Day 420

Sunday

Review — Review Day

Pi

1

Electronics (20 min): Re-draw your 'Pi OpenCV Camera Stream' circuit from memory without references. Compare to the real circuit. Note what you missed.

2

Full Stack (20 min): Explain 'React Testing with Jest & RTL' out loud for 5 minutes (rubber duck method). Update your Week 60 README with a learning summary.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output

All Week 60 code committed. README updated with learnings and next-week goals.

Week 61

Electronics: Buffer Week: Phase 4 Review | Full Stack: Buffer Week: Catch Up & Refactor

Day 421	Monday	Learn / Review	Buffer
---------	--------	----------------	--------

1 Review all electronics notes from the past 4 weeks. Re-draw 3 circuits from memory without looking at your notes.

.

2 Read through your GitHub commits for the past 4 weeks. Identify the 2 weakest areas.

.

3 Write a 'Mid-Phase Reflection' entry in your week README: what clicked, what is still unclear.

.

Output Electronics review complete. 2 weak areas identified and noted in README.

:

Day 422	Tuesday	Review / Refactor	Buffer
---------	---------	-------------------	--------

1 Review all Full Stack code from the past 4 weeks. Pick the project you are least proud of.

.

2 Refactor it: improve naming, add comments, fix any errors, add error handling.

.

3 Commit refactored code with message: 'refactor: buffer week improvements'.

.

Output Refactored code committed. README updated with improvement notes.

:

Day 423	Wednesday	Catch Up
---------	-----------	----------

1 Finish any incomplete mini-projects or exercises from the past 4 weeks.

.

2 Push all pending code to GitHub. Ensure every week folder has a complete README.

.

3 Run LeetCode: solve 3 problems you struggled with before. Time yourself.

.

Output All pending work committed. LeetCode practice done.

:

Day 424	Thursday	Deep Dive
---------	----------	-----------

1 Pick the hardest concept from the past 4 weeks. Spend the full session going deeper on it.

.

2 Build a tiny demo project that specifically exercises that concept. No tutorials.

.

3 Push the demo to GitHub with a clear README explaining what you explored.

.

Output Deep dive demo project committed to GitHub.

:

Day 425	Friday	System Design
---------	--------	---------------

1 Study system design: read one chapter of system-design-primer (GitHub, free) or watch one ByteByteGo video.

.

2 Draw the architecture diagram for your phase project. Label every component and connection.

.

3 Write the architecture explanation in your README as if explaining it to a junior developer.

.

Output Architecture diagram drawn and explained in README.

:

Day 426

Saturday

Phase Project Prep

Buffer

1 Plan the upcoming phase project in detail: list all components, features, and acceptance criteria.

.

2 Set up the project repository structure. Create all folders and stub files.

.

3 Order any missing hardware components. Write the hardware shopping list in README.

.

Output Phase project repo created with full plan in README. Hardware ordered.

:

Day 427

Sunday

Rest & Plan

1 Rest fully. No coding, no circuits. Sustainable learning requires recovery.

.

2 Optional: review next week's topics for 15 min. Bookmark 1-2 key resources.

.

3 Write 3 clear goals for the next 4 weeks at the bottom of your buffer README.

.

Output Well rested. 3 goals for next phase written. Resources bookmarked.

:

Week 62

Electronics: Pi Live Sensor Hub Build | Full Stack: Full React Dashboard App Build

MILESTONE PROJECT: Pi Live Sensor Hub + Full React Dashboard App

Day 428	Monday	Plan & Design	
1	Design the full project architecture: draw the hardware circuit and software stack on paper.		
.			
2	List all required components for 'Pi Live Sensor Hub Build' and 'Full React Dashboard App Build'. Verify you have everything.		
.			
3	Set up the project GitHub repo. Create README with: project goal, tech stack, architecture diagram placeholder.		
.			
Output	Architecture designed. Project repo created. README outline committed.		
:			
Day 429	Tuesday	Build Core	Pi
1	Build the core hardware component: 'Pi Live Sensor Hub Build'. Test it works in isolation with multimeter.		
.			
2	Set up the core software scaffold: 'Full React Dashboard App Build'. Create all files and folder structure.		
.			
3	Connect hardware and software at the most basic level. Verify data flows correctly.		
.			
Output	Core 'Pi Live Sensor Hub Build' hardware working. 'Full React Dashboard App Build' scaffold committed to GitHub.		
:			
Day 430	Wednesday	Build Features	Pi
1	Implement the main feature of 'Pi Live Sensor Hub Build'. Document every step in README.		
.			
2	Build the primary user-facing feature of 'Full React Dashboard App Build'. Write clean, commented code.		
.			
3	Integration check: hardware + software + web layer all communicating correctly.		
.			
Output	Main feature built and integrated. Code committed with documentation.		
:			
Day 431	Thursday	Build Full	Pi
1	Complete all remaining features for 'Pi Live Sensor Hub Build'. Photograph every stage.		
.			
2	Complete all remaining features for 'Full React Dashboard App Build'. Add proper error handling.		
.			
3	End-to-end test: full pipeline from hardware input to web output. Fix all issues.		
.			
Output	Full project working end-to-end. All code committed to GitHub.		
:			
Day 432	Friday	Debug & Polish	Pi

- 1 Stress test the project: run it for 30 minutes continuously. Log any failures.
.
- 2 Fix all bugs found. Add input validation and error messages. Improve UX.
.
- 3 Code review: clean up all magic numbers, add comments, ensure consistent naming.
.

Output Project stable under load. All bugs fixed. Code cleaned and committed.

:

Day 433 Saturday Document & Demo

- 1 Write comprehensive README: project overview, setup guide, usage instructions, photos.
.
- 2 Record a 2-minute demo video showing the full working project.
.
- 3 Add the project to your portfolio page. Write a 100-word project description.
.

Output README complete. Demo video recorded. Portfolio updated.

:

Day 434 Sunday Launch & Reflect

- 1 Final commit with version tag v1.0.0. Push everything to GitHub.
.
- 2 Write a 200-word reflection: what you built, what you learned, what you'd do differently.
.
- 3 Share the GitHub link with someone (social media, forum, friend). External accountability matters.
.

Output Project v1.0.0 released. Reflection written. GitHub link shared.

:

Week 63

Electronics: Pi Sensor Hub Finalise | Full Stack: React Dashboard Polish & Deploy

Day 435 **Monday** **Learn Electronics** **Pi**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Pi Sensor Hub Finalise'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Pi Sensor Hub Finalise'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-63/README.md with today's topic.

Output Notes page on 'Pi Sensor Hub Finalise' + GitHub README stub created for Week 63.

Day 436 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'React Dashboard Polish & Deploy'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'React Dashboard Polish & Deploy'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 63 - Day 2 Learn: React Dashboard Polish & Deploy'.

Output Working code examples for 'React Dashboard Polish & Deploy' committed to GitHub.

Day 437 **Wednesday** **Practice Full Stack + Electronics** **Pi**

- 1 Write 3 short coding exercises for 'React Dashboard Polish & Deploy' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Pi Sensor Hub Finalise' from memory.

Output 3 working code files for 'React Dashboard Polish & Deploy' on GitHub. 'Pi Sensor Hub Finalise' circuit sketched or simulated.

Day 438 **Thursday** **Build Electronics + Full Stack** **Pi**

- 1 Electronics (45 min): Wire the full 'Pi Sensor Hub Finalise' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'React Dashboard Polish & Deploy' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Pi Sensor Hub Finalise' circuit (photographed) + 'React Dashboard Polish & Deploy' feature committed to GitHub.

Day 439 **Friday** **Debug Electronics + Full Stack** **Pi**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Pi Sensor Hub Finalise' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'React Dashboard Polish & Deploy' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output

Bug log in README. 'Pi Sensor Hub Finalise' circuit verified. 'React Dashboard Polish & Deploy' code with error handling on GitHub.

Day 440

Saturday

Project — Project Day

Pi

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Pi Sensor Hub Finalise' + 'React Dashboard Polish & Deploy'.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 63 project milestone'.

Output

Working project milestone: 'Pi Sensor Hub Finalise' data visible via 'React Dashboard Polish & Deploy' layer. Pushed to GitHub.

Day 441

Sunday

Review — Review Day

Pi

1

Electronics (20 min): Re-draw your 'Pi Sensor Hub Finalise' circuit from memory without references. Compare to the real circuit. Note what you missed.

2

Full Stack (20 min): Explain 'React Dashboard Polish & Deploy' out loud for 5 minutes (rubber duck method). Update your Week 63 README with a learning summary.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output

All Week 63 code committed. README updated with learnings and next-week goals.

Week 64

Electronics: Phase 4 Wrap-Up & Prep | Full Stack: Phase 4 Wrap-Up & Prep

Day 442	Monday	Learn Electronics	Phase
---------	--------	-------------------	-------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Phase 4 Wrap-Up & Prep'. Pause every 10 min and write key takeaways.
.

2 Read allaboutcircuits.com for 'Phase 4 Wrap-Up & Prep'. Copy 3 key formulas or facts into your notes.
.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-64/README.md with today's topic.
.

Output Notes page on 'Phase 4 Wrap-Up & Prep' + GitHub README stub created for Week 64.
:

Day 443	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Phase 4 Wrap-Up & Prep'. Code along in VS Code — don't just watch.
.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Phase 4 Wrap-Up & Prep'. Run every code example locally.
.

3 Commit your notes and code samples to GitHub: 'Week 64 - Day 2 Learn: Phase 4 Wrap-Up & Prep'.
.

Output Working code examples for 'Phase 4 Wrap-Up & Prep' committed to GitHub.
:

Day 444	Wednesday	Practice Full Stack + Electronics	Phase
---------	-----------	-----------------------------------	-------

1 Write 3 short coding exercises for 'Phase 4 Wrap-Up & Prep' in VS Code. Each exercise tests one specific concept.
.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Phase 4 Wrap-Up & Prep' from memory.
.

Output 3 working code files for 'Phase 4 Wrap-Up & Prep' on GitHub. 'Phase 4 Wrap-Up & Prep' circuit sketched or simulated.
:

Day 445	Thursday	Build Electronics + Full Stack	Phase
---------	----------	--------------------------------	-------

1 Electronics (45 min): Wire the full 'Phase 4 Wrap-Up & Prep' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
.

2 Full Stack (60 min): Build the 'Phase 4 Wrap-Up & Prep' feature inside your week's running project. Real, working code.
.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
.

Output Working 'Phase 4 Wrap-Up & Prep' circuit (photographed) + 'Phase 4 Wrap-Up & Prep' feature committed to GitHub.
:

Day 446	Friday	Debug Electronics + Full Stack	Phase
---------	--------	--------------------------------	-------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Phase 4 Wrap-Up & Prep' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Phase 4 Wrap-Up & Prep' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Phase 4 Wrap-Up & Prep' circuit verified. 'Phase 4 Wrap-Up & Prep' code with error handling on GitHub.

:

Day 447

Saturday

Project — Project Day

Phase

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Phase 4 Wrap-Up & Prep' + 'Phase 4 Wrap-Up & Prep'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 64 project milestone'.

.

Output

Working project milestone: 'Phase 4 Wrap-Up & Prep' data visible via 'Phase 4 Wrap-Up & Prep' layer. Pushed to GitHub.

:

Day 448

Sunday

Review — Review Day

Phase

1

Electronics (20 min): Re-draw your 'Phase 4 Wrap-Up & Prep' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Phase 4 Wrap-Up & Prep' out loud for 5 minutes (rubber duck method). Update your Week 64 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 64 code committed. README updated with learnings and next-week goals.

:

Phase

5

Docker + Databases + Deployment

Months 17–20 · Weeks 65–80 · Days 449–560

Full Stack: PostgreSQL, SQL, Prisma ORM, Redis, Time-Series Data, MongoDB, Docker

DevOps: Docker Compose, CI/CD GitHub Actions, Nginx, HTTPS, Cloud Deploy, Monitoring

Milestone: Full DB + Docker Deployment + Complete Cloud Deploy Project

Week 65

Electronics: PostgreSQL Schema Design | Full Stack: Docker Fundamentals & CLI

Day 449	Monday	Learn Electronics	PostgreSQL
---------	--------	-------------------	------------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'PostgreSQL Schema Design'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'PostgreSQL Schema Design'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-65/README.md with today's topic.

Output Notes page on 'PostgreSQL Schema Design' + GitHub README stub created for Week 65.

Day 450	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Docker Fundamentals & CLI'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Docker Fundamentals & CLI'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 65 - Day 2 Learn: Docker Fundamentals & CLI'.

Output Working code examples for 'Docker Fundamentals & CLI' committed to GitHub.

Day 451	Wednesday	Practice Full Stack + Electronics	PostgreSQL
---------	-----------	-----------------------------------	------------

- 1 Write 3 short coding exercises for 'Docker Fundamentals & CLI' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'PostgreSQL Schema Design' from memory.

Output 3 working code files for 'Docker Fundamentals & CLI' on GitHub. 'PostgreSQL Schema Design' circuit sketched or simulated.

Day 452	Thursday	Build Electronics + Full Stack	PostgreSQL
---------	----------	--------------------------------	------------

- 1 Electronics (45 min): Wire the full 'PostgreSQL Schema Design' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Docker Fundamentals & CLI' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'PostgreSQL Schema Design' circuit (photographed) + 'Docker Fundamentals & CLI' feature committed to GitHub.

Day 453	Friday	Debug Electronics + Full Stack	PostgreSQL
---------	--------	--------------------------------	------------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'PostgreSQL Schema Design' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Docker Fundamentals & CLI' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'PostgreSQL Schema Design' circuit verified. 'Docker Fundamentals & CLI' code with error handling on GitHub.

Day 454

Saturday

Project — Project Day

PostgreSQL

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'PostgreSQL Schema Design' + 'Docker Fundamentals & CLI'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 65 project milestone'.

.

Output

Working project milestone: 'PostgreSQL Schema Design' data visible via 'Docker Fundamentals & CLI' layer. Pushed to GitHub.

Day 455

Sunday

Review — Review Day

PostgreSQL

1

Electronics (20 min): Re-draw your 'PostgreSQL Schema Design' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Docker Fundamentals & CLI' out loud for 5 minutes (rubber duck method). Update your Week 65 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 65 code committed. README updated with learnings and next-week goals.

Week 66

Electronics: SQL CRUD Operations | Full Stack: Dockerfile & Image Building

Day 456	Monday	Learn Electronics	SQL
---------	--------	-------------------	-----

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'SQL CRUD Operations'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'SQL CRUD Operations'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-66/README.md with today's topic.

Output Notes page on 'SQL CRUD Operations' + GitHub README stub created for Week 66.

Day 457	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Dockerfile & Image Building'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Dockerfile & Image Building'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 66 - Day 2 Learn: Dockerfile & Image Building'.

Output Working code examples for 'Dockerfile & Image Building' committed to GitHub.

Day 458	Wednesday	Practice Full Stack + Electronics	SQL
---------	-----------	-----------------------------------	-----

- 1 Write 3 short coding exercises for 'Dockerfile & Image Building' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'SQL CRUD Operations' from memory.

Output 3 working code files for 'Dockerfile & Image Building' on GitHub. 'SQL CRUD Operations' circuit sketched or simulated.

Day 459	Thursday	Build Electronics + Full Stack	SQL
---------	----------	--------------------------------	-----

- 1 Electronics (45 min): Wire the full 'SQL CRUD Operations' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Dockerfile & Image Building' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'SQL CRUD Operations' circuit (photographed) + 'Dockerfile & Image Building' feature committed to GitHub.

Day 460	Friday	Debug Electronics + Full Stack	SQL
---------	--------	--------------------------------	-----

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'SQL CRUD Operations' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Dockerfile & Image Building' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'SQL CRUD Operations' circuit verified. 'Dockerfile & Image Building' code with error handling on GitHub.

:

Day 461

Saturday

Project — Project Day

SQL

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'SQL CRUD Operations' + 'Dockerfile & Image Building'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 66 project milestone'.

.

Output

Working project milestone: 'SQL CRUD Operations' data visible via 'Dockerfile & Image Building' layer. Pushed to GitHub.

:

Day 462

Sunday

Review — Review Day

SQL

1

Electronics (20 min): Re-draw your 'SQL CRUD Operations' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Dockerfile & Image Building' out loud for 5 minutes (rubber duck method). Update your Week 66 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 66 code committed. README updated with learnings and next-week goals.

:

Week 67

Electronics: SQL Joins & Aggregations | Full Stack: Docker Compose Multi-service

Day 463 **Monday** **Learn Electronics** **SQL**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'SQL Joins & Aggregations'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'SQL Joins & Aggregations'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-67/README.md with today's topic.

Output Notes page on 'SQL Joins & Aggregations' + GitHub README stub created for Week 67.

Day 464 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Docker Compose Multi-service'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Docker Compose Multi-service'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 67 - Day 2 Learn: Docker Compose Multi-service'.

Output Working code examples for 'Docker Compose Multi-service' committed to GitHub.

Day 465 **Wednesday** **Practice Full Stack + Electronics** **SQL**

- 1 Write 3 short coding exercises for 'Docker Compose Multi-service' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'SQL Joins & Aggregations' from memory.

Output 3 working code files for 'Docker Compose Multi-service' on GitHub. 'SQL Joins & Aggregations' circuit sketched or simulated.

Day 466 **Thursday** **Build Electronics + Full Stack** **SQL**

- 1 Electronics (45 min): Wire the full 'SQL Joins & Aggregations' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Docker Compose Multi-service' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'SQL Joins & Aggregations' circuit (photographed) + 'Docker Compose Multi-service' feature committed to GitHub.

Day 467 **Friday** **Debug Electronics + Full Stack** **SQL**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'SQL Joins & Aggregations' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Docker Compose Multi-service' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'SQL Joins & Aggregations' circuit verified. 'Docker Compose Multi-service' code with error handling on GitHub.

:

Day 468

Saturday

Project — Project Day

SQL

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'SQL Joins & Aggregations' + 'Docker Compose Multi-service'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 67 project milestone'.

.

Output

Working project milestone: 'SQL Joins & Aggregations' data visible via 'Docker Compose Multi-service' layer. Pushed to GitHub.

:

Day 469

Sunday

Review — Review Day

SQL

1

Electronics (20 min): Re-draw your 'SQL Joins & Aggregations' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Docker Compose Multi-service' out loud for 5 minutes (rubber duck method). Update your Week 67 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 67 code committed. README updated with learnings and next-week goals.

:

Week 68

Electronics: PostgreSQL Indexing | Full Stack: Docker Volumes & Networking

Day 470	Monday	Learn Electronics	PostgreSQL
---------	--------	-------------------	------------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'PostgreSQL Indexing'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'PostgreSQL Indexing'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-68/README.md with today's topic.

Output Notes page on 'PostgreSQL Indexing' + GitHub README stub created for Week 68.

Day 471	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Docker Volumes & Networking'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Docker Volumes & Networking'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 68 - Day 2 Learn: Docker Volumes & Networking'.

Output Working code examples for 'Docker Volumes & Networking' committed to GitHub.

Day 472	Wednesday	Practice Full Stack + Electronics	PostgreSQL
---------	-----------	-----------------------------------	------------

- 1 Write 3 short coding exercises for 'Docker Volumes & Networking' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'PostgreSQL Indexing' from memory.

Output 3 working code files for 'Docker Volumes & Networking' on GitHub. 'PostgreSQL Indexing' circuit sketched or simulated.

Day 473	Thursday	Build Electronics + Full Stack	PostgreSQL
---------	----------	--------------------------------	------------

- 1 Electronics (45 min): Wire the full 'PostgreSQL Indexing' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Docker Volumes & Networking' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'PostgreSQL Indexing' circuit (photographed) + 'Docker Volumes & Networking' feature committed to GitHub.

Day 474	Friday	Debug Electronics + Full Stack	PostgreSQL
---------	--------	--------------------------------	------------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'PostgreSQL Indexing' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Docker Volumes & Networking' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'PostgreSQL Indexing' circuit verified. 'Docker Volumes & Networking' code with error handling on GitHub.

Day 475

Saturday

Project — Project Day

PostgreSQL

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'PostgreSQL Indexing' + 'Docker Volumes & Networking'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 68 project milestone'.

.

Output

Working project milestone: 'PostgreSQL Indexing' data visible via 'Docker Volumes & Networking' layer. Pushed to GitHub.

Day 476

Sunday

Review — Review Day

PostgreSQL

1

Electronics (20 min): Re-draw your 'PostgreSQL Indexing' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Docker Volumes & Networking' out loud for 5 minutes (rubber duck method). Update your Week 68 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 68 code committed. README updated with learnings and next-week goals.

Week 69

Electronics: Prisma ORM Integration | Full Stack: Container Registry & Docker Hub

Day 477	Monday	Learn Electronics	Prisma
---------	--------	-------------------	--------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Prisma ORM Integration'. Pause every 10 min and write key takeaways.

2 Read allaboutcircuits.com for 'Prisma ORM Integration'. Copy 3 key formulas or facts into your notes.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-69/README.md with today's topic.

Output Notes page on 'Prisma ORM Integration' + GitHub README stub created for Week 69.

Day 478	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Container Registry & Docker Hub'. Code along in VS Code — don't just watch.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Container Registry & Docker Hub'. Run every code example locally.

3 Commit your notes and code samples to GitHub: 'Week 69 - Day 2 Learn: Container Registry & Docker Hub'.

Output Working code examples for 'Container Registry & Docker Hub' committed to GitHub.

Day 479	Wednesday	Practice Full Stack + Electronics	Prisma
---------	-----------	-----------------------------------	--------

1 Write 3 short coding exercises for 'Container Registry & Docker Hub' in VS Code. Each exercise tests one specific concept.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Prisma ORM Integration' from memory.

Output 3 working code files for 'Container Registry & Docker Hub' on GitHub. 'Prisma ORM Integration' circuit sketched or simulated.

Day 480	Thursday	Build Electronics + Full Stack	Prisma
---------	----------	--------------------------------	--------

1 Electronics (45 min): Wire the full 'Prisma ORM Integration' circuit on your breadboard. Test with multimeter. Troubleshoot until working.

2 Full Stack (60 min): Build the 'Container Registry & Docker Hub' feature inside your week's running project. Real, working code.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Prisma ORM Integration' circuit (photographed) + 'Container Registry & Docker Hub' feature committed to GitHub.

Day 481	Friday	Debug Electronics + Full Stack	Prisma
---------	--------	--------------------------------	--------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Prisma ORM Integration' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Container Registry & Docker Hub' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Prisma ORM Integration' circuit verified. 'Container Registry & Docker Hub' code with error handling on GitHub.

Day 482

Saturday

Project — Project Day

Prisma

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Prisma ORM Integration' + 'Container Registry & Docker Hub'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 69 project milestone'.

.

Output

Working project milestone: 'Prisma ORM Integration' data visible via 'Container Registry & Docker Hub' layer. Pushed to GitHub.

Day 483

Sunday

Review — Review Day

Prisma

1

Electronics (20 min): Re-draw your 'Prisma ORM Integration' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Container Registry & Docker Hub' out loud for 5 minutes (rubber duck method). Update your Week 69 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 69 code committed. README updated with learnings and next-week goals.

Week 70

Electronics: Redis Caching & Pub/Sub | Full Stack: CI/CD with GitHub Actions

Day 484 **Monday** **Learn Electronics** **Redis**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Redis Caching & Pub/Sub'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Redis Caching & Pub/Sub'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-70/README.md with today's topic.

Output Notes page on 'Redis Caching & Pub/Sub' + GitHub README stub created for Week 70.

:

Day 485 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'CI/CD with GitHub Actions'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'CI/CD with GitHub Actions'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 70 - Day 2 Learn: CI/CD with GitHub Actions'.

Output Working code examples for 'CI/CD with GitHub Actions' committed to GitHub.

:

Day 486 **Wednesday** **Practice Full Stack + Electronics** **Redis**

- 1 Write 3 short coding exercises for 'CI/CD with GitHub Actions' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Redis Caching & Pub/Sub' from memory.

Output 3 working code files for 'CI/CD with GitHub Actions' on GitHub. 'Redis Caching & Pub/Sub' circuit sketched or simulated.

:

Day 487 **Thursday** **Build Electronics + Full Stack** **Redis**

- 1 Electronics (45 min): Wire the full 'Redis Caching & Pub/Sub' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'CI/CD with GitHub Actions' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Redis Caching & Pub/Sub' circuit (photographed) + 'CI/CD with GitHub Actions' feature committed to GitHub.

:

Day 488 **Friday** **Debug Electronics + Full Stack** **Redis**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Redis Caching & Pub/Sub' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'CI/CD with GitHub Actions' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output

Bug log in README. 'Redis Caching & Pub/Sub' circuit verified. 'CI/CD with GitHub Actions' code with error handling on GitHub.

Day 489

Saturday

Project — Project Day

Redis

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Redis Caching & Pub/Sub' + 'CI/CD with GitHub Actions'.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 70 project milestone'.

Output

Working project milestone: 'Redis Caching & Pub/Sub' data visible via 'CI/CD with GitHub Actions' layer. Pushed to GitHub.

Day 490

Sunday

Review — Review Day

Redis

1

Electronics (20 min): Re-draw your 'Redis Caching & Pub/Sub' circuit from memory without references. Compare to the real circuit. Note what you missed.

2

Full Stack (20 min): Explain 'CI/CD with GitHub Actions' out loud for 5 minutes (rubber duck method). Update your Week 70 README with a learning summary.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output

All Week 70 code committed. README updated with learnings and next-week goals.

Week 71

Electronics: Time-Series Data Queries | Full Stack: Automated Testing Pipeline

Day 491 **Monday** **Learn Electronics** Time-Series

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Time-Series Data Queries'. Pause every 10 min and write key takeaways.
.

2 Read allaboutcircuits.com for 'Time-Series Data Queries'. Copy 3 key formulas or facts into your notes.
.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-71/README.md with today's topic.
.

Output Notes page on 'Time-Series Data Queries' + GitHub README stub created for Week 71.
:

Day 492 **Tuesday** **Learn Full Stack**

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Automated Testing Pipeline'. Code along in VS Code — don't just watch.
.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Automated Testing Pipeline'. Run every code example locally.
.

3 Commit your notes and code samples to GitHub: 'Week 71 - Day 2 Learn: Automated Testing Pipeline'.
.

Output Working code examples for 'Automated Testing Pipeline' committed to GitHub.
:

Day 493 **Wednesday** **Practice Full Stack + Electronics** Time-Series

1 Write 3 short coding exercises for 'Automated Testing Pipeline' in VS Code. Each exercise tests one specific concept.
.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Time-Series Data Queries' from memory.
.

Output 3 working code files for 'Automated Testing Pipeline' on GitHub. 'Time-Series Data Queries' circuit sketched or simulated.
:

Day 494 **Thursday** **Build Electronics + Full Stack** Time-Series

1 Electronics (45 min): Wire the full 'Time-Series Data Queries' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
.

2 Full Stack (60 min): Build the 'Automated Testing Pipeline' feature inside your week's running project. Real, working code.
.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
.

Output Working 'Time-Series Data Queries' circuit (photographed) + 'Automated Testing Pipeline' feature committed to GitHub.
:

Day 495 **Friday** **Debug Electronics + Full Stack** Time-Series

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Time-Series Data Queries' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Automated Testing Pipeline' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Time-Series Data Queries' circuit verified. 'Automated Testing Pipeline' code with error handling on GitHub.

Day 496

Saturday

Project — Project Day

Time-Series

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Time-Series Data Queries' + 'Automated Testing Pipeline'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 71 project milestone'.

.

Output

Working project milestone: 'Time-Series Data Queries' data visible via 'Automated Testing Pipeline' layer. Pushed to GitHub.

Day 497

Sunday

Review — Review Day

Time-Series

1

Electronics (20 min): Re-draw your 'Time-Series Data Queries' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Automated Testing Pipeline' out loud for 5 minutes (rubber duck method). Update your Week 71 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 71 code committed. README updated with learnings and next-week goals.

Week 72

Electronics: Database Migrations | Full Stack: Cloud VPS Setup (Railway/Render)

Day 498	Monday	Learn Electronics	Database
---------	--------	-------------------	----------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Database Migrations'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Database Migrations'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-72/README.md with today's topic.

Output Notes page on 'Database Migrations' + GitHub README stub created for Week 72.

Day 499	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Cloud VPS Setup (Railway/Render)'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Cloud VPS Setup (Railway/Render)'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 72 - Day 2 Learn: Cloud VPS Setup (Railway/Render)'.

Output Working code examples for 'Cloud VPS Setup (Railway/Render)' committed to GitHub.

Day 500	Wednesday	Practice Full Stack + Electronics	Database
---------	-----------	-----------------------------------	----------

- 1 Write 3 short coding exercises for 'Cloud VPS Setup (Railway/Render)' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Database Migrations' from memory.

Output 3 working code files for 'Cloud VPS Setup (Railway/Render)' on GitHub. 'Database Migrations' circuit sketched or simulated.

Day 501	Thursday	Build Electronics + Full Stack	Database
---------	----------	--------------------------------	----------

- 1 Electronics (45 min): Wire the full 'Database Migrations' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Cloud VPS Setup (Railway/Render)' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Database Migrations' circuit (photographed) + 'Cloud VPS Setup (Railway/Render)' feature committed to GitHub.

Day 502	Friday	Debug Electronics + Full Stack	Database
---------	--------	--------------------------------	----------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Database Migrations' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Cloud VPS Setup (Railway/Render)' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output

Bug log in README. 'Database Migrations' circuit verified. 'Cloud VPS Setup (Railway/Render)' code with error handling on GitHub.

Day 503

Saturday

Project — Project Day

Database

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Database Migrations' + 'Cloud VPS Setup (Railway/Render)'.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 72 project milestone'.

Output

Working project milestone: 'Database Migrations' data visible via 'Cloud VPS Setup (Railway/Render)' layer. Pushed to GitHub.

Day 504

Sunday

Review — Review Day

Database

1

Electronics (20 min): Re-draw your 'Database Migrations' circuit from memory without references. Compare to the real circuit. Note what you missed.

2

Full Stack (20 min): Explain 'Cloud VPS Setup (Railway/Render)' out loud for 5 minutes (rubber duck method). Update your Week 72 README with a learning summary.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output

All Week 72 code committed. README updated with learnings and next-week goals.

Week 73

Electronics: Connection Pooling | Full Stack: Nginx Reverse Proxy Config

Day 505	Monday	Learn Electronics	Connection
---------	--------	-------------------	------------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Connection Pooling'. Pause every 10 min and write key takeaways.

.

2 Read allaboutcircuits.com for 'Connection Pooling'. Copy 3 key formulas or facts into your notes.

.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-73/README.md with today's topic.

.

Output Notes page on 'Connection Pooling' + GitHub README stub created for Week 73.

:

Day 506	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Nginx Reverse Proxy Config'. Code along in VS Code — don't just watch.

.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Nginx Reverse Proxy Config'. Run every code example locally.

.

3 Commit your notes and code samples to GitHub: 'Week 73 - Day 2 Learn: Nginx Reverse Proxy Config'.

.

Output Working code examples for 'Nginx Reverse Proxy Config' committed to GitHub.

:

Day 507	Wednesday	Practice Full Stack + Electronics	Connection
---------	-----------	-----------------------------------	------------

1 Write 3 short coding exercises for 'Nginx Reverse Proxy Config' in VS Code. Each exercise tests one specific concept.

.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.

.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Connection Pooling' from memory.

.

Output 3 working code files for 'Nginx Reverse Proxy Config' on GitHub. 'Connection Pooling' circuit sketched or simulated.

:

Day 508	Thursday	Build Electronics + Full Stack	Connection
---------	----------	--------------------------------	------------

1 Electronics (45 min): Wire the full 'Connection Pooling' circuit on your breadboard. Test with multimeter. Troubleshoot until working.

.

2 Full Stack (60 min): Build the 'Nginx Reverse Proxy Config' feature inside your week's running project. Real, working code.

.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

.

Output Working 'Connection Pooling' circuit (photographed) + 'Nginx Reverse Proxy Config' feature committed to GitHub.

:

Day 509	Friday	Debug Electronics + Full Stack	Connection
---------	--------	--------------------------------	------------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Connection Pooling' circuit. Fix each one.

.

Document them.

2

Full Stack (45 min): Review your 'Nginx Reverse Proxy Config' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Connection Pooling' circuit verified. 'Nginx Reverse Proxy Config' code with error handling on GitHub.

:

Day 510

Saturday

Project — Project Day

Connection

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Connection Pooling' + 'Nginx Reverse Proxy Config'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 73 project milestone'.

.

Output

Working project milestone: 'Connection Pooling' data visible via 'Nginx Reverse Proxy Config' layer. Pushed to GitHub.

:

Day 511

Sunday

Review — Review Day

Connection

1

Electronics (20 min): Re-draw your 'Connection Pooling' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Nginx Reverse Proxy Config' out loud for 5 minutes (rubber duck method). Update your Week 73 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 73 code committed. README updated with learnings and next-week goals.

:

Week 74

Electronics: MongoDB & Mongoose | Full Stack: HTTPS with Let's Encrypt

Day 512 **Monday** **Learn Electronics** MongoDB

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'MongoDB & Mongoose'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'MongoDB & Mongoose'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-74/README.md with today's topic.

Output Notes page on 'MongoDB & Mongoose' + GitHub README stub created for Week 74.

Day 513 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'HTTPS with Let's Encrypt'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'HTTPS with Let's Encrypt'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 74 - Day 2 Learn: HTTPS with Let's Encrypt'.

Output Working code examples for 'HTTPS with Let's Encrypt' committed to GitHub.

Day 514 **Wednesday** **Practice Full Stack + Electronics** MongoDB

- 1 Write 3 short coding exercises for 'HTTPS with Let's Encrypt' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'MongoDB & Mongoose' from memory.

Output 3 working code files for 'HTTPS with Let's Encrypt' on GitHub. 'MongoDB & Mongoose' circuit sketched or simulated.

Day 515 **Thursday** **Build Electronics + Full Stack** MongoDB

- 1 Electronics (45 min): Wire the full 'MongoDB & Mongoose' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'HTTPS with Let's Encrypt' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'MongoDB & Mongoose' circuit (photographed) + 'HTTPS with Let's Encrypt' feature committed to GitHub.

Day 516 **Friday** **Debug Electronics + Full Stack** MongoDB

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'MongoDB & Mongoose' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'HTTPS with Let's Encrypt' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'MongoDB & Mongoose' circuit verified. 'HTTPS with Let's Encrypt' code with error handling on GitHub.

Day 517

Saturday

Project — Project Day

MongoDB

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'MongoDB & Mongoose' + 'HTTPS with Let's Encrypt'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 74 project milestone'.

.

Output

Working project milestone: 'MongoDB & Mongoose' data visible via 'HTTPS with Let's Encrypt' layer. Pushed to GitHub.

:

Day 518

Sunday

Review — Review Day

MongoDB

1

Electronics (20 min): Re-draw your 'MongoDB & Mongoose' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'HTTPS with Let's Encrypt' out loud for 5 minutes (rubber duck method). Update your Week 74 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 74 code committed. README updated with learnings and next-week goals.

:

Week 75

Electronics: Database Backup & Restore | Full Stack: Environment Management & Secrets

Day 519 **Monday** **Learn Electronics** Database

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Database Backup & Restore'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Database Backup & Restore'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-75/README.md with today's topic.

Output Notes page on 'Database Backup & Restore' + GitHub README stub created for Week 75.

Day 520 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Environment Management & Secrets'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Environment Management & Secrets'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 75 - Day 2 Learn: Environment Management & Secrets'.

Output Working code examples for 'Environment Management & Secrets' committed to GitHub.

Day 521 **Wednesday** **Practice Full Stack + Electronics** Database

- 1 Write 3 short coding exercises for 'Environment Management & Secrets' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Database Backup & Restore' from memory.

Output 3 working code files for 'Environment Management & Secrets' on GitHub. 'Database Backup & Restore' circuit sketched or simulated.

Day 522 **Thursday** **Build Electronics + Full Stack** Database

- 1 Electronics (45 min): Wire the full 'Database Backup & Restore' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Environment Management & Secrets' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Database Backup & Restore' circuit (photographed) + 'Environment Management & Secrets' feature committed to GitHub.

Day 523 **Friday** **Debug Electronics + Full Stack** Database

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Database Backup & Restore' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Environment Management & Secrets' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output

Bug log in README. 'Database Backup & Restore' circuit verified. 'Environment Management & Secrets' code with error handling on GitHub.

Day 524

Saturday

Project — Project Day

Database

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Database Backup & Restore' + 'Environment Management & Secrets'.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 75 project milestone'.

Output

Working project milestone: 'Database Backup & Restore' data visible via 'Environment Management & Secrets' layer. Pushed to GitHub.

Day 525

Sunday

Review — Review Day

Database

1

Electronics (20 min): Re-draw your 'Database Backup & Restore' circuit from memory without references. Compare to the real circuit. Note what you missed.

2

Full Stack (20 min): Explain 'Environment Management & Secrets' out loud for 5 minutes (rubber duck method). Update your Week 75 README with a learning summary.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output

All Week 75 code committed. README updated with learnings and next-week goals.

Week 76

Electronics: Query Optimisation | Full Stack: Monitoring with Sentry/Uptime

Day 526	Monday	Learn Electronics	Query
---------	--------	-------------------	-------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Query Optimisation'. Pause every 10 min and write key takeaways.

2 Read allaboutcircuits.com for 'Query Optimisation'. Copy 3 key formulas or facts into your notes.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-76/README.md with today's topic.

Output Notes page on 'Query Optimisation' + GitHub README stub created for Week 76.

:

Day 527	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Monitoring with Sentry/Uptime'. Code along in VS Code — don't just watch.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Monitoring with Sentry/Uptime'. Run every code example locally.

3 Commit your notes and code samples to GitHub: 'Week 76 - Day 2 Learn: Monitoring with Sentry/Uptime'.

Output Working code examples for 'Monitoring with Sentry/Uptime' committed to GitHub.

:

Day 528	Wednesday	Practice Full Stack + Electronics	Query
---------	-----------	-----------------------------------	-------

1 Write 3 short coding exercises for 'Monitoring with Sentry/Uptime' in VS Code. Each exercise tests one specific concept.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Query Optimisation' from memory.

Output 3 working code files for 'Monitoring with Sentry/Uptime' on GitHub. 'Query Optimisation' circuit sketched or simulated.

:

Day 529	Thursday	Build Electronics + Full Stack	Query
---------	----------	--------------------------------	-------

1 Electronics (45 min): Wire the full 'Query Optimisation' circuit on your breadboard. Test with multimeter. Troubleshoot until working.

2 Full Stack (60 min): Build the 'Monitoring with Sentry/Uptime' feature inside your week's running project. Real, working code.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Query Optimisation' circuit (photographed) + 'Monitoring with Sentry/Uptime' feature committed to GitHub.

:

Day 530	Friday	Debug Electronics + Full Stack	Query
---------	--------	--------------------------------	-------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Query Optimisation' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Monitoring with Sentry/Uptime' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Query Optimisation' circuit verified. 'Monitoring with Sentry/Uptime' code with error handling on GitHub.

Day 531

Saturday

Project — Project Day

Query

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Query Optimisation' + 'Monitoring with Sentry/Uptime'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 76 project milestone'.

.

Output

Working project milestone: 'Query Optimisation' data visible via 'Monitoring with Sentry/Uptime' layer. Pushed to GitHub.

Day 532

Sunday

Review — Review Day

Query

1

Electronics (20 min): Re-draw your 'Query Optimisation' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Monitoring with Sentry/Uptime' out loud for 5 minutes (rubber duck method). Update your Week 76 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 76 code committed. README updated with learnings and next-week goals.

Week 77

Electronics: Buffer Week: Phase 5 Review | Full Stack: Buffer Week: Catch Up & Refactor

Day 533	Monday	Learn / Review	Buffer
---------	--------	----------------	--------

1 Review all electronics notes from the past 4 weeks. Re-draw 3 circuits from memory without looking at your notes.

.

2 Read through your GitHub commits for the past 4 weeks. Identify the 2 weakest areas.

.

3 Write a 'Mid-Phase Reflection' entry in your week README: what clicked, what is still unclear.

.

Output Electronics review complete. 2 weak areas identified and noted in README.

:

Day 534	Tuesday	Review / Refactor	Buffer
---------	---------	-------------------	--------

1 Review all Full Stack code from the past 4 weeks. Pick the project you are least proud of.

.

2 Refactor it: improve naming, add comments, fix any errors, add error handling.

.

3 Commit refactored code with message: 'refactor: buffer week improvements'.

.

Output Refactored code committed. README updated with improvement notes.

:

Day 535	Wednesday	Catch Up
---------	-----------	----------

1 Finish any incomplete mini-projects or exercises from the past 4 weeks.

.

2 Push all pending code to GitHub. Ensure every week folder has a complete README.

.

3 Run LeetCode: solve 3 problems you struggled with before. Time yourself.

.

Output All pending work committed. LeetCode practice done.

:

Day 536	Thursday	Deep Dive
---------	----------	-----------

1 Pick the hardest concept from the past 4 weeks. Spend the full session going deeper on it.

.

2 Build a tiny demo project that specifically exercises that concept. No tutorials.

.

3 Push the demo to GitHub with a clear README explaining what you explored.

.

Output Deep dive demo project committed to GitHub.

:

Day 537	Friday	System Design
---------	--------	---------------

1 Study system design: read one chapter of system-design-primer (GitHub, free) or watch one ByteByteGo video.

.

2 Draw the architecture diagram for your phase project. Label every component and connection.

.

3 Write the architecture explanation in your README as if explaining it to a junior developer.

.

Output Architecture diagram drawn and explained in README.

:

Day 538

Saturday

Phase Project Prep

Buffer

1 Plan the upcoming phase project in detail: list all components, features, and acceptance criteria.

.

2 Set up the project repository structure. Create all folders and stub files.

.

3 Order any missing hardware components. Write the hardware shopping list in README.

.

Output Phase project repo created with full plan in README. Hardware ordered.

:

Day 539

Sunday

Rest & Plan

1 Rest fully. No coding, no circuits. Sustainable learning requires recovery.

.

2 Optional: review next week's topics for 15 min. Bookmark 1-2 key resources.

.

3 Write 3 clear goals for the next 4 weeks at the bottom of your buffer README.

.

Output Well rested. 3 goals for next phase written. Resources bookmarked.

:

Week 78

Electronics: Full DB + Docker Deploy Build | Full Stack: Complete Cloud Deploy Build

MILESTONE PROJECT: Full DB + Docker Deployment + Complete Cloud Deploy Project

Day 540	Monday	Plan & Design	
1	Design the full project architecture: draw the hardware circuit and software stack on paper.		
.			
2	List all required components for 'Full DB + Docker Deploy Build' and 'Complete Cloud Deploy Build'. Verify you have everything.		
.			
3	Set up the project GitHub repo. Create README with: project goal, tech stack, architecture diagram placeholder.		
.			
Output	Architecture designed. Project repo created. README outline committed.		
:			
Day 541	Tuesday	Build Core	Full
1	Build the core hardware component: 'Full DB + Docker Deploy Build'. Test it works in isolation with multimeter.		
.			
2	Set up the core software scaffold: 'Complete Cloud Deploy Build'. Create all files and folder structure.		
.			
3	Connect hardware and software at the most basic level. Verify data flows correctly.		
.			
Output	Core 'Full DB + Docker Deploy Build' hardware working. 'Complete Cloud Deploy Build' scaffold committed to GitHub.		
:			
Day 542	Wednesday	Build Features	Full
1	Implement the main feature of 'Full DB + Docker Deploy Build'. Document every step in README.		
.			
2	Build the primary user-facing feature of 'Complete Cloud Deploy Build'. Write clean, commented code.		
.			
3	Integration check: hardware + software + web layer all communicating correctly.		
.			
Output	Main feature built and integrated. Code committed with documentation.		
:			
Day 543	Thursday	Build Full	Full
1	Complete all remaining features for 'Full DB + Docker Deploy Build'. Photograph every stage.		
.			
2	Complete all remaining features for 'Complete Cloud Deploy Build'. Add proper error handling.		
.			
3	End-to-end test: full pipeline from hardware input to web output. Fix all issues.		
.			
Output	Full project working end-to-end. All code committed to GitHub.		
:			
Day 544	Friday	Debug & Polish	Full

- 1 Stress test the project: run it for 30 minutes continuously. Log any failures.
.
- 2 Fix all bugs found. Add input validation and error messages. Improve UX.
.
- 3 Code review: clean up all magic numbers, add comments, ensure consistent naming.
.

Output Project stable under load. All bugs fixed. Code cleaned and committed.

:

Day 545 Saturday Document & Demo

- 1 Write comprehensive README: project overview, setup guide, usage instructions, photos.
.
- 2 Record a 2-minute demo video showing the full working project.
.
- 3 Add the project to your portfolio page. Write a 100-word project description.
.

Output README complete. Demo video recorded. Portfolio updated.

:

Day 546 Sunday Launch & Reflect

- 1 Final commit with version tag v1.0.0. Push everything to GitHub.
.
- 2 Write a 200-word reflection: what you built, what you learned, what you'd do differently.
.
- 3 Share the GitHub link with someone (social media, forum, friend). External accountability matters.
.

Output Project v1.0.0 released. Reflection written. GitHub link shared.

:

Week 79

Electronics: Cloud Deploy Finalise | Full Stack: Cloud Deploy Polish & Monitor

Day 547	Monday	Learn Electronics	Cloud
---------	--------	-------------------	-------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Cloud Deploy Finalise'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Cloud Deploy Finalise'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-79/README.md with today's topic.

Output Notes page on 'Cloud Deploy Finalise' + GitHub README stub created for Week 79.

Day 548	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Cloud Deploy Polish & Monitor'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Cloud Deploy Polish & Monitor'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 79 - Day 2 Learn: Cloud Deploy Polish & Monitor'.

Output Working code examples for 'Cloud Deploy Polish & Monitor' committed to GitHub.

Day 549	Wednesday	Practice Full Stack + Electronics	Cloud
---------	-----------	-----------------------------------	-------

- 1 Write 3 short coding exercises for 'Cloud Deploy Polish & Monitor' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Cloud Deploy Finalise' from memory.

Output 3 working code files for 'Cloud Deploy Polish & Monitor' on GitHub. 'Cloud Deploy Finalise' circuit sketched or simulated.

Day 550	Thursday	Build Electronics + Full Stack	Cloud
---------	----------	--------------------------------	-------

- 1 Electronics (45 min): Wire the full 'Cloud Deploy Finalise' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Cloud Deploy Polish & Monitor' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Cloud Deploy Finalise' circuit (photographed) + 'Cloud Deploy Polish & Monitor' feature committed to GitHub.

Day 551	Friday	Debug Electronics + Full Stack	Cloud
---------	--------	--------------------------------	-------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Cloud Deploy Finalise' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Cloud Deploy Polish & Monitor' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Cloud Deploy Finalise' circuit verified. 'Cloud Deploy Polish & Monitor' code with error handling on GitHub.

Day 552

Saturday

Project — Project Day

Cloud

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Cloud Deploy Finalise' + 'Cloud Deploy Polish & Monitor'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 79 project milestone'.

.

Output

Working project milestone: 'Cloud Deploy Finalise' data visible via 'Cloud Deploy Polish & Monitor' layer. Pushed to GitHub.

Day 553

Sunday

Review — Review Day

Cloud

1

Electronics (20 min): Re-draw your 'Cloud Deploy Finalise' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Cloud Deploy Polish & Monitor' out loud for 5 minutes (rubber duck method). Update your Week 79 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 79 code committed. README updated with learnings and next-week goals.

Week 80

Electronics: Phase 5 Wrap-Up & Prep | Full Stack: Phase 5 Wrap-Up & Prep

Day 554	Monday	Learn Electronics	Phase
---------	--------	-------------------	-------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Phase 5 Wrap-Up & Prep'. Pause every 10 min and write key takeaways.
.

2 Read allaboutcircuits.com for 'Phase 5 Wrap-Up & Prep'. Copy 3 key formulas or facts into your notes.
.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-80/README.md with today's topic.
.

Output Notes page on 'Phase 5 Wrap-Up & Prep' + GitHub README stub created for Week 80.
:

Day 555	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Phase 5 Wrap-Up & Prep'. Code along in VS Code — don't just watch.
.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Phase 5 Wrap-Up & Prep'. Run every code example locally.
.

3 Commit your notes and code samples to GitHub: 'Week 80 - Day 2 Learn: Phase 5 Wrap-Up & Prep'.
.

Output Working code examples for 'Phase 5 Wrap-Up & Prep' committed to GitHub.
:

Day 556	Wednesday	Practice Full Stack + Electronics	Phase
---------	-----------	-----------------------------------	-------

1 Write 3 short coding exercises for 'Phase 5 Wrap-Up & Prep' in VS Code. Each exercise tests one specific concept.
.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Phase 5 Wrap-Up & Prep' from memory.
.

Output 3 working code files for 'Phase 5 Wrap-Up & Prep' on GitHub. 'Phase 5 Wrap-Up & Prep' circuit sketched or simulated.
:

Day 557	Thursday	Build Electronics + Full Stack	Phase
---------	----------	--------------------------------	-------

1 Electronics (45 min): Wire the full 'Phase 5 Wrap-Up & Prep' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
.

2 Full Stack (60 min): Build the 'Phase 5 Wrap-Up & Prep' feature inside your week's running project. Real, working code.
.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
.

Output Working 'Phase 5 Wrap-Up & Prep' circuit (photographed) + 'Phase 5 Wrap-Up & Prep' feature committed to GitHub.
:

Day 558	Friday	Debug Electronics + Full Stack	Phase
---------	--------	--------------------------------	-------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Phase 5 Wrap-Up & Prep' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Phase 5 Wrap-Up & Prep' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output

Bug log in README. 'Phase 5 Wrap-Up & Prep' circuit verified. 'Phase 5 Wrap-Up & Prep' code with error handling on GitHub.

Day 559

Saturday

Project — Project Day

Phase

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Phase 5 Wrap-Up & Prep' + 'Phase 5 Wrap-Up & Prep'.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 80 project milestone'.

Output

Working project milestone: 'Phase 5 Wrap-Up & Prep' data visible via 'Phase 5 Wrap-Up & Prep' layer. Pushed to GitHub.

Day 560

Sunday

Review — Review Day

Phase

1

Electronics (20 min): Re-draw your 'Phase 5 Wrap-Up & Prep' circuit from memory without references. Compare to the real circuit. Note what you missed.

2

Full Stack (20 min): Explain 'Phase 5 Wrap-Up & Prep' out loud for 5 minutes (rubber duck method). Update your Week 80 README with a learning summary.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output

All Week 80 code committed. README updated with learnings and next-week goals.

Phase

6

MQTT + IoT Integration + Final Product

Months 21–24 · Weeks 81–104 · Days 561–730

IoT: MQTT Deep Dive, Mosquitto, QoS, TFLite Edge AI, KiCad PCB Design

Full Stack: TypeScript, Next.js SSR/SSG, NextAuth, Stripe, SaaS Architecture, k6

Milestone: Final IoT SaaS Product — Custom PCB + MQTT + Next.js + Stripe

Week 81

Electronics: MQTT Protocol Deep Dive | Full Stack: TypeScript Fundamentals

Day 561	Monday	Learn Electronics	MQTT
---------	--------	-------------------	------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'MQTT Protocol Deep Dive'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'MQTT Protocol Deep Dive'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-81/README.md with today's topic.
- Output** Notes page on 'MQTT Protocol Deep Dive' + GitHub README stub created for Week 81.

Day 562	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'TypeScript Fundamentals'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'TypeScript Fundamentals'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 81 - Day 2 Learn: TypeScript Fundamentals'.
- Output** Working code examples for 'TypeScript Fundamentals' committed to GitHub.

Day 563	Wednesday	Practice Full Stack + Electronics	MQTT
---------	-----------	-----------------------------------	------

- 1 Write 3 short coding exercises for 'TypeScript Fundamentals' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'MQTT Protocol Deep Dive' from memory.
- Output** 3 working code files for 'TypeScript Fundamentals' on GitHub. 'MQTT Protocol Deep Dive' circuit sketched or simulated.

Day 564	Thursday	Build Electronics + Full Stack	MQTT
---------	----------	--------------------------------	------

- 1 Electronics (45 min): Wire the full 'MQTT Protocol Deep Dive' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'TypeScript Fundamentals' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'MQTT Protocol Deep Dive' circuit (photographed) + 'TypeScript Fundamentals' feature committed to GitHub.

Day 565	Friday	Debug Electronics + Full Stack	MQTT
---------	--------	--------------------------------	------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'MQTT Protocol Deep Dive' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'TypeScript Fundamentals' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'MQTT Protocol Deep Dive' circuit verified. 'TypeScript Fundamentals' code with error handling on GitHub.

Day 566

Saturday

Project — Project Day

MQTT

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'MQTT Protocol Deep Dive' + 'TypeScript Fundamentals'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 81 project milestone'.

.

Output

Working project milestone: 'MQTT Protocol Deep Dive' data visible via 'TypeScript Fundamentals' layer. Pushed to GitHub.

Day 567

Sunday

Review — Review Day

MQTT

1

Electronics (20 min): Re-draw your 'MQTT Protocol Deep Dive' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'TypeScript Fundamentals' out loud for 5 minutes (rubber duck method). Update your Week 81 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 81 code committed. README updated with learnings and next-week goals.

Week 82

Electronics: Mosquitto Broker on Pi | Full Stack: TypeScript with Express

Day 568	Monday	Learn Electronics	Mosquitto
---------	--------	-------------------	-----------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Mosquitto Broker on Pi'. Pause every 10 min and write key takeaways.

.

2 Read allaboutcircuits.com for 'Mosquitto Broker on Pi'. Copy 3 key formulas or facts into your notes.

.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-82/README.md with today's topic.

.

Output Notes page on 'Mosquitto Broker on Pi' + GitHub README stub created for Week 82.

:

Day 569	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'TypeScript with Express'. Code along in VS Code — don't just watch.

.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'TypeScript with Express'. Run every code example locally.

.

3 Commit your notes and code samples to GitHub: 'Week 82 - Day 2 Learn: TypeScript with Express'.

.

Output Working code examples for 'TypeScript with Express' committed to GitHub.

:

Day 570	Wednesday	Practice Full Stack + Electronics	Mosquitto
---------	-----------	-----------------------------------	-----------

1 Write 3 short coding exercises for 'TypeScript with Express' in VS Code. Each exercise tests one specific concept.

.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.

.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Mosquitto Broker on Pi' from memory.

.

Output 3 working code files for 'TypeScript with Express' on GitHub. 'Mosquitto Broker on Pi' circuit sketched or simulated.

:

Day 571	Thursday	Build Electronics + Full Stack	Mosquitto
---------	----------	--------------------------------	-----------

1 Electronics (45 min): Wire the full 'Mosquitto Broker on Pi' circuit on your breadboard. Test with multimeter. Troubleshoot until working.

.

2 Full Stack (60 min): Build the 'TypeScript with Express' feature inside your week's running project. Real, working code.

.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

.

Output Working 'Mosquitto Broker on Pi' circuit (photographed) + 'TypeScript with Express' feature committed to GitHub.

:

Day 572	Friday	Debug Electronics + Full Stack	Mosquitto
---------	--------	--------------------------------	-----------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Mosquitto Broker on Pi' circuit. Fix each one.

.

Document them.

2

Full Stack (45 min): Review your 'TypeScript with Express' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Mosquitto Broker on Pi' circuit verified. 'TypeScript with Express' code with error handling on GitHub.

:

Day 573

Saturday

Project — Project Day

Mosquitto

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Mosquitto Broker on Pi' + 'TypeScript with Express'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 82 project milestone'.

.

Output

Working project milestone: 'Mosquitto Broker on Pi' data visible via 'TypeScript with Express' layer. Pushed to GitHub.

:

Day 574

Sunday

Review — Review Day

Mosquitto

1

Electronics (20 min): Re-draw your 'Mosquitto Broker on Pi' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'TypeScript with Express' out loud for 5 minutes (rubber duck method). Update your Week 82 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 82 code committed. README updated with learnings and next-week goals.

:

Week 83

Electronics: MQTT QoS Levels & Topics | Full Stack: Next.js Introduction & Routing

Day 575 **Monday** **Learn Electronics** MQTT

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'MQTT QoS Levels & Topics'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'MQTT QoS Levels & Topics'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-83/README.md with today's topic.

Output Notes page on 'MQTT QoS Levels & Topics' + GitHub README stub created for Week 83.

Day 576 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Next.js Introduction & Routing'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Next.js Introduction & Routing'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 83 - Day 2 Learn: Next.js Introduction & Routing'.

Output Working code examples for 'Next.js Introduction & Routing' committed to GitHub.

Day 577 **Wednesday** **Practice Full Stack + Electronics** MQTT

- 1 Write 3 short coding exercises for 'Next.js Introduction & Routing' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'MQTT QoS Levels & Topics' from memory.

Output 3 working code files for 'Next.js Introduction & Routing' on GitHub. 'MQTT QoS Levels & Topics' circuit sketched or simulated.

Day 578 **Thursday** **Build Electronics + Full Stack** MQTT

- 1 Electronics (45 min): Wire the full 'MQTT QoS Levels & Topics' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Next.js Introduction & Routing' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'MQTT QoS Levels & Topics' circuit (photographed) + 'Next.js Introduction & Routing' feature committed to GitHub.

Day 579 **Friday** **Debug Electronics + Full Stack** MQTT

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'MQTT QoS Levels & Topics' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Next.js Introduction & Routing' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'MQTT QoS Levels & Topics' circuit verified. 'Next.js Introduction & Routing' code with error handling on GitHub.

Day 580

Saturday

Project — Project Day

MQTT

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'MQTT QoS Levels & Topics' + 'Next.js Introduction & Routing'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 83 project milestone'.

.

Output

Working project milestone: 'MQTT QoS Levels & Topics' data visible via 'Next.js Introduction & Routing' layer. Pushed to GitHub.

Day 581

Sunday

Review — Review Day

MQTT

1

Electronics (20 min): Re-draw your 'MQTT QoS Levels & Topics' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Next.js Introduction & Routing' out loud for 5 minutes (rubber duck method). Update your Week 83 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 83 code committed. README updated with learnings and next-week goals.

Week 84

Electronics: MQTT Retained Messages | Full Stack: Next.js SSR & SSG

Day 582 **Monday** **Learn Electronics** MQTT

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'MQTT Retained Messages'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'MQTT Retained Messages'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-84/README.md with today's topic.

Output Notes page on 'MQTT Retained Messages' + GitHub README stub created for Week 84.

Day 583 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Next.js SSR & SSG'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Next.js SSR & SSG'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 84 - Day 2 Learn: Next.js SSR & SSG'.

Output Working code examples for 'Next.js SSR & SSG' committed to GitHub.

Day 584 **Wednesday** **Practice Full Stack + Electronics** MQTT

- 1 Write 3 short coding exercises for 'Next.js SSR & SSG' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'MQTT Retained Messages' from memory.

Output 3 working code files for 'Next.js SSR & SSG' on GitHub. 'MQTT Retained Messages' circuit sketched or simulated.

Day 585 **Thursday** **Build Electronics + Full Stack** MQTT

- 1 Electronics (45 min): Wire the full 'MQTT Retained Messages' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Next.js SSR & SSG' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'MQTT Retained Messages' circuit (photographed) + 'Next.js SSR & SSG' feature committed to GitHub.

Day 586 **Friday** **Debug Electronics + Full Stack** MQTT

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'MQTT Retained Messages' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Next.js SSR & SSG' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'MQTT Retained Messages' circuit verified. 'Next.js SSR & SSG' code with error handling on GitHub.

:

Day 587

Saturday

Project — Project Day

MQTT

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'MQTT Retained Messages' + 'Next.js SSR & SSG'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 84 project milestone'.

.

Output

Working project milestone: 'MQTT Retained Messages' data visible via 'Next.js SSR & SSG' layer. Pushed to GitHub.

:

Day 588

Sunday

Review — Review Day

MQTT

1

Electronics (20 min): Re-draw your 'MQTT Retained Messages' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Next.js SSR & SSG' out loud for 5 minutes (rubber duck method). Update your Week 84 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 84 code committed. README updated with learnings and next-week goals.

:

Week 85

Electronics: MQTT Bridge & Federation | Full Stack: Next.js API Routes

Day 589 **Monday** **Learn Electronics** MQTT

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'MQTT Bridge & Federation'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'MQTT Bridge & Federation'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-85/README.md with today's topic.
- Output** Notes page on 'MQTT Bridge & Federation' + GitHub README stub created for Week 85.

Day 590 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Next.js API Routes'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Next.js API Routes'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 85 - Day 2 Learn: Next.js API Routes'.
- Output** Working code examples for 'Next.js API Routes' committed to GitHub.

Day 591 **Wednesday** **Practice Full Stack + Electronics** MQTT

- 1 Write 3 short coding exercises for 'Next.js API Routes' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'MQTT Bridge & Federation' from memory.
- Output** 3 working code files for 'Next.js API Routes' on GitHub. 'MQTT Bridge & Federation' circuit sketched or simulated.

Day 592 **Thursday** **Build Electronics + Full Stack** MQTT

- 1 Electronics (45 min): Wire the full 'MQTT Bridge & Federation' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'Next.js API Routes' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'MQTT Bridge & Federation' circuit (photographed) + 'Next.js API Routes' feature committed to GitHub.

Day 593 **Friday** **Debug Electronics + Full Stack** MQTT

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'MQTT Bridge & Federation' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Next.js API Routes' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'MQTT Bridge & Federation' circuit verified. 'Next.js API Routes' code with error handling on GitHub.

:

Day 594

Saturday

Project — Project Day

MQTT

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'MQTT Bridge & Federation' + 'Next.js API Routes'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 85 project milestone'.

.

Output

Working project milestone: 'MQTT Bridge & Federation' data visible via 'Next.js API Routes' layer. Pushed to GitHub.

:

Day 595

Sunday

Review — Review Day

MQTT

1

Electronics (20 min): Re-draw your 'MQTT Bridge & Federation' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Next.js API Routes' out loud for 5 minutes (rubber duck method). Update your Week 85 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 85 code committed. README updated with learnings and next-week goals.

:

Week 86

Electronics: Edge AI with TFLite | Full Stack: Stripe Payments Integration

Day 596	Monday	Learn Electronics	Edge
---------	--------	-------------------	------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Edge AI with TFLite'. Pause every 10 min and write key takeaways.

.

2 Read allaboutcircuits.com for 'Edge AI with TFLite'. Copy 3 key formulas or facts into your notes.

.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-86/README.md with today's topic.

.

Output Notes page on 'Edge AI with TFLite' + GitHub README stub created for Week 86.

:

Day 597	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Stripe Payments Integration'. Code along in VS Code — don't just watch.

.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Stripe Payments Integration'. Run every code example locally.

.

3 Commit your notes and code samples to GitHub: 'Week 86 - Day 2 Learn: Stripe Payments Integration'.

.

Output Working code examples for 'Stripe Payments Integration' committed to GitHub.

:

Day 598	Wednesday	Practice Full Stack + Electronics	Edge
---------	-----------	-----------------------------------	------

1 Write 3 short coding exercises for 'Stripe Payments Integration' in VS Code. Each exercise tests one specific concept.

.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.

.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Edge AI with TFLite' from memory.

.

Output 3 working code files for 'Stripe Payments Integration' on GitHub. 'Edge AI with TFLite' circuit sketched or simulated.

:

Day 599	Thursday	Build Electronics + Full Stack	Edge
---------	----------	--------------------------------	------

1 Electronics (45 min): Wire the full 'Edge AI with TFLite' circuit on your breadboard. Test with multimeter. Troubleshoot until working.

.

2 Full Stack (60 min): Build the 'Stripe Payments Integration' feature inside your week's running project. Real, working code.

.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

.

Output Working 'Edge AI with TFLite' circuit (photographed) + 'Stripe Payments Integration' feature committed to GitHub.

:

Day 600	Friday	Debug Electronics + Full Stack	Edge
---------	--------	--------------------------------	------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Edge AI with TFLite' circuit. Fix each one.

.

Document them.

2

Full Stack (45 min): Review your 'Stripe Payments Integration' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Edge AI with TFLite' circuit verified. 'Stripe Payments Integration' code with error handling on GitHub.

:

Day 601

Saturday

Project — Project Day

Edge

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Edge AI with TFLite' + 'Stripe Payments Integration'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 86 project milestone'.

.

Output

Working project milestone: 'Edge AI with TFLite' data visible via 'Stripe Payments Integration' layer. Pushed to GitHub.

:

Day 602

Sunday

Review — Review Day

Edge

1

Electronics (20 min): Re-draw your 'Edge AI with TFLite' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Stripe Payments Integration' out loud for 5 minutes (rubber duck method). Update your Week 86 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 86 code committed. README updated with learnings and next-week goals.

:

Week 87

Electronics: PCB Design with KiCad | Full Stack: Next.js Auth with NextAuth.js

Day 603 **Monday** **Learn Electronics** **PCB**

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'PCB Design with KiCad'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'PCB Design with KiCad'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-87/README.md with today's topic.

Output Notes page on 'PCB Design with KiCad' + GitHub README stub created for Week 87.

Day 604 **Tuesday** **Learn Full Stack**

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Next.js Auth with NextAuth.js'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Next.js Auth with NextAuth.js'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 87 - Day 2 Learn: Next.js Auth with NextAuth.js'.

Output Working code examples for 'Next.js Auth with NextAuth.js' committed to GitHub.

Day 605 **Wednesday** **Practice Full Stack + Electronics** **PCB**

- 1 Write 3 short coding exercises for 'Next.js Auth with NextAuth.js' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'PCB Design with KiCad' from memory.

Output 3 working code files for 'Next.js Auth with NextAuth.js' on GitHub. 'PCB Design with KiCad' circuit sketched or simulated.

Day 606 **Thursday** **Build Electronics + Full Stack** **PCB**

- 1 Electronics (45 min): Wire the full 'PCB Design with KiCad' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Next.js Auth with NextAuth.js' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'PCB Design with KiCad' circuit (photographed) + 'Next.js Auth with NextAuth.js' feature committed to GitHub.

Day 607 **Friday** **Debug Electronics + Full Stack** **PCB**

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'PCB Design with KiCad' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Next.js Auth with NextAuth.js' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'PCB Design with KiCad' circuit verified. 'Next.js Auth with NextAuth.js' code with error handling on GitHub.

:

Day 608

Saturday

Project — Project Day

PCB

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'PCB Design with KiCad' + 'Next.js Auth with NextAuth.js'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 87 project milestone'.

.

Output

Working project milestone: 'PCB Design with KiCad' data visible via 'Next.js Auth with NextAuth.js' layer. Pushed to GitHub.

:

Day 609

Sunday

Review — Review Day

PCB

1

Electronics (20 min): Re-draw your 'PCB Design with KiCad' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Next.js Auth with NextAuth.js' out loud for 5 minutes (rubber duck method). Update your Week 87 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 87 code committed. README updated with learnings and next-week goals.

:

Week 88

Electronics: PCB Layout & DRC Check | Full Stack: SaaS Multi-tenant Architecture

Day 610	Monday	Learn Electronics	PCB
---------	--------	-------------------	-----

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'PCB Layout & DRC Check'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'PCB Layout & DRC Check'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-88/README.md with today's topic.
- Output** Notes page on 'PCB Layout & DRC Check' + GitHub README stub created for Week 88.

Day 611	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'SaaS Multi-tenant Architecture'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'SaaS Multi-tenant Architecture'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 88 - Day 2 Learn: SaaS Multi-tenant Architecture'.
- Output** Working code examples for 'SaaS Multi-tenant Architecture' committed to GitHub.

Day 612	Wednesday	Practice Full Stack + Electronics	PCB
---------	-----------	-----------------------------------	-----

- 1 Write 3 short coding exercises for 'SaaS Multi-tenant Architecture' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'PCB Layout & DRC Check' from memory.
- Output** 3 working code files for 'SaaS Multi-tenant Architecture' on GitHub. 'PCB Layout & DRC Check' circuit sketched or simulated.

Day 613	Thursday	Build Electronics + Full Stack	PCB
---------	----------	--------------------------------	-----

- 1 Electronics (45 min): Wire the full 'PCB Layout & DRC Check' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'SaaS Multi-tenant Architecture' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'PCB Layout & DRC Check' circuit (photographed) + 'SaaS Multi-tenant Architecture' feature committed to GitHub.

Day 614	Friday	Debug Electronics + Full Stack	PCB
---------	--------	--------------------------------	-----

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'PCB Layout & DRC Check' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'SaaS Multi-tenant Architecture' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output

Bug log in README. 'PCB Layout & DRC Check' circuit verified. 'SaaS Multi-tenant Architecture' code with error handling on GitHub.

Day 615

Saturday

Project — Project Day

PCB

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'PCB Layout & DRC Check' + 'SaaS Multi-tenant Architecture'.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 88 project milestone'.

Output

Working project milestone: 'PCB Layout & DRC Check' data visible via 'SaaS Multi-tenant Architecture' layer. Pushed to GitHub.

Day 616

Sunday

Review — Review Day

PCB

1

Electronics (20 min): Re-draw your 'PCB Layout & DRC Check' circuit from memory without references. Compare to the real circuit. Note what you missed.

2

Full Stack (20 min): Explain 'SaaS Multi-tenant Architecture' out loud for 5 minutes (rubber duck method). Update your Week 88 README with a learning summary.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output

All Week 88 code committed. README updated with learnings and next-week goals.

Week 89

Electronics: Gerber Export & Review | Full Stack: WebRTC Basics for Video Streams

Day 617	Monday	Learn Electronics	Gerber
---------	--------	-------------------	--------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Gerber Export & Review'. Pause every 10 min and write key takeaways.

2 Read allaboutcircuits.com for 'Gerber Export & Review'. Copy 3 key formulas or facts into your notes.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-89/README.md with today's topic.

Output Notes page on 'Gerber Export & Review' + GitHub README stub created for Week 89.

:

Day 618	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'WebRTC Basics for Video Streams'. Code along in VS Code — don't just watch.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'WebRTC Basics for Video Streams'. Run every code example locally.

3 Commit your notes and code samples to GitHub: 'Week 89 - Day 2 Learn: WebRTC Basics for Video Streams'.

Output Working code examples for 'WebRTC Basics for Video Streams' committed to GitHub.

:

Day 619	Wednesday	Practice Full Stack + Electronics	Gerber
---------	-----------	-----------------------------------	--------

1 Write 3 short coding exercises for 'WebRTC Basics for Video Streams' in VS Code. Each exercise tests one specific concept.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Gerber Export & Review' from memory.

Output 3 working code files for 'WebRTC Basics for Video Streams' on GitHub. 'Gerber Export & Review' circuit sketched or simulated.

:

Day 620	Thursday	Build Electronics + Full Stack	Gerber
---------	----------	--------------------------------	--------

1 Electronics (45 min): Wire the full 'Gerber Export & Review' circuit on your breadboard. Test with multimeter. Troubleshoot until working.

2 Full Stack (60 min): Build the 'WebRTC Basics for Video Streams' feature inside your week's running project. Real, working code.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Gerber Export & Review' circuit (photographed) + 'WebRTC Basics for Video Streams' feature committed to GitHub.

:

Day 621	Friday	Debug Electronics + Full Stack	Gerber
---------	--------	--------------------------------	--------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Gerber Export & Review' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'WebRTC Basics for Video Streams' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Gerber Export & Review' circuit verified. 'WebRTC Basics for Video Streams' code with error handling on GitHub.

:

Day 622

Saturday

Project — Project Day

Gerber

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Gerber Export & Review' + 'WebRTC Basics for Video Streams'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 89 project milestone'.

.

Output

Working project milestone: 'Gerber Export & Review' data visible via 'WebRTC Basics for Video Streams' layer. Pushed to GitHub.

:

Day 623

Sunday

Review — Review Day

Gerber

1

Electronics (20 min): Re-draw your 'Gerber Export & Review' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'WebRTC Basics for Video Streams' out loud for 5 minutes (rubber duck method). Update your Week 89 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 89 code committed. README updated with learnings and next-week goals.

:

Week 90

Electronics: 3D Enclosure Design | Full Stack: System Design Fundamentals

Day 624 Monday Learn Electronics

3D

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on '3D Enclosure Design'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for '3D Enclosure Design'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-90/README.md with today's topic.

Output Notes page on '3D Enclosure Design' + GitHub README stub created for Week 90.

Day 625 Tuesday Learn Full Stack

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'System Design Fundamentals'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'System Design Fundamentals'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 90 - Day 2 Learn: System Design Fundamentals'.

Output Working code examples for 'System Design Fundamentals' committed to GitHub.

Day 626 Wednesday Practice Full Stack + Electronics

3D

- 1 Write 3 short coding exercises for 'System Design Fundamentals' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for '3D Enclosure Design' from memory.

Output 3 working code files for 'System Design Fundamentals' on GitHub. '3D Enclosure Design' circuit sketched or simulated.

Day 627 Thursday Build Electronics + Full Stack

3D

- 1 Electronics (45 min): Wire the full '3D Enclosure Design' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'System Design Fundamentals' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working '3D Enclosure Design' circuit (photographed) + 'System Design Fundamentals' feature committed to GitHub.

Day 628 Friday Debug Electronics + Full Stack

3D

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your '3D Enclosure Design' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'System Design Fundamentals' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output Bug log in README. '3D Enclosure Design' circuit verified. 'System Design Fundamentals' code with error handling on GitHub.

Day 629 Saturday Project — Project Day

3D

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects '3D Enclosure Design' + 'System Design Fundamentals'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 90 project milestone'.

.

Output Working project milestone: '3D Enclosure Design' data visible via 'System Design Fundamentals' layer. Pushed to GitHub.

Day 630 Sunday Review — Review Day

3D

1 Electronics (20 min): Re-draw your '3D Enclosure Design' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'System Design Fundamentals' out loud for 5 minutes (rubber duck method). Update your Week 90 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output All Week 90 code committed. README updated with learnings and next-week goals.

Week 91

Electronics: Hardware Testing & QA | Full Stack: Load Testing with k6

Day 631	Monday	Learn Electronics	Hardware
---------	--------	-------------------	----------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Hardware Testing & QA'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Hardware Testing & QA'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-91/README.md with today's topic.

Output Notes page on 'Hardware Testing & QA' + GitHub README stub created for Week 91.

Day 632	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Load Testing with k6'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Load Testing with k6'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 91 - Day 2 Learn: Load Testing with k6'.

Output Working code examples for 'Load Testing with k6' committed to GitHub.

Day 633	Wednesday	Practice Full Stack + Electronics	Hardware
---------	-----------	-----------------------------------	----------

- 1 Write 3 short coding exercises for 'Load Testing with k6' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Hardware Testing & QA' from memory.

Output 3 working code files for 'Load Testing with k6' on GitHub. 'Hardware Testing & QA' circuit sketched or simulated.

Day 634	Thursday	Build Electronics + Full Stack	Hardware
---------	----------	--------------------------------	----------

- 1 Electronics (45 min): Wire the full 'Hardware Testing & QA' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Load Testing with k6' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Hardware Testing & QA' circuit (photographed) + 'Load Testing with k6' feature committed to GitHub.

Day 635	Friday	Debug Electronics + Full Stack	Hardware
---------	--------	--------------------------------	----------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Hardware Testing & QA' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Load Testing with k6' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Hardware Testing & QA' circuit verified. 'Load Testing with k6' code with error handling on GitHub.

:

Day 636

Saturday

Project — Project Day

Hardware

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Hardware Testing & QA' + 'Load Testing with k6'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 91 project milestone'.

.

Output

Working project milestone: 'Hardware Testing & QA' data visible via 'Load Testing with k6' layer. Pushed to GitHub.

:

Day 637

Sunday

Review — Review Day

Hardware

1

Electronics (20 min): Re-draw your 'Hardware Testing & QA' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Load Testing with k6' out loud for 5 minutes (rubber duck method). Update your Week 91 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 91 code committed. README updated with learnings and next-week goals.

:

Week 92

Electronics: Full IoT System Integration | Full Stack: Final App Polish & SEO

Day 638	Monday	Learn Electronics	Full
---------	--------	-------------------	------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Full IoT System Integration'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'Full IoT System Integration'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-92/README.md with today's topic.
- Output** Notes page on 'Full IoT System Integration' + GitHub README stub created for Week 92.

Day 639	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Final App Polish & SEO'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Final App Polish & SEO'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 92 - Day 2 Learn: Final App Polish & SEO'.
- Output** Working code examples for 'Final App Polish & SEO' committed to GitHub.

Day 640	Wednesday	Practice Full Stack + Electronics	Full
---------	-----------	-----------------------------------	------

- 1 Write 3 short coding exercises for 'Final App Polish & SEO' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Full IoT System Integration' from memory.
- Output** 3 working code files for 'Final App Polish & SEO' on GitHub. 'Full IoT System Integration' circuit sketched or simulated.

Day 641	Thursday	Build Electronics + Full Stack	Full
---------	----------	--------------------------------	------

- 1 Electronics (45 min): Wire the full 'Full IoT System Integration' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'Final App Polish & SEO' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'Full IoT System Integration' circuit (photographed) + 'Final App Polish & SEO' feature committed to GitHub.

Day 642	Friday	Debug Electronics + Full Stack	Full
---------	--------	--------------------------------	------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Full IoT System Integration' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Final App Polish & SEO' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Full IoT System Integration' circuit verified. 'Final App Polish & SEO' code with error handling on GitHub.

:

Day 643

Saturday

Project — Project Day

Full

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Full IoT System Integration' + 'Final App Polish & SEO'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 92 project milestone'.

.

Output

Working project milestone: 'Full IoT System Integration' data visible via 'Final App Polish & SEO' layer. Pushed to GitHub.

:

Day 644

Sunday

Review — Review Day

Full

1

Electronics (20 min): Re-draw your 'Full IoT System Integration' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Final App Polish & SEO' out loud for 5 minutes (rubber duck method). Update your Week 92 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 92 code committed. README updated with learnings and next-week goals.

:

Week 93

Electronics: Buffer Week: Phase 6 Review | Full Stack: Buffer Week: Catch Up & Refactor

Day 645	Monday	Learn / Review	Buffer
---------	--------	----------------	--------

1 Review all electronics notes from the past 4 weeks. Re-draw 3 circuits from memory without looking at your notes.

.

2 Read through your GitHub commits for the past 4 weeks. Identify the 2 weakest areas.

.

3 Write a 'Mid-Phase Reflection' entry in your week README: what clicked, what is still unclear.

.

Output Electronics review complete. 2 weak areas identified and noted in README.

:

Day 646	Tuesday	Review / Refactor	Buffer
---------	---------	-------------------	--------

1 Review all Full Stack code from the past 4 weeks. Pick the project you are least proud of.

.

2 Refactor it: improve naming, add comments, fix any errors, add error handling.

.

3 Commit refactored code with message: 'refactor: buffer week improvements'.

.

Output Refactored code committed. README updated with improvement notes.

:

Day 647	Wednesday	Catch Up
---------	-----------	----------

1 Finish any incomplete mini-projects or exercises from the past 4 weeks.

.

2 Push all pending code to GitHub. Ensure every week folder has a complete README.

.

3 Run LeetCode: solve 3 problems you struggled with before. Time yourself.

.

Output All pending work committed. LeetCode practice done.

:

Day 648	Thursday	Deep Dive
---------	----------	-----------

1 Pick the hardest concept from the past 4 weeks. Spend the full session going deeper on it.

.

2 Build a tiny demo project that specifically exercises that concept. No tutorials.

.

3 Push the demo to GitHub with a clear README explaining what you explored.

.

Output Deep dive demo project committed to GitHub.

:

Day 649	Friday	System Design
---------	--------	---------------

1 Study system design: read one chapter of system-design-primer (GitHub, free) or watch one ByteByteGo video.

.

2 Draw the architecture diagram for your phase project. Label every component and connection.

.

3 Write the architecture explanation in your README as if explaining it to a junior developer.

.

Output Architecture diagram drawn and explained in README.

:

Day 650

Saturday

Phase Project Prep

Buffer

1 Plan the upcoming phase project in detail: list all components, features, and acceptance criteria.

.

2 Set up the project repository structure. Create all folders and stub files.

.

3 Order any missing hardware components. Write the hardware shopping list in README.

.

Output Phase project repo created with full plan in README. Hardware ordered.

:

Day 651

Sunday

Rest & Plan

1 Rest fully. No coding, no circuits. Sustainable learning requires recovery.

.

2 Optional: review next week's topics for 15 min. Bookmark 1-2 key resources.

.

3 Write 3 clear goals for the next 4 weeks at the bottom of your buffer README.

.

Output Well rested. 3 goals for next phase written. Resources bookmarked.

:

Week 94

Electronics: Final IoT Product Build Wk1 | Full Stack: SaaS Dashboard Build Wk1

Day 652	Monday	Learn Electronics	Final
---------	--------	-------------------	-------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Final IoT Product Build Wk1'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'Final IoT Product Build Wk1'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-94/README.md with today's topic.
- Output** Notes page on 'Final IoT Product Build Wk1' + GitHub README stub created for Week 94.

Day 653	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'SaaS Dashboard Build Wk1'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'SaaS Dashboard Build Wk1'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 94 - Day 2 Learn: SaaS Dashboard Build Wk1'.
- Output** Working code examples for 'SaaS Dashboard Build Wk1' committed to GitHub.

Day 654	Wednesday	Practice Full Stack + Electronics	Final
---------	-----------	-----------------------------------	-------

- 1 Write 3 short coding exercises for 'SaaS Dashboard Build Wk1' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Final IoT Product Build Wk1' from memory.
- Output** 3 working code files for 'SaaS Dashboard Build Wk1' on GitHub. 'Final IoT Product Build Wk1' circuit sketched or simulated.

Day 655	Thursday	Build Electronics + Full Stack	Final
---------	----------	--------------------------------	-------

- 1 Electronics (45 min): Wire the full 'Final IoT Product Build Wk1' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'SaaS Dashboard Build Wk1' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'Final IoT Product Build Wk1' circuit (photographed) + 'SaaS Dashboard Build Wk1' feature committed to GitHub.

Day 656	Friday	Debug Electronics + Full Stack	Final
---------	--------	--------------------------------	-------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Final IoT Product Build Wk1' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'SaaS Dashboard Build Wk1' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output

Bug log in README. 'Final IoT Product Build Wk1' circuit verified. 'SaaS Dashboard Build Wk1' code with error handling on GitHub.

Day 657

Saturday

Project — Project Day

Final

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Final IoT Product Build Wk1' + 'SaaS Dashboard Build Wk1'.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 94 project milestone'.

Output

Working project milestone: 'Final IoT Product Build Wk1' data visible via 'SaaS Dashboard Build Wk1' layer. Pushed to GitHub.

Day 658

Sunday

Review — Review Day

Final

1

Electronics (20 min): Re-draw your 'Final IoT Product Build Wk1' circuit from memory without references. Compare to the real circuit. Note what you missed.

2

Full Stack (20 min): Explain 'SaaS Dashboard Build Wk1' out loud for 5 minutes (rubber duck method). Update your Week 94 README with a learning summary.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output

All Week 94 code committed. README updated with learnings and next-week goals.

Week 95

Electronics: Final IoT Product Build Wk2 | Full Stack: SaaS Dashboard Build Wk2

Day 659	Monday	Learn Electronics	Final
---------	--------	-------------------	-------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Final IoT Product Build Wk2'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'Final IoT Product Build Wk2'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-95/README.md with today's topic.
- Output** Notes page on 'Final IoT Product Build Wk2' + GitHub README stub created for Week 95.

Day 660	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'SaaS Dashboard Build Wk2'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'SaaS Dashboard Build Wk2'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 95 - Day 2 Learn: SaaS Dashboard Build Wk2'.
- Output** Working code examples for 'SaaS Dashboard Build Wk2' committed to GitHub.

Day 661	Wednesday	Practice Full Stack + Electronics	Final
---------	-----------	-----------------------------------	-------

- 1 Write 3 short coding exercises for 'SaaS Dashboard Build Wk2' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Final IoT Product Build Wk2' from memory.
- Output** 3 working code files for 'SaaS Dashboard Build Wk2' on GitHub. 'Final IoT Product Build Wk2' circuit sketched or simulated.

Day 662	Thursday	Build Electronics + Full Stack	Final
---------	----------	--------------------------------	-------

- 1 Electronics (45 min): Wire the full 'Final IoT Product Build Wk2' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'SaaS Dashboard Build Wk2' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'Final IoT Product Build Wk2' circuit (photographed) + 'SaaS Dashboard Build Wk2' feature committed to GitHub.

Day 663	Friday	Debug Electronics + Full Stack	Final
---------	--------	--------------------------------	-------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Final IoT Product Build Wk2' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'SaaS Dashboard Build Wk2' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output

Bug log in README. 'Final IoT Product Build Wk2' circuit verified. 'SaaS Dashboard Build Wk2' code with error handling on GitHub.

Day 664

Saturday

Project — Project Day

Final

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Final IoT Product Build Wk2' + 'SaaS Dashboard Build Wk2'.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 95 project milestone'.

Output

Working project milestone: 'Final IoT Product Build Wk2' data visible via 'SaaS Dashboard Build Wk2' layer. Pushed to GitHub.

Day 665

Sunday

Review — Review Day

Final

1

Electronics (20 min): Re-draw your 'Final IoT Product Build Wk2' circuit from memory without references. Compare to the real circuit. Note what you missed.

2

Full Stack (20 min): Explain 'SaaS Dashboard Build Wk2' out loud for 5 minutes (rubber duck method). Update your Week 95 README with a learning summary.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output

All Week 95 code committed. README updated with learnings and next-week goals.

Week 96

Electronics: Final IoT Product Build Wk3 | Full Stack: SaaS Dashboard Build Wk3

Day 666	Monday	Learn Electronics	Final
---------	--------	-------------------	-------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Final IoT Product Build Wk3'. Pause every 10 min and write key takeaways.
 - 2 Read allaboutcircuits.com for 'Final IoT Product Build Wk3'. Copy 3 key formulas or facts into your notes.
 - 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-96/README.md with today's topic.
- Output** Notes page on 'Final IoT Product Build Wk3' + GitHub README stub created for Week 96.

Day 667	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'SaaS Dashboard Build Wk3'. Code along in VS Code — don't just watch.
 - 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'SaaS Dashboard Build Wk3'. Run every code example locally.
 - 3 Commit your notes and code samples to GitHub: 'Week 96 - Day 2 Learn: SaaS Dashboard Build Wk3'.
- Output** Working code examples for 'SaaS Dashboard Build Wk3' committed to GitHub.

Day 668	Wednesday	Practice Full Stack + Electronics	Final
---------	-----------	-----------------------------------	-------

- 1 Write 3 short coding exercises for 'SaaS Dashboard Build Wk3' in VS Code. Each exercise tests one specific concept.
 - 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
 - 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Final IoT Product Build Wk3' from memory.
- Output** 3 working code files for 'SaaS Dashboard Build Wk3' on GitHub. 'Final IoT Product Build Wk3' circuit sketched or simulated.

Day 669	Thursday	Build Electronics + Full Stack	Final
---------	----------	--------------------------------	-------

- 1 Electronics (45 min): Wire the full 'Final IoT Product Build Wk3' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
 - 2 Full Stack (60 min): Build the 'SaaS Dashboard Build Wk3' feature inside your week's running project. Real, working code.
 - 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.
- Output** Working 'Final IoT Product Build Wk3' circuit (photographed) + 'SaaS Dashboard Build Wk3' feature committed to GitHub.

Day 670	Friday	Debug Electronics + Full Stack	Final
---------	--------	--------------------------------	-------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Final IoT Product Build Wk3' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'SaaS Dashboard Build Wk3' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output

Bug log in README. 'Final IoT Product Build Wk3' circuit verified. 'SaaS Dashboard Build Wk3' code with error handling on GitHub.

Day 671

Saturday

Project — Project Day

Final

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Final IoT Product Build Wk3' + 'SaaS Dashboard Build Wk3'.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 96 project milestone'.

Output

Working project milestone: 'Final IoT Product Build Wk3' data visible via 'SaaS Dashboard Build Wk3' layer. Pushed to GitHub.

Day 672

Sunday

Review — Review Day

Final

1

Electronics (20 min): Re-draw your 'Final IoT Product Build Wk3' circuit from memory without references. Compare to the real circuit. Note what you missed.

2

Full Stack (20 min): Explain 'SaaS Dashboard Build Wk3' out loud for 5 minutes (rubber duck method). Update your Week 96 README with a learning summary.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output

All Week 96 code committed. README updated with learnings and next-week goals.

Week 97

Electronics: Final IoT Product Testing | Full Stack: SaaS Dashboard Testing

Day 673	Monday	Learn Electronics	Final
---------	--------	-------------------	-------

1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Final IoT Product Testing'. Pause every 10 min and write key takeaways.

2 Read allaboutcircuits.com for 'Final IoT Product Testing'. Copy 3 key formulas or facts into your notes.

3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-97/README.md with today's topic.

Output Notes page on 'Final IoT Product Testing' + GitHub README stub created for Week 97.

:

Day 674	Tuesday	Learn Full Stack
---------	---------	------------------

1 Watch 30-45 min: Traversy Media or The Net Ninja on 'SaaS Dashboard Testing'. Code along in VS Code — don't just watch.

2 Read the official docs (MDN / React Docs / Node.js Docs) for 'SaaS Dashboard Testing'. Run every code example locally.

3 Commit your notes and code samples to GitHub: 'Week 97 - Day 2 Learn: SaaS Dashboard Testing'.

Output Working code examples for 'SaaS Dashboard Testing' committed to GitHub.

:

Day 675	Wednesday	Practice Full Stack + Electronics	Final
---------	-----------	-----------------------------------	-------

1 Write 3 short coding exercises for 'SaaS Dashboard Testing' in VS Code. Each exercise tests one specific concept.

2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.

3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Final IoT Product Testing' from memory.

Output 3 working code files for 'SaaS Dashboard Testing' on GitHub. 'Final IoT Product Testing' circuit sketched or simulated.

:

Day 676	Thursday	Build Electronics + Full Stack	Final
---------	----------	--------------------------------	-------

1 Electronics (45 min): Wire the full 'Final IoT Product Testing' circuit on your breadboard. Test with multimeter. Troubleshoot until working.

2 Full Stack (60 min): Build the 'SaaS Dashboard Testing' feature inside your week's running project. Real, working code.

3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Final IoT Product Testing' circuit (photographed) + 'SaaS Dashboard Testing' feature committed to GitHub.

:

Day 677	Friday	Debug Electronics + Full Stack	Final
---------	--------	--------------------------------	-------

1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Final IoT Product Testing' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'SaaS Dashboard Testing' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Final IoT Product Testing' circuit verified. 'SaaS Dashboard Testing' code with error handling on GitHub.

Day 678

Saturday

Project — Project Day

Final

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Final IoT Product Testing' + 'SaaS Dashboard Testing'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 97 project milestone'.

.

Output

Working project milestone: 'Final IoT Product Testing' data visible via 'SaaS Dashboard Testing' layer. Pushed to GitHub.

:

Day 679

Sunday

Review — Review Day

Final

1

Electronics (20 min): Re-draw your 'Final IoT Product Testing' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'SaaS Dashboard Testing' out loud for 5 minutes (rubber duck method). Update your Week 97 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 97 code committed. README updated with learnings and next-week goals.

:

Week 98

Electronics: Final IoT Product Polish | Full Stack: SaaS Dashboard Polish

Day 680	Monday	Learn Electronics	Final
---------	--------	-------------------	-------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Final IoT Product Polish'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Final IoT Product Polish'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-98/README.md with today's topic.

Output Notes page on 'Final IoT Product Polish' + GitHub README stub created for Week 98.

Day 681	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'SaaS Dashboard Polish'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'SaaS Dashboard Polish'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 98 - Day 2 Learn: SaaS Dashboard Polish'.

Output Working code examples for 'SaaS Dashboard Polish' committed to GitHub.

Day 682	Wednesday	Practice Full Stack + Electronics	Final
---------	-----------	-----------------------------------	-------

- 1 Write 3 short coding exercises for 'SaaS Dashboard Polish' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Final IoT Product Polish' from memory.

Output 3 working code files for 'SaaS Dashboard Polish' on GitHub. 'Final IoT Product Polish' circuit sketched or simulated.

Day 683	Thursday	Build Electronics + Full Stack	Final
---------	----------	--------------------------------	-------

- 1 Electronics (45 min): Wire the full 'Final IoT Product Polish' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'SaaS Dashboard Polish' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Final IoT Product Polish' circuit (photographed) + 'SaaS Dashboard Polish' feature committed to GitHub.

Day 684	Friday	Debug Electronics + Full Stack	Final
---------	--------	--------------------------------	-------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Final IoT Product Polish' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'SaaS Dashboard Polish' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Final IoT Product Polish' circuit verified. 'SaaS Dashboard Polish' code with error handling on GitHub.

:

Day 685

Saturday

Project — Project Day

Final

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Final IoT Product Polish' + 'SaaS Dashboard Polish'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 98 project milestone'.

.

Output

Working project milestone: 'Final IoT Product Polish' data visible via 'SaaS Dashboard Polish' layer. Pushed to GitHub.

:

Day 686

Sunday

Review — Review Day

Final

1

Electronics (20 min): Re-draw your 'Final IoT Product Polish' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'SaaS Dashboard Polish' out loud for 5 minutes (rubber duck method). Update your Week 98 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 98 code committed. README updated with learnings and next-week goals.

:

Week 99

Electronics: Final IoT Product Docs | Full Stack: SaaS Dashboard Docs & API

Day 687	Monday	Learn Electronics	Final
---------	--------	-------------------	-------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Final IoT Product Docs'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Final IoT Product Docs'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-99/README.md with today's topic.

Output Notes page on 'Final IoT Product Docs' + GitHub README stub created for Week 99.

Day 688	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'SaaS Dashboard Docs & API'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'SaaS Dashboard Docs & API'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 99 - Day 2 Learn: SaaS Dashboard Docs & API'.

Output Working code examples for 'SaaS Dashboard Docs & API' committed to GitHub.

Day 689	Wednesday	Practice Full Stack + Electronics	Final
---------	-----------	-----------------------------------	-------

- 1 Write 3 short coding exercises for 'SaaS Dashboard Docs & API' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Final IoT Product Docs' from memory.

Output 3 working code files for 'SaaS Dashboard Docs & API' on GitHub. 'Final IoT Product Docs' circuit sketched or simulated.

Day 690	Thursday	Build Electronics + Full Stack	Final
---------	----------	--------------------------------	-------

- 1 Electronics (45 min): Wire the full 'Final IoT Product Docs' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'SaaS Dashboard Docs & API' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Final IoT Product Docs' circuit (photographed) + 'SaaS Dashboard Docs & API' feature committed to GitHub.

Day 691	Friday	Debug Electronics + Full Stack	Final
---------	--------	--------------------------------	-------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Final IoT Product Docs' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'SaaS Dashboard Docs & API' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Final IoT Product Docs' circuit verified. 'SaaS Dashboard Docs & API' code with error handling on GitHub.

Day 692

Saturday

Project — Project Day

Final

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Final IoT Product Docs' + 'SaaS Dashboard Docs & API'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 99 project milestone'.

.

Output

Working project milestone: 'Final IoT Product Docs' data visible via 'SaaS Dashboard Docs & API' layer. Pushed to GitHub.

Day 693

Sunday

Review — Review Day

Final

1

Electronics (20 min): Re-draw your 'Final IoT Product Docs' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'SaaS Dashboard Docs & API' out loud for 5 minutes (rubber duck method). Update your Week 99 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 99 code committed. README updated with learnings and next-week goals.

Week 100

Electronics: Final IoT Product Launch | Full Stack: Production Deployment & Go-live

MILESTONE PROJECT: Final IoT SaaS Product — Custom PCB + MQTT + Next.js + Stripe + Docker

Day 694	Monday	Plan & Design	
1	Design the full project architecture: draw the hardware circuit and software stack on paper.		
.			
2	List all required components for 'Final IoT Product Launch' and 'Production Deployment & Go-live'. Verify you have everything.		
.			
3	Set up the project GitHub repo. Create README with: project goal, tech stack, architecture diagram placeholder.		
.			
Output	Architecture designed. Project repo created. README outline committed.		
:			
Day 695	Tuesday	Build Core	Final
1	Build the core hardware component: 'Final IoT Product Launch'. Test it works in isolation with multimeter.		
.			
2	Set up the core software scaffold: 'Production Deployment & Go-live'. Create all files and folder structure.		
.			
3	Connect hardware and software at the most basic level. Verify data flows correctly.		
.			
Output	Core 'Final IoT Product Launch' hardware working. 'Production Deployment & Go-live' scaffold committed to GitHub.		
:			
Day 696	Wednesday	Build Features	Final
1	Implement the main feature of 'Final IoT Product Launch'. Document every step in README.		
.			
2	Build the primary user-facing feature of 'Production Deployment & Go-live'. Write clean, commented code.		
.			
3	Integration check: hardware + software + web layer all communicating correctly.		
.			
Output	Main feature built and integrated. Code committed with documentation.		
:			
Day 697	Thursday	Build Full	Final
1	Complete all remaining features for 'Final IoT Product Launch'. Photograph every stage.		
.			
2	Complete all remaining features for 'Production Deployment & Go-live'. Add proper error handling.		
.			
3	End-to-end test: full pipeline from hardware input to web output. Fix all issues.		
.			
Output	Full project working end-to-end. All code committed to GitHub.		
:			
Day 698	Friday	Debug & Polish	Final

- 1 Stress test the project: run it for 30 minutes continuously. Log any failures.
.
- 2 Fix all bugs found. Add input validation and error messages. Improve UX.
.
- 3 Code review: clean up all magic numbers, add comments, ensure consistent naming.
.

Output Project stable under load. All bugs fixed. Code cleaned and committed.

:

Day 699 Saturday Document & Demo

- 1 Write comprehensive README: project overview, setup guide, usage instructions, photos.
.
- 2 Record a 2-minute demo video showing the full working project.
.
- 3 Add the project to your portfolio page. Write a 100-word project description.
.

Output README complete. Demo video recorded. Portfolio updated.

:

Day 700 Sunday Launch & Reflect

- 1 Final commit with version tag v1.0.0. Push everything to GitHub.
.
- 2 Write a 200-word reflection: what you built, what you learned, what you'd do differently.
.
- 3 Share the GitHub link with someone (social media, forum, friend). External accountability matters.
.

Output Project v1.0.0 released. Reflection written. GitHub link shared.

:

Week 101

Electronics: Portfolio Finalise Wk1 | Full Stack: Portfolio Finalise Wk1

Day 701	Monday	Learn Electronics	Portfolio
---------	--------	-------------------	-----------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Portfolio Finalise Wk1'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Portfolio Finalise Wk1'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-101/README.md with today's topic.

Output Notes page on 'Portfolio Finalise Wk1' + GitHub README stub created for Week 101.

Day 702	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Portfolio Finalise Wk1'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Portfolio Finalise Wk1'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 101 - Day 2 Learn: Portfolio Finalise Wk1'.

Output Working code examples for 'Portfolio Finalise Wk1' committed to GitHub.

Day 703	Wednesday	Practice Full Stack + Electronics	Portfolio
---------	-----------	-----------------------------------	-----------

- 1 Write 3 short coding exercises for 'Portfolio Finalise Wk1' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Portfolio Finalise Wk1' from memory.

Output 3 working code files for 'Portfolio Finalise Wk1' on GitHub. 'Portfolio Finalise Wk1' circuit sketched or simulated.

Day 704	Thursday	Build Electronics + Full Stack	Portfolio
---------	----------	--------------------------------	-----------

- 1 Electronics (45 min): Wire the full 'Portfolio Finalise Wk1' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Portfolio Finalise Wk1' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Portfolio Finalise Wk1' circuit (photographed) + 'Portfolio Finalise Wk1' feature committed to GitHub.

Day 705	Friday	Debug Electronics + Full Stack	Portfolio
---------	--------	--------------------------------	-----------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Portfolio Finalise Wk1' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Portfolio Finalise Wk1' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Portfolio Finalise Wk1' circuit verified. 'Portfolio Finalise Wk1' code with error handling on GitHub.

:

Day 706

Saturday

Project — Project Day

Portfolio

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Portfolio Finalise Wk1' + 'Portfolio Finalise Wk1'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 101 project milestone'.

.

Output

Working project milestone: 'Portfolio Finalise Wk1' data visible via 'Portfolio Finalise Wk1' layer. Pushed to GitHub.

:

Day 707

Sunday

Review — Review Day

Portfolio

1

Electronics (20 min): Re-draw your 'Portfolio Finalise Wk1' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Portfolio Finalise Wk1' out loud for 5 minutes (rubber duck method). Update your Week 101 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 101 code committed. README updated with learnings and next-week goals.

:

Week 102

Electronics: Portfolio Finalise Wk2 | Full Stack: Portfolio Finalise Wk2

Day 708	Monday	Learn Electronics	Portfolio
---------	--------	-------------------	-----------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Portfolio Finalise Wk2'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Portfolio Finalise Wk2'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-102/README.md with today's topic.

Output Notes page on 'Portfolio Finalise Wk2' + GitHub README stub created for Week 102.

Day 709	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Portfolio Finalise Wk2'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Portfolio Finalise Wk2'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 102 - Day 2 Learn: Portfolio Finalise Wk2'.

Output Working code examples for 'Portfolio Finalise Wk2' committed to GitHub.

Day 710	Wednesday	Practice Full Stack + Electronics	Portfolio
---------	-----------	-----------------------------------	-----------

- 1 Write 3 short coding exercises for 'Portfolio Finalise Wk2' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Portfolio Finalise Wk2' from memory.

Output 3 working code files for 'Portfolio Finalise Wk2' on GitHub. 'Portfolio Finalise Wk2' circuit sketched or simulated.

Day 711	Thursday	Build Electronics + Full Stack	Portfolio
---------	----------	--------------------------------	-----------

- 1 Electronics (45 min): Wire the full 'Portfolio Finalise Wk2' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Portfolio Finalise Wk2' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Portfolio Finalise Wk2' circuit (photographed) + 'Portfolio Finalise Wk2' feature committed to GitHub.

Day 712	Friday	Debug Electronics + Full Stack	Portfolio
---------	--------	--------------------------------	-----------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Portfolio Finalise Wk2' circuit. Fix each one. Document them.

2

Full Stack (45 min): Review your 'Portfolio Finalise Wk2' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

.

3

Write a short bug log in your README: what broke, what caused it, how you fixed it.

.

Output

Bug log in README. 'Portfolio Finalise Wk2' circuit verified. 'Portfolio Finalise Wk2' code with error handling on GitHub.

:

Day 713

Saturday

Project — Project Day

Portfolio

1

Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Portfolio Finalise Wk2' + 'Portfolio Finalise Wk2'.

.

2

Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

.

3

Test the full flow. Fix any issues. Push to GitHub: 'Week 102 project milestone'.

.

Output

Working project milestone: 'Portfolio Finalise Wk2' data visible via 'Portfolio Finalise Wk2' layer. Pushed to GitHub.

:

Day 714

Sunday

Review — Review Day

Portfolio

1

Electronics (20 min): Re-draw your 'Portfolio Finalise Wk2' circuit from memory without references. Compare to the real circuit. Note what you missed.

.

2

Full Stack (20 min): Explain 'Portfolio Finalise Wk2' out loud for 5 minutes (rubber duck method). Update your Week 102 README with a learning summary.

.

3

GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

.

Output

All Week 102 code committed. README updated with learnings and next-week goals.

:

Week 103

Electronics: Career Prep & Job Apps Wk1 | Full Stack: Career Prep & Job Apps Wk1

Day 715	Monday	Learn Electronics	Career
---------	--------	-------------------	--------

- 1 Watch 30-45 min: GreatScott or Andreas Spiess on 'Career Prep & Job Apps Wk1'. Pause every 10 min and write key takeaways.
- 2 Read allaboutcircuits.com for 'Career Prep & Job Apps Wk1'. Copy 3 key formulas or facts into your notes.
- 3 Open GitHub. Create/update your 'iot-roadmap' repo. Add week-103/README.md with today's topic.

Output Notes page on 'Career Prep & Job Apps Wk1' + GitHub README stub created for Week 103.

Day 716	Tuesday	Learn Full Stack
---------	---------	------------------

- 1 Watch 30-45 min: Traversy Media or The Net Ninja on 'Career Prep & Job Apps Wk1'. Code along in VS Code — don't just watch.
- 2 Read the official docs (MDN / React Docs / Node.js Docs) for 'Career Prep & Job Apps Wk1'. Run every code example locally.
- 3 Commit your notes and code samples to GitHub: 'Week 103 - Day 2 Learn: Career Prep & Job Apps Wk1'.

Output Working code examples for 'Career Prep & Job Apps Wk1' committed to GitHub.

Day 717	Wednesday	Practice Full Stack + Electronics	Career
---------	-----------	-----------------------------------	--------

- 1 Write 3 short coding exercises for 'Career Prep & Job Apps Wk1' in VS Code. Each exercise tests one specific concept.
- 2 Add a comment to each exercise explaining what it does. Push all 3 to GitHub.
- 3 Spend 20 min on breadboard/Tinkercad: attempt the basic circuit for 'Career Prep & Job Apps Wk1' from memory.

Output 3 working code files for 'Career Prep & Job Apps Wk1' on GitHub. 'Career Prep & Job Apps Wk1' circuit sketched or simulated.

Day 718	Thursday	Build Electronics + Full Stack	Career
---------	----------	--------------------------------	--------

- 1 Electronics (45 min): Wire the full 'Career Prep & Job Apps Wk1' circuit on your breadboard. Test with multimeter. Troubleshoot until working.
- 2 Full Stack (60 min): Build the 'Career Prep & Job Apps Wk1' feature inside your week's running project. Real, working code.
- 3 Take a photo of your circuit. Write 2-3 sentences describing how it works in your README.

Output Working 'Career Prep & Job Apps Wk1' circuit (photographed) + 'Career Prep & Job Apps Wk1' feature committed to GitHub.

Day 719	Friday	Debug Electronics + Full Stack	Career
---------	--------	--------------------------------	--------

- 1 Electronics (30 min): Find and intentionally introduce 3 common mistakes in your 'Career Prep & Job Apps Wk1' circuit. Fix each one. Document them.

2 Full Stack (45 min): Review your 'Career Prep & Job Apps Wk1' code. Add try/catch or conditional error handling. Add 3 console.log statements tracing data flow.

3 Write a short bug log in your README: what broke, what caused it, how you fixed it.

Output Bug log in README. 'Career Prep & Job Apps Wk1' circuit verified. 'Career Prep & Job Apps Wk1' code with error handling on GitHub.

Day 720

Saturday

Project — Project Day

Career

1 Spend 10 min planning: what does today's integration need? Define the simplest version that connects 'Career Prep & Job Apps Wk1' + 'Career Prep & Job Apps Wk1'.

2 Build the integration: wire your hardware output to your web frontend (even hardcoded data is fine to start).

3 Test the full flow. Fix any issues. Push to GitHub: 'Week 103 project milestone'.

Output Working project milestone: 'Career Prep & Job Apps Wk1' data visible via 'Career Prep & Job Apps Wk1' layer. Pushed to GitHub.

Day 721

Sunday

Review — Review Day

Career

1 Electronics (20 min): Re-draw your 'Career Prep & Job Apps Wk1' circuit from memory without references. Compare to the real circuit. Note what you missed.

2 Full Stack (20 min): Explain 'Career Prep & Job Apps Wk1' out loud for 5 minutes (rubber duck method). Update your Week 103 README with a learning summary.

3 GitHub wrap-up (15 min): Ensure all code is committed. Write 3 specific goals for next week at the bottom of the README.

Output All Week 103 code committed. README updated with learnings and next-week goals.

Week 104

Electronics: Career Prep & Job Apps Wk2 | Full Stack: Career Prep — Day 730 Complete

MILESTONE PROJECT: Day 730 — Journey Complete: Portfolio Live, Product Deployed, Job Ready

Day 722	Monday	Plan & Design	
1	Design the full project architecture: draw the hardware circuit and software stack on paper.		
.			
2	List all required components for 'Career Prep & Job Apps Wk2' and 'Career Prep — Day 730 Complete'. Verify you have everything.		
.			
3	Set up the project GitHub repo. Create README with: project goal, tech stack, architecture diagram placeholder.		
.			
Output	Architecture designed. Project repo created. README outline committed.		
:			
Day 723	Tuesday	Build Core	Career
1	Build the core hardware component: 'Career Prep & Job Apps Wk2'. Test it works in isolation with multimeter.		
.			
2	Set up the core software scaffold: 'Career Prep — Day 730 Complete'. Create all files and folder structure.		
.			
3	Connect hardware and software at the most basic level. Verify data flows correctly.		
.			
Output	Core 'Career Prep & Job Apps Wk2' hardware working. 'Career Prep — Day 730 Complete' scaffold committed to GitHub.		
:			
Day 724	Wednesday	Build Features	Career
1	Implement the main feature of 'Career Prep & Job Apps Wk2'. Document every step in README.		
.			
2	Build the primary user-facing feature of 'Career Prep — Day 730 Complete'. Write clean, commented code.		
.			
3	Integration check: hardware + software + web layer all communicating correctly.		
.			
Output	Main feature built and integrated. Code committed with documentation.		
:			
Day 725	Thursday	Build Full	Career
1	Complete all remaining features for 'Career Prep & Job Apps Wk2'. Photograph every stage.		
.			
2	Complete all remaining features for 'Career Prep — Day 730 Complete'. Add proper error handling.		
.			
3	End-to-end test: full pipeline from hardware input to web output. Fix all issues.		
.			
Output	Full project working end-to-end. All code committed to GitHub.		
:			
Day 726	Friday	Debug & Polish	Career

- 1 Stress test the project: run it for 30 minutes continuously. Log any failures.
.
- 2 Fix all bugs found. Add input validation and error messages. Improve UX.
.
- 3 Code review: clean up all magic numbers, add comments, ensure consistent naming.
.

Output Project stable under load. All bugs fixed. Code cleaned and committed.

:

Day 727 Saturday Document & Demo

- 1 Write comprehensive README: project overview, setup guide, usage instructions, photos.
.
- 2 Record a 2-minute demo video showing the full working project.
.
- 3 Add the project to your portfolio page. Write a 100-word project description.
.

Output README complete. Demo video recorded. Portfolio updated.

:

Day 728 Sunday Launch & Reflect

- 1 Final commit with version tag v1.0.0. Push everything to GitHub.
.
- 2 Write a 200-word reflection: what you built, what you learned, what you'd do differently.
.
- 3 Share the GitHub link with someone (social media, forum, friend). External accountability matters.
.

Output Project v1.0.0 released. Reflection written. GitHub link shared.

: